

3.19美团面试

1. 面试题

Q1：开发过程中遇到了哪些难题？

绘制百万级数据的折线图

使用 ECharts 绘制百万数据点的折线图和条形图时，通过以下优化提高性能：

1. **数据采样**：采用固定步长的取点方式减少数据量。
2. **大数据模式**：启用 `large: true` 和 `largeThreshold` 配置，优化渲染大数据集。
3. **渐进式渲染**：配置 `progressive` 和 `progressiveThreshold`，分批加载数据点。
4. **禁用不必要的功能**：关闭 `tooltip`、动画，简化图形样式。
5. **使用 `canvas` 渲染**：借助 GPU 加速，提高渲染效率。

1.数据采样（减少数据点，提高渲染速度）

通过 `step` 采样减少数据量，提升渲染速度

2.大数据量/渐进式渲染

大数据模式： `large:true;largeThreshold:10000`

大于10000开启大数据模式（默认2000）

渐进式渲染： `progressive:5000,progressiveThreshold:10000`

避免 UI 阻塞，保证流畅度。**每次渲染 5000 个数据点**，逐步绘制，不会一次性占满主线程。**数据量超过 10000 时，启用渐进式渲染。**

3.样式优化

禁用 `tooltip` 提示（避免频繁计算）

关闭动画（避免高计算量）

优化线条

- 细一点，降低 `linewidth`
- 透明度降低，减少重绘复杂度
- 关闭 `symbol`（默认的小圆点）

4.使用dataZoom

初始只加载部分数据，提高首屏速度，滚动时加载更多

5.使用Canvas渲染

`canvas` 直接修改 **像素数据**，避免了浏览器对每个图形进行 **几何计算、重排 (Reflow)** 和 **重绘 (Repaint)**

基于位图，直接在 **像素级** 进行绘制，一旦绘制后，图形不会单独存在于 DOM 树中。

Canvas 只是一个画布，**所有绘制逻辑在 JS 内部执行**，减少了浏览器的 **DOM 操作开销**。

`canvas` 依赖 **WebGL (GPU加速)** 时，能进一步加快渲染。

```
//适用于超大量的

var option = {
  tooltip: {
    trigger: 'axis',
    throttle: 100, // 限制触发频率 (100ms)
    renderMode: 'html' // 使用 HTML 渲染 tooltip, 提高性能
  },
  xAxis: {
    type: 'category',
    data: downsampleData(largeXAxisData)
  },
  yAxis: {
    type: 'value'
  },
  dataZoom: [
    { type: 'inside' }, // 支持缩放
    { type: 'slider' }
  ],
  series: Array.from({ length: 200 }, (_, i) => ({
    type: 'line',
    data: downsampleData(largeSeriesData[i]),
    large: true, // 开启大数据优化
    largeThreshold: 4000, // 当数据点大于 4000 时自动启用
    symbol: 'none', // 关闭折线点, 减少绘制消耗
    progressive: 500, // 每次渲染 500 个点, 提高流畅度
    progressiveThreshold: 3000, // 仅在数据量超过 3000 时启用
    smooth: false, // 关闭平滑曲线, 提高渲染速度
    linestyle: {
      opacity: 0.8, // 降低透明度, 提高性能
      width: 0.5 // 细线降低渲染压力
    }
  })))
};

chart.setOption(option);
```

Q2: 在地图上做了哪些工作?

图层调用，识别点和弹窗，弹窗位置的改变

Q3：虚拟列表的实现？（使用了组件）

一次请求完所有数据，只渲染部分元素

当数据量过大（如上千条数据），直接渲染整个列表会导致 **DOM 过多**，**页面卡顿**，影响性能。Vue 默认的 `v-for` 会渲染所有数据，导致 **长列表性能问题**，所以需要进行优化。

虚拟列表 (Virtual List) 是一种**优化大数据量渲染**的方法，主要用于前端长列表（如数千行数据）的渲染优化。其核心思想是“**只渲染可见区域的元素**”，而未显示的部分不渲染，**从而减少 DOM 节点数量**，提高页面性能。

实现方式：

- **计算可视区域 (viewport)**：通过 `scrollTop` 确定当前滚动位置。
- 计算需要渲染的元素范围：
 - 假设 **列表高度 10000px**，单个 `item` 高度 `50px`，那么**一共 200 个元素**。
 - 但视口 (**viewport**) 高度为 `500px`，最多只能显示 `500 / 50 = 10` 个元素。
 - **所以每次只渲染 10~20 个元素**，其他的用占位 (`padding/margin`)。
- 动态加载新数据：
 - 监听 `scroll` 事件，滚动时计算当前应该显示的 `item`，并更新 `startIndex` 和 `endIndex` 进行**数据更新**。

Q4：响应式原理

1. Vue2 的响应式原理

Vue2 的响应式系统基于 `Object.defineProperty()`，通过**数据劫持和发布-订阅模式**实现双向数据绑定。其核心包括 **数据劫持 (Observer)**、**依赖收集 (Dep)**、**派发更新 (Watcher)** 等关键步骤。

1. 数据劫持 (Observer)

Vue2 通过 `Object.defineProperty()` 劫持对象属性的 **getter** 和 **setter** 方法，实现对数据的监听和依赖收集。

- **初始化时**，Vue 会遍历 `data` 对象的所有属性，使用 `Object.defineProperty()` 为每个属性定义 `getter` 和 `setter` 方法。
- **当访问数据时**，`getter` 触发 **依赖收集 (Dep)**。
- **当修改数据时**，`setter` 触发 **派发更新 (Watcher)**。

```
function defineReactive(obj, key, val) {
  Object.defineProperty(obj, key, {
    get() {
      console.log(`获取 ${key} 值:`, val);
      return val;
    },
    set(newVal) {
      console.log(`设置 ${key} 值:`, newVal);
      val = newVal;
    }
  });
}

let data = {};
defineReactive(data, "message", "Hello Vue2");
```

```
console.log(data.message); // 触发 getter
data.message = "Hello world"; // 触发 setter
```

2. 依赖收集 (Dep)

Vue 通过 Dep 依赖收集，将响应式数据和依赖 (Watcher) 关联。

- Dep 是一个依赖管理器，维护一个 subs 数组存储所有订阅该数据的 watcher。
- 当数据被访问 (getter 触发) 时，当前的 watcher (渲染函数或计算属性) 会被收集到 Dep 中。
- 当数据更新 (setter 触发) 时，会通知所有 watcher 进行更新。

```
class Dep {
  constructor() {
    this.subs = []; // 存储 watcher
  }
  addSub(watcher) {
    this.subs.push(watcher);
  }
  notify() {
    this.subs.forEach(watcher => watcher.update());
  }
}
```

3. 派发更新 (Watcher)

Watcher 作为观察者，负责监听数据变化并更新视图。

- 渲染 Watcher：用于触发组件的重新渲染。
- 计算属性 Watcher：用于计算属性缓存和依赖管理。
- 侦听器 Watcher：用于 \$watch 监听数据变化。

Watcher 的主要作用：

1. 初始化时 触发 getter，收集依赖 (Dep)。
2. 当数据变化 时，被 Dep 通知，执行 update() 触发视图更新。

```
class Watcher {
  constructor(updateFn) {
    this.updateFn = updateFn;
  }
  update() {
    this.updateFn();
  }
}

// 创建一个 Dep 实例
let dep = new Dep();
let watcher = new Watcher(() => console.log("视图更新!"));

// 依赖收集
dep.addSub(watcher);

// 数据变化时触发通知
```

```
dep.notify();
```

4. Vue2 响应式的实现流程

1. **初始化 Vue 实例**：调用 `observe(data)` 递归处理 `data` 对象，使其变成响应式对象。
2. **数据劫持**：使用 `Object.defineProperty()` 定义 `getter` 和 `setter`。
3. 依赖收集：
 - 访问数据时触发 `getter`，将 `watcher` 收集到 `Dep` 中。
 - `Dep` 维护所有依赖该数据的 `watcher`。
4. 数据变更触发更新：
 - 设置数据时触发 `setter`，`Dep` 通知所有 `watcher` 进行视图更新。

5. Vue2 响应式的局限性

1. 无法监听数组的索引变化（例如 `arr[1] = 10` 不能触发视图更新，原因：**数组的索引是动态的**，而 `defineProperty()` 只能劫持**已有的属性**，无法监听新添加的索引。Vue 2 仅对 **数组的方法**（`push`、`pop`、`shift`、`unshift`、`splice`、`sort`、`reverse`）进行了重写以触发视图更新，但**直接修改索引不会触发 setter**，因此不会引起视图更新。）。
 - Vue2 通过重写数组方法（如 `push()`、`pop()`）解决大部分情况。
2. 无法监听新增属性（如 `obj.newProp = 123`）
 - 需要使用 `$set(obj, 'newProp', 123)` 让 Vue 监听新属性。
3. 深层次数据仍需递归监听，导致性能开销大。
 - Vue3 通过 `Proxy` 解决了这些问题。

6. 总结

- Vue2 使用 `Object.defineProperty()` 实现数据劫持，使 `data` 变成响应式对象。
- **依赖收集 (Dep)** 维护所有依赖数据的 `watcher`，在 `getter` 触发时收集依赖。
- **派发更新 (Watcher)** 监听数据变化，在 `setter` 触发时通知更新。
- 存在 **无法监听新增属性、数组索引变更** 等问题，Vue3 用 `Proxy` 替代 `Object.defineProperty()` 进行了优化。

2. Vue3 的响应式原理（基于 Proxy）

Vue3 使用 `Proxy` 取代 Vue2 的 `Object.defineProperty()`，从而克服 Vue2 的局限性，并提升性能。Vue3 响应式系统的核心包括 **响应式对象 (Reactive)**、**依赖收集 (Effect)**、**追踪与触发 (Track & Trigger)**，基于 **Proxy + Reflect** 实现 **全对象劫持**，提高响应式能力和优化性能。

1. 响应式对象 (Reactive 与 Ref)

Vue3 主要通过 `reactive()` 和 `ref()` 使数据变成响应式：

- `reactive()` 用于创建 **深度响应式对象**。
- `ref()` 用于创建 **基本类型和对象的响应式包装**。

```
import { reactive, ref } from 'vue';

const obj = reactive({ count: 0 });
const number = ref(10);

console.log(obj.count); // 触发 getter
obj.count++;           // 触发 setter, 通知更新

console.log(number.value); // ref 需要 .value 访问
number.value = 20;        // 触发 setter
```

`reactive()` 适用于对象, `ref()` 适用于基本数据类型, 且 `ref` 内部也是基于 `reactive` 实现的。

Proxy 与 Reflect

Vue3 通过 `Proxy` 劫持整个对象, 而非 Vue2 的逐个属性监听

```
const obj = new Proxy(
  { count: 0 },
  {
    get(target, key) {
      console.log(`获取 ${key}:`, target[key]);
      return Reflect.get(target, key);
    },
    set(target, key, value) {
      console.log(`设置 ${key}:`, value);
      return Reflect.set(target, key, value);
    }
  }
);

console.log(obj.count); // 触发 getter
obj.count = 10;        // 触发 setter
```

`Reflect` 提供默认操作, 确保 `Proxy` 行为符合预期。

2. 依赖收集 (Effect & Track)

Vue3 使用 `Proxy` 代理 `reactive()` 或 `ref()` 创建的响应式数据, 并在 `getter` 中进行依赖收集 (`track`), 在 `setter` 中进行派发更新 (`trigger`)。

```
import { reactive, effect } from "vue";

const state = reactive({ count: 0 });

effect(() => {
  console.log("Effect 执行, count:", state.count);
});

state.count++; // 修改数据
```

当 `state.count++` 触发更新时, 执行顺序如下:

(1) 触发 `getter` (读取数据)

- `effect()` 首次运行时, 读取 `state.count`, 触发 `getter`, 进行依赖收集。
- `track()` 记录当前 `effect` (副作用函数)。

(2) 触发 `effect` (收集依赖)

- `track()` 将 `effect` 存入 `Dep` (依赖集合)。
- 这样后续数据变更时, 能通知 `effect` 重新执行。

(3) 触发 `setter` (修改数据)

- `state.count++` 触发 `setter`, 修改 `state.count` 值。
- `trigger()` 负责通知所有 `effect` 进行更新。

(4) 执行 `trigger` (通知更新)

- `trigger()` 遍历 `Dep`, 找到所有依赖 `state.count` 的 `effect`, 并执行它们。

(5) 视图更新 (Effect 重新执行)

- `effect()` 重新运行, 获取新的 `state.count` 值, 并重新渲染视图。

步骤	触发事件	具体行为
1	访问响应式数据 (<code>state.count</code>)	触发 <code>getter</code> , 调用 <code>track()</code> 进行依赖收集
2	<code>effect()</code> 初始化	读取 <code>state.count</code> , 执行 <code>console.log()</code>
3	修改数据 (<code>state.count++</code>)	触发 <code>setter</code> , 调用 <code>trigger()</code> 通知更新
4	<code>trigger()</code> 遍历依赖	依次执行所有 <code>effect()</code>
5	重新执行 <code>effect()</code>	读取 <code>state.count</code> , 触发 <code>getter</code> , 更新视图

3. 区别

特性	Vue2 (Object.defineProperty)	Vue3 (Proxy)
监听方式	逐个属性劫持	整个对象劫持
监听新增属性	不能监听 (需 <code>\$set</code>)	可以监听
监听数组索引	不能监听	可以监听
支持数据结构	仅支持对象和数组	支持 <code>Map</code> 、 <code>Set</code> 等
性能	递归遍历对象, 初始化开销大	代理整个对象, 性能更优
源码复杂度	需要 <code>Dep</code> 、 <code>Watcher</code> 进行依赖管理	通过 <code>Proxy</code> 简化

Q5: vue3怎么进行数据的响应式更新?

Vue3 响应式系统的核心实现:

Vue3 通过 **Proxy 代理**实现数据响应式, 其运作机制可分为三个关键环节:

1. 代理劫持

使用 `Proxy` 拦截对象操作（`get/set/deleteProperty` 等），在访问数据时自动收集依赖（如组件的渲染函数），在修改数据时触发更新。相较于 Vue2 的 `Object.defineProperty`，`Proxy` 能检测属性增删、数组索引修改及支持 `Map/Set` 等数据结构。

2. 依赖管理

- **Track (追踪)**：通过 `effect` 函数注册副作用（如视图渲染），数据被访问时记录依赖关系。
- **Trigger (触发)**：数据变更时，根据依赖关系精准执行关联副作用，实现组件更新或计算属性重算。

3. 分层 API 设计

- `reactive()`：深度代理对象/数组，嵌套属性自动响应化。
- `ref()`：包装基本类型（通过 `.value` 访问），解决值类型响应问题。
- `computed/watch`：基于响应式系统构建，提供计算属性和侦听能力。

核心优势：

- 按需代理提升性能，避免 Vue2 全量递归初始化的开销。
- 全面覆盖数据操作类型，无需 `vue.set/delete` 等手动干预。
- 更好的 TS 支持与内存管理，适配现代 JS 数据结构。

示例回答：

"Vue3 利用 Proxy 代理对象，在数据访问时通过 track 收集依赖（如组件渲染函数），在数据修改时通过 trigger 触发更新。通过 reactive 处理对象、ref 处理值类型，结合 effect 管理副作用，实现了高效的响应式系统。相比 Vue2，它能自动检测更多数据变化类型，且性能更优。"

Q6: vue2和vue3的区别？

1. 双向绑定原理发生变化

vue2 利用 `Object.defineProperty()` 对数据进行劫持并结合发布者订阅者模式

vue3 使用 proxy 进行数据代理，可以实现对整个对象的监听，也可以检测到数组内部的变化

2. Vue3 的组件可以有多个根节点，Vue2 只能有一个根节点

3. vue2 是选项性 API，在代码里分割了不同的属性 data、computed、methods 等，Vue3 是组合格 API 用方法来分割，这样代码会更整洁。

4. vue2 中响应数据直接放入 data 属性中，vue3 需要把响应数据放入 setup() 方法

5. 生命周期不同

6. vue3 的响应式系统使用 weakmap，当一个响应式对象已经被垃圾回收机制所回收时，就不需要存储 key-value 了

map 的键可以是任何类型，weakmap 的键只能是引用类型（对象，数组，函数，正则（RegExp）和日期（Date））

weakmap 对 key 是弱引用，不影响垃圾回收机制工作，不会造成内存溢出

weakmap 使用场景，用于存储那些只有当 key 所引用的对象存在时，才有价值的信息

Q7: 用了哪些大模型？

chatgpt, deepseek, 豆包, 文心一言

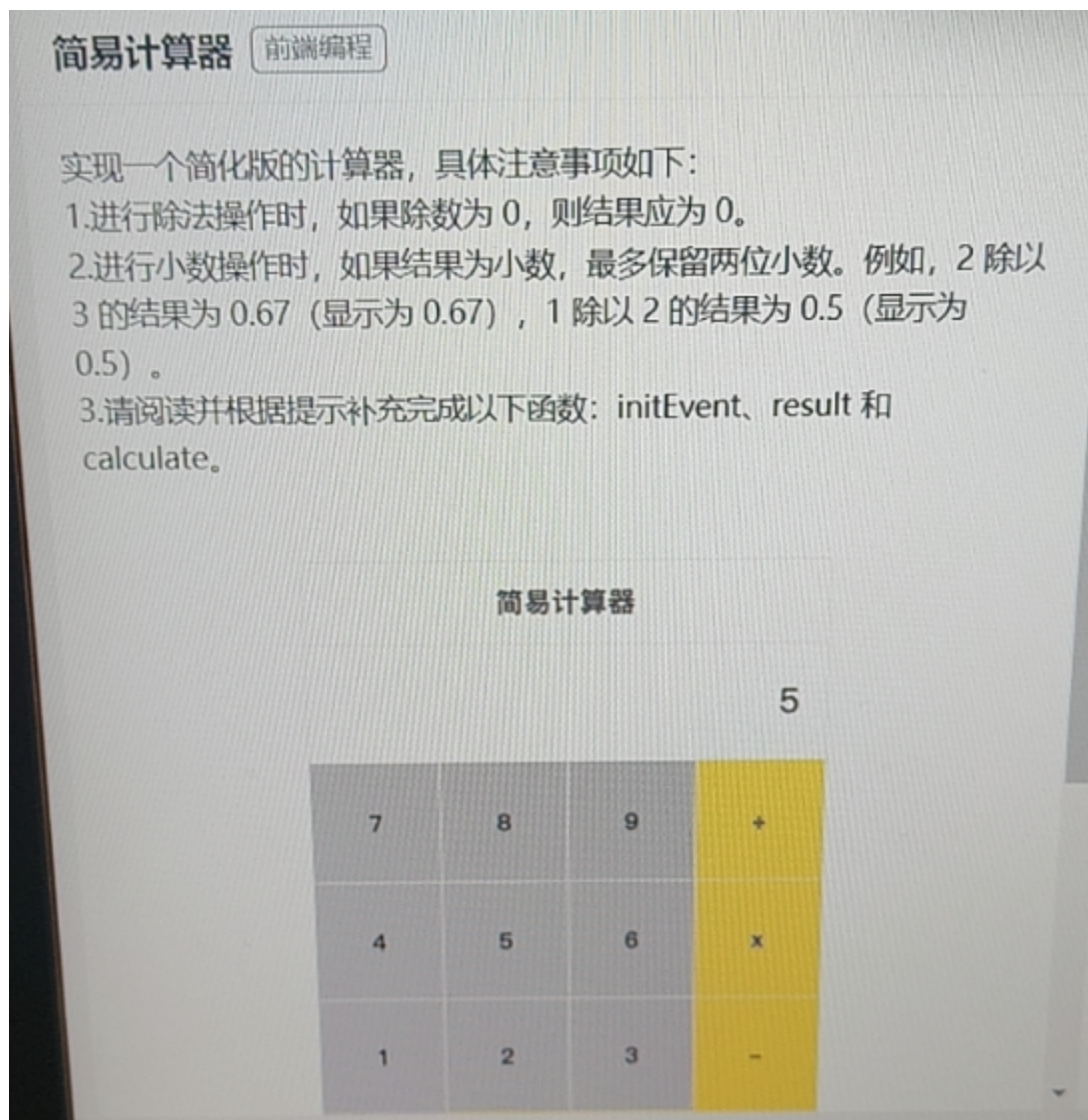
Q8: 本科和研究生期间学了哪些课程?

本科: python, java, c语言, 408, 数据库

研究生：智能优化，高等数学，算法

2. 代码题

题目1: 简易计算器



```
<!DOCTYPE html>

<html>

<head>
    <meta charset="UTF-8" />
    <style>
        body,
        ul,
        li,
        select {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
    </style>

```

```
ul,
li {
    list-style: none;
}

.calculator {
    max-width: 300px;
    margin: 20px auto;
    border: 1px solid #eee;
    border-radius: 3px;
}

.cal-header {
    font-size: 16px;
    color: #333;
    font-weight: bold;
    height: 48px;
    line-height: 48px;
    border-bottom: 1px solid #eee;
    text-align: center;
}

.cal-main {
    font-size: 14px;
}

.cal-main .origin-value {
    padding: 15px;
    height: 40px;
    line-height: 40px;
    font-size: 20px;
    text-align: right;
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
}

.cal-main .origin-type,
.cal-main .target-type {
    padding-left: 5px;
    width: 70px;
    font-size: 14px;
    height: 30px;
    border: 1px solid #eee;
    background-color: #fff;
    vertical-align: middle;
    margin-right: 10px;
    border-radius: 3px;
}

.cal-keyboard {
    overflow: hidden;
}

.cal-items {
    overflow: hidden;
}
```

```

.cal-items li {
    user-select: none;
    float: left;
    display: inline-block;
    width: 75px;
    height: 75px;
    color: #000;
    font-weight: bold;
    text-align: center;
    line-height: 75px;
    border-top: 1px solid #eee;
    border-left: 1px solid #eee;
    box-sizing: border-box;
    cursor: pointer;
    background-color: rgba(0, 0, 0, 0.3);
}

.cal-items li:hover {
    opacity: 0.8;
}

li:nth-of-type(4n + 1) {
    border-left: none;
}

li[data-action="operator"] {
    background: #ffd100;
}

.buttons {
    float: left;
    width: 75px;
}

.buttons .btn {
    width: 75px;
    background-color: #fff;
    border-top: 1px solid #eee;
    border-left: 1px solid #eee;
    height: 150px;
    line-height: 150px;
    text-align: center;
}

.btn-esc {
    color: #ff5a34;
}

.btn-backspace {
    position: relative;
}
</style>
</head>

<body>
    <div class="calculator">
        <header class="cal-header">简易计算器</header>
        <main class="cal-main">

```

```

<div class="origin-value">0</div>
<div class="cal-keyboard">
  <ul class="cal-items">
    <li data-action="num">7</li>
    <li data-action="num">8</li>
    <li data-action="num">9</li>
    <li data-action="operator">÷</li>
    <li data-action="num">4</li>
    <li data-action="num">5</li>
    <li data-action="num">6</li>
    <li data-action="operator">x</li>
    <li data-action="num">1</li>
    <li data-action="num">2</li>
    <li data-action="num">3</li>
    <li data-action="operator">-</li>
    <li data-action="num">0</li>
    <li data-action="operator">清空</li>
    <li data-action="operator">=</li>
    <li data-action="operator">+</li>
  </ul>
</div>
</main>
</div>
<script type="text/javascript">
  var calculator = {
    init: function () {
      var that = this;
      if (!that.isInitied) {
        that.isInitied = true;
        // 保存操作信息
        // total: Number, 总的结果
        // next: String, 下一个和 total 进行运算的数据
        // action: String, 操作符号
        that.data = { total: 0, next: "", action: "" };
        that.bindEvent();
      }
    },
    bindEvent: function () {
      var that = this;
      // TODO: 获取 .cal-keyboard 元素
      var keyboardEl = document.querySelector(".cal-keyboard");
      keyboardEl &&
        keyboardEl.addEventListener("click", function (event) {
          // TODO: 获取当前点击的dom元素
          var target = event.target;
          // console.log(target);
          // TODO: 获取target的 data-action 值
          var action = target.dataset.action;
          // console.log(action);
          // TODO: 获取target的内容
          var value = target.textContent;
          if (action === "num" || action === "operator") {
            that.result(value, action === "num");
          }
        });
    },
    result: function (action, isNum) {
      var that = this;

```

```

var data = that.data;
if (isNum) {
    data.next = data.next === "0" ? action : data.next + action;
    !data.action && (data.total = 0);
} else if (action === "清空") {
    // TODO: 设置清空时的对应状态
    data.total = 0;
    data.next = "";
    data.action = "";
} else if (action === "=") {
    if (data.next || data.action) {
        data.total = that.calculate(data.total, data.next,
data.action);

        data.next = "";
        data.action = "";
    }
} else if (!data.next) {
    data.action = action;
} else if (data.action) {
    data.total = that.calculate(data.total, data.next,
data.action);

    data.next = "";
    data.action = action;
} else {
    data.total = +data.next || 0;
    data.next = "";
    data.action = action;
}

// TODO: 获取 .origin-value 元素

var valEl = document.querySelector(".origin-value");

valEl && (valEl.innerHTML = data.next || data.total || "0");
},
calculate: function (n1, n2, operator) {
    n1 = +n1 || 0;
    n2 = +n2 || 0;
    let result = 0;
    if (operator === "÷") {
        // TODO: 获取除法的结果
        result = n2 === 0 ? 0 : n1 / n2;
    } else if (operator === "x") {
        // TODO: 获取乘法的结果
        result = n1 * n2;
    } else if (operator === "+") {
        // TODO: 获取加法的结果
        result = n1 + n2;
    } else if (operator === "-") {
        // TODO: 获取减法的结果
        result = n2 - n1;
    }
    return parseFloat(result.toFixed(2))
    currentValue = ""
    operator = ""
},
};
calculator.init();

```

```
</script>
</body>

</html>
```

题目二：零钱兑换

[题目描述](#) [题解](#) [提交记录](#)



518. 零钱兑换 II

中等

相关标签

相关企业

Ax

给你一个整数数组 `coins` 表示不同面额的硬币，另给一个整数 `amount` 表示总金额。

请你计算并返回可以凑成总金额的硬币组合数。如果任何硬币组合都无法凑出总金额，返回 `0`。

假设每一种面额的硬币有无限个。

题目数据保证结果符合 32 位带符号整数。

示例 1:

输入: `amount = 5, coins = [1, 2, 5]`

输出: 4

解释: 有四种方式可以凑成总金额:

`5=5`

`5=2+2+1`

`5=2+1+1+1`

`5=1+1+1+1+1`

示例 2:

输入: `amount = 3, coins = [2]`

输出: 0

解释: 只用面额 2 的硬币不能凑成总金额 3。

示例 3:

输入: `amount = 10, coins = [10]`

输出: 1

确定dp数组以及下标的含义：凑成总金额j的货币组合数为dp[j]

确定递推公式：dp[j] 就是所有的dp[j - coins[i]]（考虑coins[i]的情况）相加。

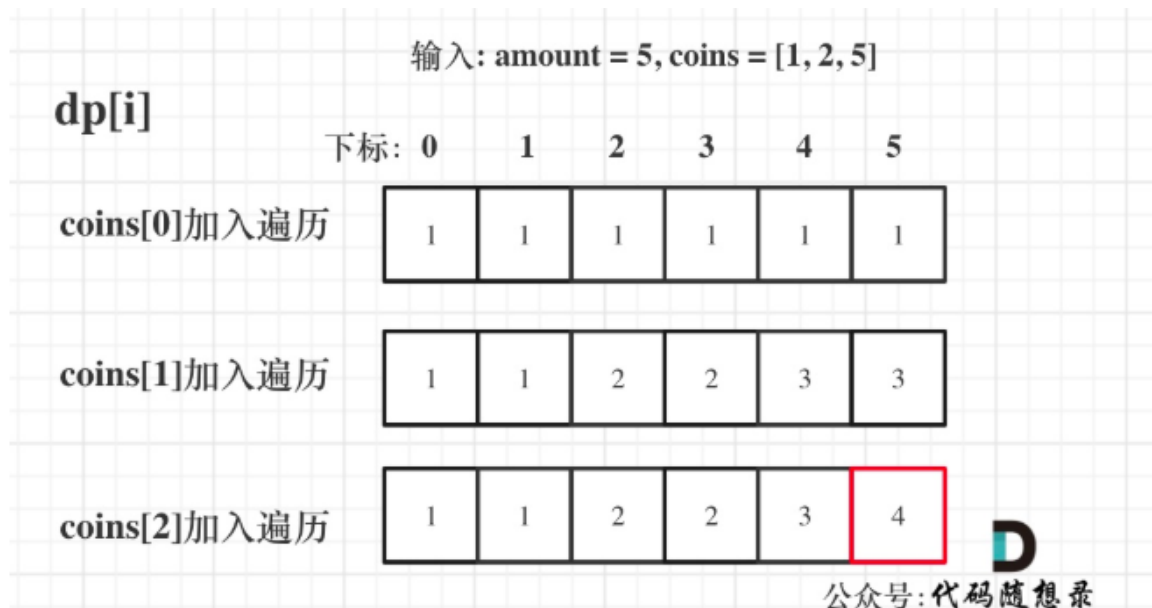
链接: <https://leetcode.cn/problems/coin-change-ii/solutions/851288/dai-ma-sui-xiang-lu-518-ling-qian-dui-hu-qzgm/>

```
/**
 * @param {number} amount
 * @param {number[]} coins
 * @return {number}
```

```

*/
var change = function(amount, coins) {
  let dp = Array(amount + 1).fill(0)
  dp[0] = 1
  for(let i = 0; i < coins.length; i++){
    for(j = coins[i]; j <= amount; j++){
      dp[j] += dp[j - coins[i]]
    }
  }
  console.log(dp)
  return dp[amount]
};

```



3.20快手面试

1. 面试题

Q1: 现在在学校还是在实习?

还在学校, 最近正在找暑期实习

Q2: 这两个项目是实验室项目还是网上的? 会上线吗?

导师接的项目, 和别的实验室合作的

Q3: 项目是几个人做的?

两个前端三个后端

Q4: 介绍一下Openlayers和Cesium实现了什么功能

调用地图过慢, 分辨率优化问题

Q5: 怎么获取经纬度和像素点的位置?

```
import { transform } from 'ol/proj';

map.on('click', (event) => {
  // 获取地图点击坐标 (默认返回地图投影坐标系 EPSG:3857)
  const coordinate = event.coordinate;

  // 转换为 WGS84 经纬度
  const lonlat = transform(coordinate, 'EPSG:3857', 'EPSG:4326');

  console.log('经度:', lonlat[0]);
  console.log('纬度:', lonlat[1]);
});
```

```
// 地理坐标转像素位置
const pixel = map.getPixelFromCoordinate(coordinate);

// 像素位置转地理坐标
const coordFromPixel = map.getCoordinateFromPixel(pixel);
```

Q6: JS的继承是怎么实现的?

1. 原型链继承

原理

- 让子类的原型 (prototype) 指向父类的实例, 从而继承父类的属性和方法。

优点

- 子类可以访问父类的属性和方法。

缺点

- 所有子类实例共享同一个父类实例**, 如果其中一个实例修改了父类的引用类型属性, 所有实例都会受到影响。
- 无法向父类构造函数传参。**

```
child1.__proto__ ---> Child.prototype
                        ↓
                    new Parent() (实例)
                        ↓
                    Parent.prototype
                        ↓
                    Object.prototype
                        ↓
                      null
```

当访问 `child1.sayHello()` 时:

- 先在 `child1` 实例对象查找 `sayHello`, 找不到。
- 沿 `__proto__` 找到 `Child.prototype`, 依然找不到。
- 继续沿 `__proto__` 找到 `Parent.prototype`, 找到了 `sayHello`, 执行方法。
- 方法执行结束, 查找过程终止。

```

function Parent() {
  this.name = "Parent";
  this.arr = [1, 2, 3];
}

Parent.prototype.sayHello = function () {
  console.log("Hello from Parent");
};

function Child() {}

// 继承 Parent
Child.prototype = new Parent(); // 关键：让子类的原型对象指向父类的实例
Child.prototype.constructor = Child; // 纠正 constructor 指向

let child1 = new Child();
child1.age = 18
let child2 = new Child();
child1.arr.push(4);

console.log(child1); // [1, 2, 3, 4]
console.log(child2); // [1, 2, 3, 4] 受影响

```

```

child1  ▾ Child {age: 18} i
  age: 18
  ▾ [[Prototype]]: Parent
    ▶ arr: (4) [1, 2, 3, 4]
    ▶ constructor: f Child()
    name: "Parent"
    ▾ [[Prototype]]: Object
      ▶ sayHello: f ()
      ▶ constructor: f Parent()
      ▶ [[Prototype]]: Object

child2  ▾ Child {} i
  ▾ [[Prototype]]: Parent
    ▶ arr: (4) [1, 2, 3, 4]
    ▶ constructor: f Child()
    name: "Parent"
    ▾ [[Prototype]]: Object
      ▶ sayHello: f ()
      ▶ constructor: f Parent()
      ▶ [[Prototype]]: Object

```

2. 构造函数继承

原理

- 在子类的构造函数中调用 `Parent.call(this, args...)`，借用父类的构造函数。

优点

- 解决了 原型链继承中的引用类型共享问题。
- 可以向父类构造函数传参。

缺点

- 无法继承父类的原型方法（只能继承构造函数中的属性）。

- 每个子类实例都会创建一份父类的方法，增加内存消耗。

示例

```
function Parent(name) {  
  this.name = name;  
  this.arr = [1, 2, 3];  
}  
  
function Child(name, age) {  
  Parent.call(this, name); // 继承 Parent, 并传递参数  
  this.age = age;  
}  
  
let child1 = new Child("Alice", 20);  
let child2 = new Child("Bob", 25);  
  
child1.arr.push(4);  
console.log(child1); // [1, 2, 3, 4]  
console.log(child2); // [1, 2, 3] 不受影响
```

`Parent.call(this, name);` 相当于把 `Parent` 里的代码拷贝到 `Child` 里面运行，并把 `this` 绑定到 `Child` 的实例上。这样 `Child` 的实例就能访问 `Parent` 构造函数中的属性。

```
child1  ▼ Child {name: 'Alice', arr: Array(4), age: 20} ⓘ  
    age: 20  
    ▶ arr: (4) [1, 2, 3, 4]  
    name: "Alice"  
    ▼ [[Prototype]]: Object  
    ▶ constructor: f Child(name, age)  
    ▶ [[Prototype]]: Object  
  
child2  ▼ Child {name: 'Bob', arr: Array(3), age: 25} ⓘ  
    age: 25  
    ▶ arr: (3) [1, 2, 3]  
    name: "Bob"  
    ▶ [[Prototype]]: Object
```

3. 组合继承

原理

- 原型链继承（继承方法）
- 借用构造函数继承（继承属性）

优点

- 既能继承父类的实例属性，又能继承原型上的方法。
- 不会共享引用类型数据。
- 可以向父类构造函数传参。

缺点

- 调用了两次父类构造函数，造成 不必要的内存占用。

示例

```
function Parent(name) {  
  this.name = name;  
  this.arr = [1, 2, 3];  
}  
  
Parent.prototype.sayHello = function () {  
  console.log("Hello from Parent");  
};  
  
function Child(name, age) {  
  Parent.call(this, name); // 继承属性  
  this.age = age;  
}  
  
Child.prototype = new Parent(); // 继承方法// 这里调用了一次 Parent 构造函数  
Child.prototype.constructor = Child; // 修正 constructor 指向  
  
let child1 = new Child("Alice", 20); // 这里又调用了一次 Parent 构造函数  
let child2 = new Child("Bob", 25);  
  
child1.arr.push(4);  
console.log(child1); // [1, 2, 3, 4]  
console.log(child2); // [1, 2, 3] 不受影响  
  
child1.sayHello(); // Hello from Parent
```

注意： `Child.prototype = new Parent();` 会导致 `Parent` 构造函数执行两次。

```
child1  ▼ Child {name: 'Alice', arr: Array(4), age: 20} ⓘ  
    age: 20  
    ▶ arr: (4) [1, 2, 3, 4]  
    name: "Alice"  
    ▼ [[Prototype]]: Parent  
        ▶ arr: (3) [1, 2, 3]  
        ▶ constructor: f Child(name, age)  
        name: undefined  
        ▶ [[Prototype]]: Object  


---

child2  ▼ Child {name: 'Bob', arr: Array(3), age: 25} ⓘ  
    age: 25  
    ▶ arr: (3) [1, 2, 3]  
    name: "Bob"  
    ▼ [[Prototype]]: Parent  
        ▶ arr: (3) [1, 2, 3]  
        ▶ constructor: f Child(name, age)  
        name: undefined  
        ▶ [[Prototype]]: Object  


---

Hello from Parent
```

4. 原型式继承

原理

- 直接基于一个对象创建新的对象，新对象可以访问原对象的属性。
- 本质上是 `Object.create(proto)`。

优点

- 比原型链继承更简单，不需要使用构造函数。

缺点

- 所有实例共享引用类型数据，与原型链继承的问题相同。

示例

```
let parent = {  
  name: "Parent",  
  arr: [1, 2, 3],  
  sayHello: function () {  
    console.log("Hello from Parent");  
  },  
};  
  
let child = Object.create(parent); // 继承 parent  
child.name = "Child";  
  
console.log(child); // Child  
child.arr.push(4);
```

```
console.log(parent); // [1, 2, 3, 4] 受影响
```

注意：修改 `child.arr` 会影响 `parent.arr`，因为它们共享同一个对象。

```
child  ▼ {name: 'Child'} ⓘ
  name: "Child"
  ▼ [[Prototype]]: Object
    ▶ arr: (4) [1, 2, 3, 4]
      name: "Parent"
    ▶ sayHello: f ()
    ▶ [[Prototype]]: Object

parent  ▼ {name: 'Parent', arr: Array(4), sayHello: f} ⓘ
  ▶ arr: (4) [1, 2, 3, 4]
    name: "Parent"
  ▶ sayHello: f ()
  ▼ [[Prototype]]: Object
    ▶ constructor: f Object()
    ▶ hasOwnProperty: f hasOwnProperty()
    ▶ isPrototypeOf: f isPrototypeOf()
    ▶ propertyIsEnumerable: f propertyIsEnumerable()
    ▶ toLocaleString: f toLocaleString()
    ▶ toString: f toString()
    ▶ valueOf: f valueOf()
    ▶ __defineGetter__: f __defineGetter__()
    ▶ __defineSetter__: f __defineSetter__()
    ▶ __lookupGetter__: f __lookupGetter__()
    ▶ __lookupSetter__: f __lookupSetter__()
    __proto__: (Object)
    ▶ get __proto__: f __proto__()
    ▶ set __proto__: f __proto__()
```

5. 寄生式继承

原理

- 在原型式继承的基础上，返回一个增强的新对象。

优点

- 可以增加额外的方法，比原型式继承更灵活。

缺点

- 共享引用类型数据，与原型式继承相同。

示例

```
function createChild(parentObj) {
  let clone = Object.create(parentObj);
```

```

clone.sayHi = function () {
  console.log("Hi from child");
};
return clone;
}

let parent = { name: "Parent", arr: [1, 2, 3] };
let child = createChild(parent);

child.sayHi(); // Hi from child
child.arr.push(4);
console.log(child);
console.log(parent); // [1, 2, 3, 4] 受影响

```

Hi from child

```

child  ▼ {sayHi: f} ⓘ
  ▶ sayHi: f ()
  ▶ [[Prototype]]: Object

parent  ▼ {name: 'Parent', arr: Array(4)} ⓘ
  ▶ arr: (4) [1, 2, 3, 4]
  ▶ name: "Parent"
  ▶ [[Prototype]]: Object

```

6. 寄生组合式继承

原理

- 结合 借用构造函数 和 原型式继承，避免了 组合继承调用父类构造函数两次的问题。

优点

- 最佳继承方式，减少了内存浪费。
- 子类原型继承父类的方法，同时构造函数不重复调用。

示例

```

function Parent(name) {
  this.name = name;
  this.arr = [1, 2, 3];
}

Parent.prototype.sayHello = function () {
  console.log("Hello from Parent");
};

function Child(name, age) {
  Parent.call(this, name); // 继承实例属性
  this.age = age;
}

// 关键：创建新对象继承 Parent.prototype，而不是 `new Parent()`
Child.prototype = Object.create(Parent.prototype);

```

```
Child.prototype.constructor = Child; // 修正 constructor
```

```
let child1 = new Child("Alice", 20);  
child1.sayHello(); // Hello from Parent
```

优势:

- 只调用了一次 `Parent.call(this, name)`, 不会重复调用构造函数。
- 子类不会继承父类实例上的属性, 但仍然继承了原型方法。

Hello from Parent

```
child1  ▾ Child {name: 'Alice', arr: Array(3), age: 20} ⓘ  
  age: 20  
  ▶ arr: (3) [1, 2, 3]  
  name: "Alice"  
  ▾ [[Prototype]]: Parent  
    ▶ constructor: f Child(name, age)  
    ▾ [[Prototype]]: Object  
      ▶ sayHello: f ()  
      ▶ constructor: f Parent(name)  
      ▶ [[Prototype]]: Object
```

```
Child f Child(name, age) {  
  Parent.call(this, name); // 继承实例属性  
  this.age = age;  
}
```

```
Parent f Parent(name) {  
  this.name = name;  
  this.arr = [1, 2, 3];  
}
```

```
Child.prototype  ▾ Parent {} ⓘ  
  ▶ constructor: f Child(name, age)  
  ▶ [[Prototype]]: Object
```

```
Parent.prototype  ▾ {sayHello: f} ⓘ  
  ▶ sayHello: f ()  
  ▶ constructor: f Parent(name)  
  ▶ [[Prototype]]: Object
```

7. ES6 Class 继承

示例

```
class Parent {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

```

    sayHello() {
      console.log("Hello from Parent");
    }
  }

  class Child extends Parent {
    constructor(name, age) {
      super(name); // 调用父类构造函数
      this.age = age;
    }
  }

  let child1 = new Child("Alice", 20);
  child1.sayHello(); // Hello from Parent

```

Hello from Parent

```

child1  ▼ Child {name: 'Alice', age: 20} ⓘ
    age: 20
    name: "Alice"
    ▶ [[Prototype]]: Parent
      ▶ constructor: class Child
      ▶ [[Prototype]]: Object

```

```

Child.prototype  ▼ Parent {} ⓘ
    ▶ constructor: class Child
    ▶ [[Prototype]]: Object

```

```

Parent.prototype  ▼ {sayHello: f} ⓘ
    ▶ constructor: class Parent
    ▶ sayHello: f sayHello()
    ▶ [[Prototype]]: Object

```

8. 总结

继承方式	适用场景	主要问题
原型链继承	共享方法	共享引用数据
构造函数继承	继承实例属性	不能继承方法
组合继承	经典继承方式	调用两次构造函数
原型式继承	创建新对象	共享引用数据
寄生式继承	添加额外方法	共享引用数据
寄生组合继承	最佳实践	无明显缺点
ES6 class 继承	现代方式	语法糖

推荐使用：寄生组合继承 或 ES6 class 继承

Q7：深拷贝、浅拷贝（引用拷贝）的底层逻辑是什么？

- 浅拷贝仅复制第一层，对象或数组的引用仍然指向原内存。
- 如果对象嵌套较深，推荐使用 `JSON.parse(JSON.stringify(obj))`（有局限）或深拷贝方案。

方法	适用于	是否拷贝对象引用	是否能拷贝多层嵌套对象
<code>Object.assign()</code>	对象	是	否
<code>{ ...obj }</code>	对象	是	否
<code>slice()</code>	数组	是	否
<code>concat()</code>	数组	是	否
<code>for...in</code> / <code>Object.keys()</code>	对象	是	否

```
//深拷贝
function deepClone(obj) {
  let cache = new WeakMap();

  function _deepClone(obj) {
    if (obj === null || typeof obj !== 'object') {
      return obj; // 直接返回基本数据类型
    }

    if (cache.has(obj)) {
      return cache.get(obj); // 避免循环引用
    }

    let result = Array.isArray(obj) ? [] : {}; // 兼容数组和对象
    cache.set(obj, result); // 先存入缓存，避免后续递归时遇到相同引用

    for (let key in obj) {
      if (obj.hasOwnProperty(key)) { //排除继承属性
        result[key] = _deepClone(obj[key]); // 递归拷贝
      }
    }

    return result;
  }

  return _deepClone(obj);
}
```

```
//浅拷贝
function shallowClone(obj) {
  if (typeof obj !== 'object' || obj === null) {
    return obj; // 处理基本数据类型
  }

  let result = Array.isArray(obj) ? [] : {}; // 判断是否是数组

  for (let key in obj) {
```

```

    if (obj.hasOwnProperty(key)) { // 过滤继承属性
      result[key] = obj[key]; // 直接赋值（引用数据类型仍然共享同一块内存）
    }
  }

  return result;
}

```

Q8: Object.assign的底层实现原理？

1. 类型检查

- 如果 `target` 为 `null` 或 `undefined`，抛出 `TypeError`。
- 目标 `target` 会被强制转换为对象（如果是基本类型 `Number/String/Boolean`）。

2. 遍历 `sources`

- 逐个遍历 `sources`，跳过 `null` 或 `undefined`。

3. 拷贝属性

- 仅拷贝**可枚举的自有属性**（不包含继承属性、不可枚举属性）。
- 直接赋值到 `target`，不会进行深拷贝。

```

if (typeof Object.assign !== 'function') {
  Object.assign = function (target) {
    if (target == null) {
      throw new TypeError('Cannot convert undefined or null to object');
    }

    let to = Object(target); // 确保 target 是对象

    for (let i = 1; i < arguments.length; i++) {
      let nextSource = arguments[i];
      if (nextSource != null) { // 跳过 null 和 undefined
        for (let key in nextSource) {
          if (Object.prototype.hasOwnProperty.call(nextSource, key)) {
            to[key] = nextSource[key]; // 复制可枚举自有属性
          }
        }
      }
    }

    return to;
  };
}

```

Q9: `Object.assign(target.prototype, source.prototype)` 实现了什么功能？

```

// 示例1
function Parent() {}
Parent.prototype.sayHello = function () {
  console.log('Hello from Parent');
};

```

```
function Child() {}
Object.assign(Child.prototype, Parent.prototype);

const child = new Child();
child.sayHello(); // "Hello from Parent"

//示例2
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };
const result = Object.assign({}, obj1, obj2);
console.log(result); // { a: 1, b: 3, c: 4 }
```

(1) 实现的效果

- 把 `Parent.prototype` 的方法复制到 `Child.prototype` 上, 使 `Child` 的实例能访问 `Parent.prototype` 的方法。
- 并不修改 `Child` 的原型链, `Child.prototype.__proto__` 仍然是 `Object.prototype`, 而不是 `Parent.prototype`。

(2) 与 `Child.prototype = new Parent();` 的区别

方法	<code>Object.assign(Child.prototype, Parent.prototype)</code>	<code>Child.prototype = new Parent();</code>
继承方式	拷贝 <code>Parent.prototype</code> 的方法	通过 <code>new Parent()</code> 创建对象
影响原型链?	✗ 不改变 <code>Child.prototype.__proto__</code>	☑ <code>Child.prototype.__proto__ = Parent.prototype</code>
执行 <code>Parent</code> 构造函数?	✗ 不会执行	☑ 会执行一次 <code>Parent()</code>
适用场景	只想复制 <code>Parent.prototype</code> 的方法	想让 <code>Child</code> 继承 <code>Parent</code> 并建立原型链

(3) 适用场景

- `Object.assign(Child.prototype, Parent.prototype)` 适用于“混入模式” (Mixin), 即把多个类的方法合并到 `Child.prototype` 上, 而不影响原型链。
- 如果希望 `Child` 继承 `Parent` 并形成原型链, 建议用 `Object.create(Parent.prototype)` 或 `extends` 关键字。

Q10: `response.json()` 实现了什么功能? 为什么要使用?

1. `response.json()` 的功能

- `response.json()` 是 `Fetch API` 提供的方法, 用于解析 HTTP 响应体的 JSON 数据。
- 它返回一个 `Promise`, 解析后得到 JavaScript 对象。

2. 为什么要使用 `response.json()` ?

- Fetch API 的 `response` 默认是 `ReadableStream` 类型，不能直接访问 JSON 数据。
- `response.json()` 读取流数据并将其解析为 JavaScript 对象，便于后续操作。

3. `response.json()` 的执行过程

//`fetch()` 是 API，它可以发送各种 HTTP 请求（GET、PUT、POST、DELETE），默认情况下 `fetch(url)` 就是 GET 请求。

```
fetch('https://api.example.com/data')
  .then(response => response.json()) // 解析 JSON
  .then(data => console.log(data))   // 处理 JSON 数据
  .catch(error => console.error('Error:', error));
```

✓ `response.json()` 只能使用一次，因为 `response.body` 是 `ReadableStream`，读取后无法再次访问。

当 `fetch()` 执行后，`response` 是一个 `Response` 对象：

The screenshot shows the browser's developer console. At the top, a `Response` object is logged from `index.js:6`. It has properties like `type: 'cors'`, `url: 'http://172.27.160.161:8080/list/getresourcelist'`, `status: 200`, and `body: ReadableStream`. A red box highlights the `body` property. Below it, the `response.json()` method is called, returning a `Promise` object (logged from `index.js:8`). A red box highlights the `[[PromiseResult]]` property of the promise, which is an object containing `code: 200`, `data`, and `msg`. A red arrow points from the `data` property of this object to another red box highlighting the `data` property of the final resolved value (logged from `index.js:11`), which is an object with `code: 200`, `data`, and `msg`.

```
Response {
  type: "cors",
  url: "https://api.example.com/data",
  redirected: false,
  status: 200,
  ok: true,
  statusText: "OK",
  headers: Headers,
  body: ReadableStream
}
```

关键属性解析：

属性	值	解释
<code>type</code>	<code>"cors"</code>	表示跨域请求
<code>url</code>	<code>"https://api.example.com/data"</code>	请求的 URL
<code>status</code>	<code>200</code>	HTTP 状态码 (成功)
<code>ok</code>	<code>true</code>	状态码在 200-299 之间
<code>statusText</code>	<code>"OK"</code>	状态文本
<code>headers</code>	<code>Headers</code>	HTTP 头部信息
<code>body</code>	<code>ReadableStream</code>	响应体 (数据流, 需要解析)

对象	作用	示例
<code>response</code>	HTTP 响应对象	<code>{ status: 200, ok: true, body: ReadableStream }</code>
<code>response.json()</code>	解析 JSON 数据, 返回 <code>Promise</code>	<code>{ "id": 1, "name": "Alice", "age": 25 }</code>
<code>data</code>	解析后的 JavaScript 对象	<code>{ id: 1, name: "Alice", age: 25 }</code>

3. `fetch()` vs `axios`

	<code>fetch()</code>	<code>axios</code>
返回值	<code>Promise<Response></code>	<code>Promise<{ data, status, headers... }></code>
数据解析	需手动 <code>.json()</code>	自动解析
错误处理	仅网络错误进入 <code>.catch()</code> , 状态码错误 (如 404) 不会	状态码错误也会进入 <code>.catch()</code>
适用于	原生 API, 现代浏览器	更强大的 HTTP 客户端, 支持更多功能 (如拦截器)

Q11: `response.ok` 代表什么? 一般什么情况下返回的不是 `ok`?

1. `response.ok` 代表的意义

- `response.ok` 是 `Fetch API` 的 `Response` 对象的一个属性, 表示 HTTP 响应状态是否在 200~299 之间。
- `response.ok` 的返回值:
 - `true`: HTTP 状态码 200 - 299 (表示成功)。
 - `false`: HTTP 状态码 小于 200 或大于 299 (表示失败)。

2. 什么情况下 `response.ok` 不是 `true`?

- 常见的失败状态码

状态码	描述	可能原因
400	Bad Request	请求参数错误
401	Unauthorized	未授权访问
403	Forbidden	没有权限访问
404	Not Found	资源不存在
500	Internal Server Error	服务器内部错误
503	Service Unavailable	服务器不可用

Q12: `response` 返回的是什么类型?

- `fetch()` 返回的是一个 `Promise<Response>`，即：

```
fetch(url).then(response => {  
  console.log(response); // Response 对象  
});
```

- `response` 是 `Response` 对象，包含 HTTP 响应的所有信息。
- `Promise<Response>` 代表一个异步 HTTP 请求，最终解析（`resolve`）成 `Response` 对象。

Response 对象的常见属性

属性	作用
<code>status</code>	HTTP 状态码（如 200, 404, 500）
<code>statusText</code>	HTTP 状态信息（如 "OK", "Not Found"）
<code>ok</code>	<code>true</code> 表示状态码在 200-299，否则 <code>false</code>
<code>headers</code>	Headers 对象，存放 HTTP 头部信息
<code>url</code>	响应的 URL
<code>json()</code>	解析 JSON 响应体，返回 <code>Promise<Object></code>
<code>text()</code>	解析为 <code>String</code> ，返回 <code>Promise<String></code>
<code>blob()</code>	解析为 <code>Blob</code> （如图片、文件）
<code>arrayBuffer()</code>	解析为 <code>ArrayBuffer</code> （二进制数据）

问题	回答
<code>response.json()</code> 实现了什么功能? 为什么要使用?	解析 <code>response.body</code> 为 JSON 并返回 <code>Promise<Object></code> , 因为 <code>response.body</code> 是 <code>ReadableStream</code> , 需要解析才能使用
<code>response.ok</code> 代表什么? 一般什么情况下返回的不是 <code>ok</code> ?	<code>response.ok</code> 表示 HTTP 状态码在 200-299 之间, 失败的请求 (如 404, 500, 403) 会返回 <code>false</code>
<code>response</code> 返回的是什么类型?	<code>fetch()</code> 返回 <code>Promise<Response></code> , 其中 <code>response</code> 是 <code>Response</code> 对象, 包含 HTTP 响应信息和解析方法

Q13: call、apply、bind

在 JavaScript 中, `apply`、`call` 和 `bind` 主要用于 **改变 `this` 指向**, 它们是函数对象的内置方法, 允许我们在不同的上下文 (`this`) 中调用函数。

1. `call`、`apply`、`bind` 的基本用法

```
function sayHello(age, city) {
  console.log(`Hello, my name is ${this.name}, I am ${age} years old and live in ${city}.`);
}

const person = { name: "Alice" };
```

方法	语法	作用
<code>call</code>	<code>fn.call(thisArg, arg1, arg2, ...)</code>	立即调用 <code>fn</code> , <code>this</code> 指向 <code>thisArg</code> , 参数 逐个传递
<code>apply</code>	<code>fn.apply(thisArg, [arg1, arg2, ...])</code>	立即调用 <code>fn</code> , <code>this</code> 指向 <code>thisArg</code> , 参数 以数组形式传递
<code>bind</code>	<code>fn.bind(thisArg, arg1, arg2, ...)</code>	返回一个新函数 , <code>this</code> 绑定为 <code>thisArg</code> , 参数预填充, 不立即执行

示例:

```
sayHello.call(person, 25, "Beijing"); // Hello, my name is Alice, I am 25 years old and live in Beijing.
sayHello.apply(person, [30, "Shanghai"]); // Hello, my name is Alice, I am 30 years old and live in Shanghai.

const boundFunc = sayHello.bind(person, 35, "Guangzhou");
boundFunc(); // Hello, my name is Alice, I am 35 years old and live in Guangzhou.
```

2. call 和 apply 的区别

call 和 apply 的主要区别在于 参数的传递方式:

- call(thisArg, arg1, arg2, arg3, ...) → 逐个传递
- apply(thisArg, [arg1, arg2, arg3, ...]) → 数组传递

使用场景

(1) apply 适用于参数已在数组中的情况

示例: 求数组中的最大值

```
const numbers = [5, 2, 8, 3, 7];

// 使用 apply 直接传递数组
const max = Math.max.apply(null, numbers);
console.log(max); // 8
```

等价于:

```
console.log(Math.max(...numbers)); // 8 (ES6 展开运算符)
```

(2) call 适用于参数数量固定、已知的情况

示例: 借用方法

```
const obj1 = { name: "Tom" };
const obj2 = { name: "Jerry" };

function greet() {
  console.log(`Hello, I am ${this.name}`);
}

greet.call(obj1); // Hello, I am Tom
greet.call(obj2); // Hello, I am Jerry
```

3. bind 的特殊性

bind 和 call、apply 的最大不同点是:

- call 和 apply 立即执行 函数
- bind 返回一个新函数, 不会立即执行, 只会绑定 this 和预填充参数

示例:

```
function introduce(job) {
  console.log(`Hi, I am ${this.name}, and I work as a ${job}.`);
}

const person1 = { name: "John" };

const boundIntroduce = introduce.bind(person1, "developer");
boundIntroduce(); // Hi, I am John, and I work as a developer.
```

4. apply、call、bind 的常见使用场景

(1) 借用构造函数

有时，我们需要借用另一个构造函数，以便重用已有的初始化逻辑。

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

function Student(name, age, grade) {
  Person.call(this, name, age); // 继承 Person 的属性
  this.grade = grade;
}

const student = new Student("Jack", 20, "A");
console.log(student); // { name: 'Jack', age: 20, grade: 'A' }
```

(2) 数组转类数组

示例：将 arguments 伪数组转换为真正的数组

```
function sum() {
  const args = Array.prototype.slice.call(arguments); // arguments 是类数组
  return args.reduce((a, b) => a + b, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
```

迭代次数	a (累积值)	b (当前元素)	计算结果
第 1 次	0	1	0 + 1 = 1
第 2 次	1	2	1 + 2 = 3
第 3 次	3	3	3 + 3 = 6
第 4 次	6	4	6 + 4 = 10

(3) bind 实现偏函数

预填充参数，创建一个新的函数

```
function multiply(a, b) {
  return a * b;
}

// 创建一个新的函数，固定 `a=2`
const double = multiply.bind(null, 2);
console.log(double(5)); // 10
```

(4) 事件监听中使用 bind

在事件回调中，默认 `this` 指向 事件目标对象，我们可以使用 `bind` 让其指向我们需要的对象。

```
const obj = {
  name: "Lisa",
  showName: function () {
    console.log(this.name);
  }
};

// 直接传递会导致 `this` 指向 `button` 而不是 `obj`
document.getElementById("btn").addEventListener("click",
obj.showName.bind(obj));
```

5. bind 的模拟实现

```
Function.prototype._call = function (ctx, ...args) {
  const o = (ctx === undefined || ctx === null) ? window : Object(ctx)
  o.fn = this
  const result = o.fn(...args)
  delete o.fn
  return result
}

Function.prototype._apply = function (ctx, array = []) {
  const o = (ctx === undefined || ctx === null) ? window : Object(ctx)
  o.a = this
  const result = o.a(...array)
  delete o.a
  return result
}

Function.prototype._bind = function (ctx, ...args) {
  const _self = this
  return function (...rest) {
    return _self._call(ctx, ...args, ...rest)
    // return _self._apply(ctx, [...args, ...rest])
  }
}
```

6. call 的模拟实现

```
Function.prototype._call = function (ctx, ...args) {
  const o = (ctx === undefined || ctx === null) ? window : Object(ctx)
  o.fn = this
  const result = o.fn(...args)
  delete o.fn
  return result
}

fn._call(obj, 1, 2)
```

7. 总结

方法	作用	是否立即执行	参数形式	返回值
<code>call</code>	改变 <code>this</code> 指向并调用函数	✓	逐个传递	调用函数返回值
<code>apply</code>	改变 <code>this</code> 指向并调用函数	✓	数组传递	调用函数返回值
<code>bind</code>	返回一个绑定 <code>this</code> 的新函数	✗	逐个传递	绑定 <code>this</code> 后的新函数

最佳应用场景

- `call`：适用于**参数数量固定**的情况，如方法借用、构造函数继承。
- `apply`：适用于**参数是数组**的情况，如 `Math.max.apply(null, array)`。
- `bind`：适用于**事件监听、定时器、偏函数**，用于创建 `this` 绑定的新函数。

2. 代码题

Q1：请实现一个extend函数，使C继承A和B的方法

```
class A {
  eat() {
    console.log("A eat")
  }
}
class B {
  drink() {
    console.log("B drink")
  }
}
class C {}

function extend() {
  //请写出你的代码
  sources.forEach(source => {
    Object.assign(target.prototype, source.prototype);
  });
}
extend(C, A, B)
let c = new C()
c.eat()
c.drink()
```

Q2: 实现一个函数，可以完成请求失败后自动重试n次的功能（可以用js请求库）

```
function fetchWithRetry(url, n){
  return new Promise((resolve, reject) => {
    function attemptFetch() {
      fetch(url)
      .then(response => {
        if(!response.ok) {
          throw new Error("网络错误")
        }
        return response.json()
      })
      .then(data => {
        resolve(data)
      })
      .catch(error => {
        if(n > 0) {
          //防止高并发
          setTimeout(attemptFetch, 1000)
        } else {
          reject(new Error("没有重试次数"))
        }
      })
    }
    attemptFetch()
  })
}
```

Q3: 请实现一个最近联系人列表的功能组件，只考虑js逻辑

```
class RecentList {
  constructor(capacity) {
    //初始化联系人列表和最大容量
    this.capacity = capacity
    this.contactsList = []
  }
  get() {
    //返回当前联系人列表
    return this.contactsList
  }
  put(contact) {
    //联系人已存在就将其移除，并添加至列表开头
    const index = this.contactsList.indexOf(contact)
    if(index !== -1) {
      this.contactsList.splice(index, 1)
    }
    //将联系人添加到列表开头
    this.contactsList.unshift(contact)

    //如果联系人数量超过最大容量，则移除最后一个联系人
    if(this.contactsList.length > capacity) {
      this.contactsList.pop()
    }
  }
}
```

```
}
```

Q4: 怎么在O(1)时间复杂度下实现最近联系人的功能列表?

为了在 **O(1) 时间复杂度** 内高效地实现最近联系人列表 (RecentList)，我们可以 **结合 Map 和 LinkedList (双向链表)** 进行优化。

- 使用 **Map 存储联系人**，键是联系人，值是该联系人的链表节点指针。这样可以 **O(1) 时间** 内检查是否存在联系人并删除。
- 使用 **Doubly Linked List (双向链表)** 存储访问顺序，以 **O(1) 时间** 在头部插入最新联系人，在尾部删除最旧联系人。
- **put(contact)** 操作：
 1. 如果 **contact** 已经存在，则将其从链表中删除 (**O(1)**)。
 2. 在链表头部插入 **contact** (**O(1)**)。
 3. 如果列表超过容量，删除链表尾部节点 (**O(1)**)。
- **get()** 操作：
 1. 直接返回链表数据 (**O(1)**)。

🚀 这种方式实现了 **O(1)** 时间复杂度的最近联系人列表，非常适用于 LRU (最近最少使用) 缓存等场景!

```
class ListNode {
    constructor(value) {
        this.value = value;
        this.prev = null;
        this.next = null;
    }
}

class RecentList {
    constructor(capacity) {
        this.capacity = capacity;
        this.map = new Map(); // 存储联系人 -> ListNode 指针
        this.head = new ListNode(null); // 虚拟头节点
        this.tail = new ListNode(null); // 虚拟尾节点
        this.head.next = this.tail;
        this.tail.prev = this.head;
    }

    // 在头部插入节点
    _addToHead(node) {
        node.next = this.head.next;
        node.prev = this.head;
        this.head.next.prev = node;
        this.head.next = node;
    }

    // 删除节点
    _removeNode(node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    get() {
```

```

    let result = [];
    let curr = this.head.next;
    while (curr !== this.tail) {
        result.push(curr.value);
        curr = curr.next;
    }
    return result;
}

put(contact) {
    if (this.map.has(contact)) {
        let existingNode = this.map.get(contact);
        this._removeNode(existingNode); // O(1) 删除
    }

    let newNode = new ListNode(contact);
    this._addToHead(newNode); // O(1) 插入
    this.map.set(contact, newNode);

    // 如果超出容量，移除最后一个节点
    if (this.map.size > this.capacity) {
        let lastNode = this.tail.prev;
        this._removeNode(lastNode);
        this.map.delete(lastNode.value);
    }
}

// 使用示例
let recentContacts = new RecentList(3);
recentContacts.put("Alice");
recentContacts.put("Bob");
recentContacts.put("Charlie");
console.log(recentContacts.get()); // ["Charlie", "Bob", "Alice"]
recentContacts.put("Alice");
console.log(recentContacts.get()); // ["Alice", "Charlie", "Bob"]
recentContacts.put("Dave");
console.log(recentContacts.get()); // ["Dave", "Alice", "Charlie"]

```

3.24作业帮面试

Q1：浏览器输入url到页面渲染的过程

分为如下步骤：

1. 解析 URL 获取请求信息。
2. DNS 解析 获取 IP 地址。
3. 建立 TCP 连接（三次握手）。
4. 发送 HTTP 请求。
5. 服务器处理请求 并返回响应。
6. 浏览器渲染页面（HTML → DOM → CSSOM → 渲染）。
7. 执行 JavaScript（修改 DOM、异步加载）。
8. 关闭 TCP 连接（四次挥手）。

1. 解析 URL

用户输入的 URL 需要进行解析，以确保其符合 HTTP 请求的标准格式，并提取关键组成部分。

URL 组成

一个完整的 URL 形如：

```
https://www.example.com:443/path/to/resource?query=123#section
```

主要组成部分：

- **协议 (Scheme)**: `http://` 或 `https://`，决定使用的网络协议。
- **主机名 (Host)**: `www.example.com`，服务器的域名。
- **端口号 (Port)**: `443` (HTTPS 默认 443, HTTP 默认 80)。
- **路径 (Path)**: `/path/to/resource`，服务器上的资源路径。
- **查询参数 (Query String)**: `?query=123`，用于携带客户端提供的数据。
- **片段标识符 (Fragment)**: `#section`，仅用于前端，不会发送到服务器。

解析过程

1. 检查用户输入

- 如果输入是 **纯文本** (如 `example`)，浏览器可能跳转默认搜索引擎。
- 如果是 **缺少协议的 URL** (如 `www.example.com`)，浏览器会自动补全 `http://` 或 `https://`。

2. 确定请求方式

- 默认使用 `GET` 请求。
- 若是表单提交，则可能使用 `POST`。

2. DNS 解析

为了访问服务器，浏览器需要将域名解析为 IP 地址。

DNS 解析流程

1. **浏览器缓存**：浏览器会先检查本地是否缓存了该域名对应的 IP 地址。
2. **本地 DNS 服务器 (LDNS)**：向运营商的 DNS 服务器查询。
3. 递归解析 (如果 LDNS 没有缓存)：
 - 访问 **根域名服务器** (.) 获取 `.com` 的 NS 记录。
 - 访问 **顶级域名服务器** (.com) 获取 `example.com` 的 NS 记录。
 - 访问 **权威域名服务器** 获取 `www.example.com` 的 A 记录 (IP 地址)。
4. **缓存结果**：解析结果会缓存一段时间 (由 `TTL` 设定)。

3. 建立 TCP 连接

浏览器获取服务器 IP 后，使用 **TCP 三次握手** 建立连接。

三次握手流程

1. **客户端 → 服务器**：发送 `SYN` (同步序列号) 请求连接。
2. **服务器 → 客户端**：发送 `SYN-ACK` (同步确认)。
3. **客户端 → 服务器**：发送 `ACK` (确认)，连接建立。

⚠ 如果是 HTTPS，还需进行 **TLS/SSL 握手**：

- 客户端请求服务器的 **SSL 证书** 并验证合法性。
- 双方协商 **对称密钥**，后续通信加密。

4. 发送 HTTP 请求

TCP 连接建立后，浏览器发送 HTTP 请求。

请求报文结构

```
GET /path/to/resource HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html,application/xhtml+xml
Cookie: session=xyz
```

主要部分：

- **请求行**：包含方法（GET/POST）、路径、协议版本（HTTP/1.1）。
- **请求头**：
 - **Host**：目标服务器域名。
 - **User-Agent**：浏览器类型。
 - **Cookie**：用户状态信息。
- **请求体**（仅 POST 请求有）。

5. 服务器处理请求并返回响应

服务器收到 HTTP 请求后，进行处理。

服务器处理过程

1. 解析 HTTP 请求

- 检查 **Host** 确定目标网站。
- 解析路径 **/path/to/resource** 定位资源。

2. 执行后端逻辑

- 可能涉及数据库查询、计算任务、权限验证等。

3. 构造 HTTP 响应

- 成功：

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1024
```

- 失败：

```
HTTP/1.1 404 Not Found
```

4. 返回数据

- HTML 页面、JSON 数据、图片等。

6. 浏览器渲染页面

浏览器解析服务器返回的 HTML 并渲染页面。

渲染过程

1. 解析 HTML

- 构建 **DOM 树** (Document Object Model) 。
- 遇到外部资源 (CSS、JS、图片) :
 - CSS 和 JS 可能会阻塞 HTML 解析。

2. 解析 CSS

- 构建 **CSSOM** (CSS 对象模型) 。
- 计算每个元素的样式。

3. 合成渲染树

- DOM 树 + CSSOM → **渲染树** (Render Tree) 。

4. 计算布局 (Layout)

- 计算每个元素的位置、大小。

5. 绘制 (Painting)

- 将渲染树转换为像素绘制到屏幕上。
- 可能涉及 GPU 硬件加速。

7. 执行 JavaScript

当浏览器遇到 `<script>` 标签时，会执行 JavaScript 代码。

JS 执行方式

1. 同步执行

- 阻塞 HTML 解析，直到 JS 执行完成。

2. 异步加载

- `async`：脚本下载完立即执行（不保证顺序）。
- `defer`：等 HTML 解析完成后按顺序执行。

3. 事件循环

- 处理异步任务（如 AJAX 请求、定时器）。

4. 可能触发页面更新

- JS 可能修改 DOM，导致 **回流 (Reflow)** 或 **重绘 (Repaint)** 。

8. 关闭 TCP 连接

如果使用的是 **HTTP/1.0**，服务器在响应后立即关闭连接。如果使用 **HTTP/1.1 或 HTTP/2**，默认使用 **长连接 (keep-alive)**，多个请求可复用同一 TCP 连接。

TCP 四次挥手 (断开连接)

1. **客户端** → **服务器**：发送 `FIN` (终止) 。
2. **服务器** → **客户端**：返回 `ACK` (确认) 。
3. **服务器** → **客户端**：发送 `FIN` (终止) 。
4. **客户端** → **服务器**：返回 `ACK` (确认)，连接关闭。

Q2：缓存的作用是什么？

HTTP 缓存的主要作用是减少 **网络请求**，加快 **页面加载速度**，降低 **服务器压力**，优化用户体验。

缓存机制的核心目标：

- **减少带宽占用**：避免重复下载相同资源。
- **加快响应速度**：直接从本地缓存获取数据。
- **减轻服务器压力**：减少服务器请求次数，提高性能。

Q3：强缓存和协商缓存怎么判断？设置的字段具体有什么作用？

HTTP 缓存主要分为 **强缓存** 和 **协商缓存**：

缓存类型	触发条件	服务器参与	主要字段
强缓存	资源未过期	不向服务器发送请求	<code>Expires</code> 、 <code>Cache-Control: max-age</code>
协商缓存	资源过期或未命中强缓存	向服务器发送请求，服务器决定是否使用缓存	<code>Last-Modified</code> 、 <code>ETag</code>

1. 强缓存（Expires / Cache-Control）

强缓存命中时，浏览器不会向服务器请求资源，而是直接从 **本地缓存** 读取，提高页面加载速度。

判断方式

如果资源命中 **强缓存**，浏览器控制台的 **Network** 面板会显示 **(from disk cache)** 或 **(from memory cache)**，并且不会发起网络请求。

设置字段

① `Expires`（HTTP/1.0，基于绝对时间）

`Expires: wed, 24 Mar 2025 12:00:00 GMT`

- 表示资源的过期时间，**在此时间之前，浏览器不会重新请求该资源。**
- 缺点：
 - 受客户端 **本地时间影响**，如果用户手动修改本地时间，缓存可能失效。

② `Cache-Control`（HTTP/1.1，基于相对时间）

`Cache-Control: max-age=3600, public`

- `max-age=3600`：资源的有效期是 **3600 秒**（相对时间）。
- `public`：允许所有用户缓存（包括代理服务器）。
- `private`：只能存储于浏览器本地缓存，不允许代理缓存。
- `no-cache`：不直接使用缓存，必须与服务器进行 **协商缓存** 验证。
- `no-store`：**不缓存**，每次都要重新下载资源。

强缓存示例

```
Cache-Control: max-age=86400 # 24 小时
```

流程

1. 浏览器检查缓存中是否有该资源，并判断 `max-age` 是否过期。
2. 如果 **未过期**，直接使用缓存（**不发送请求**）。
3. 如果 **已过期**，则进行 **协商缓存**。

2. 协商缓存（ETag / Last-Modified）

当资源过期后，浏览器会向服务器发起请求，服务器决定是否返回 **304 Not Modified** 让浏览器使用本地缓存，或返回 **新的资源**。

判断方式

如果资源命中 **协商缓存**，浏览器控制台的 **Network** 面板会显示 **304 Not Modified**，表示服务器允许使用缓存。

设置字段

① ETag（基于文件内容哈希）

服务器返回资源时，提供一个唯一标识（哈希值）：

```
ETag: "abc123"
```

下次请求时，浏览器带上：

```
If-None-Match: "abc123"
```

服务器对比：

- **哈希值相同**，返回 `304 Not Modified`。
- **哈希值不同**，返回 `200 OK`，并更新 `ETag`。

优势

- **精准识别** 资源变化，避免 `Last-Modified` 的秒级精度问题。
- 适用于 **动态资源**，如 JSON、API 返回的数据。

② Last-Modified（基于文件修改时间）

服务器在响应资源时，返回资源的最后修改时间：

```
Last-Modified: Wed, 24 Mar 2025 12:00:00 GMT
```

浏览器下次请求时，会在请求头带上：

```
If-Modified-Since: Wed, 24 Mar 2025 12:00:00 GMT
```

服务器对比：

- **文件未修改**，返回 `304 Not Modified`，浏览器使用本地缓存。

- 文件已修改，返回 200 OK，并更新 Last-Modified。

缺点

- 只能精确到 秒级，若文件 短时间内多次修改，可能无法正确识别更新。

强缓存 + 协商缓存 示例

```
Cache-Control: max-age=86400
ETag: "abc123"
Last-Modified: Wed, 24 Mar 2025 12:00:00 GMT
```

流程

1. 资源未过期 (max-age 生效) → 直接使用强缓存。
2. 资源过期 → 浏览器发送 If-None-Match & If-Modified-Since 请求服务器。
3. 服务器比较 ETag 和 Last-Modified：
 - 匹配 → 返回 304 Not Modified (浏览器用缓存)。
 - 不匹配 → 返回 200 OK (下载新资源)。

3. 浏览器缓存策略

场景	适用缓存策略	说明
静态资源 (JS、CSS、图片)	强缓存 (Cache-Control: max-age)	资源少变更时，减少请求数，提高性能
HTML 文件	协商缓存 (ETag、Last-Modified)	防止用户访问旧版页面
API 数据	协商缓存 或 Cache-Control: no-cache	避免获取过时数据
安全性高的数据	不缓存 (Cache-Control: no-store)	例如登录信息、用户隐私数据

4. 强缓存和协商缓存的对比

对比项	强缓存	协商缓存
服务器是否参与	✗ 不参与	✓ 服务器判断
是否发送 HTTP 请求	✗ 不发送	✓ 发送请求
响应状态码	200 OK (from cache)	304 Not Modified
设置字段	Expires / Cache-Control	ETag / Last-Modified
适用场景	静态资源 (图片、JS、CSS)	HTML、API 数据

5. 总结

- **强缓存：**
 - 通过 `Cache-Control: max-age` 或 `Expires` 设置资源过期时间。
 - 命中时，**不发起请求**，直接使用本地缓存。
- **协商缓存：**
 - 通过 `ETag` / `Last-Modified` 让服务器判断资源是否更新。
 - **发送请求**，但服务器可能返回 `304 Not Modified`。
- **浏览器缓存策略：**
 - **静态资源** (JS、CSS、图片)：优先使用 **强缓存**。
 - **HTML / API 数据**：使用 **协商缓存**，避免过时内容。
 - **敏感数据** (用户信息)：使用 `Cache-Control: no-store`，禁止缓存。

Q4: max-age的时间是相对于响应到达客户端的时间计算的？相对于相应到达服务器的时间可以吗？

`max-age` 是 `Cache-Control` 响应头中的一个字段，用于控制资源的缓存时间。`max-age` 的时间是相对于 **响应到达客户端的时间** 来计算的，而不是相对于服务器的时间。

1. max-age 是相对于客户端的时间

- `max-age` 的时间单位是秒，从 **浏览器接收到响应** 的时刻开始计算。
- **示例**
 - 如果服务器返回一个资源，响应头如下：

```
Cache-Control: max-age=3600
```

- 浏览器接收到响应的时间为 `2025-03-24 10:00:00 GMT`，那么：
 - 资源的 **有效时间** 为 `2025-03-24 11:00:00 GMT` (即 1 小时后)。
 - 如果用户再次请求该资源的时间为 `2025-03-24 10:30:00 GMT`，则 **仍然命中缓存**，因为 `max-age` 设定的 3600 秒 (1 小时) 并未到期。
- **结论：**`max-age` 从 **客户端收到响应的时间** 开始计算，**不是从服务器的时间**。

2. 为什么不能相对于服务器时间计算？

- **服务器时间无法反映客户端的实际时间**，不同的客户端可能会有不同的时区或时间设置。
- 服务器和客户端时间的不同步会导致 **缓存过期时间的混乱**。为了保证缓存控制的准确性，浏览器根据 **实际接收到响应的时间** 来判断缓存的有效性。

3. 关于 `Cache-Control: max-age` 的一些补充

- **客户端行为：**
 - 在 `max-age` 内，浏览器 **直接使用缓存**，不会向服务器发起请求。
 - **超过 `max-age` 时**，缓存过期，浏览器会向服务器发起请求，可能会进行协商缓存 (返回 `304 Not Modified` 或 `200 OK`)。
- **为什么从客户端时间开始：**

- 缓存策略应该基于客户端使用的时间来判断缓存是否有效，因为缓存内容是由客户端决定是否使用的。

总结：

`max-age` 的计算时间是 **从客户端收到响应的时间** 开始的，而不是从服务器返回响应时开始计算。这样做是为了避免由于客户端和服务器的时间差异而导致缓存控制的不一致性。

Q5：还有什么有关缓存的字段？

1. `Cache-Control`：控制缓存的主要字段，支持 `max-age`、`no-cache` 等指令。
2. `Expires`：指定资源的过期时间，基于绝对时间。
3. `Last-Modified` 和 `ETag`：用于实现协商缓存，通过对比修改时间或版本标识符判断资源是否更新。
4. `If-Modified-Since` 和 `If-None-Match`：客户端发送的请求头字段，用于与服务器进行缓存验证。
5. `Pragma`：主要用于 HTTP/1.0，控制缓存行为。
6. `Vary`：控制代理服务器根据哪些请求头决定缓存策略。

1. `Expires`

`Expires` 是一个 HTTP 响应头，表示资源的过期时间，基于 **绝对时间**。与 `Cache-Control` 不同，`Expires` 是固定的时间（**优先**），表示该资源在此时间之前有效。

示例：

```
Expires: Wed, 24 Mar 2025 12:00:00 GMT
```

2. `Cache-Control`

`Cache-Control` 是最常用的缓存控制字段，可以包含多个指令，决定缓存的行为。它主要用于控制 **缓存存储的时长、类型** 和 **是否进行缓存验证**。

常用的 `Cache-Control` 指令：

- `max-age=<seconds>`：定义缓存的最大有效时间（秒）。超过该时间，缓存失效。
- `no-cache`：表示缓存资源必须经过服务器验证后再使用。客户端每次请求时，必须向服务器发起请求检查资源是否修改。
- `no-store`：表示不缓存资源，每次请求都会向服务器发起请求。
- `public`：允许任何缓存机制（包括共享缓存，如 CDN）缓存资源。
- `private`：只允许私有缓存存储资源，适用于用户特定内容（如用户的个人页面）。
- `must-revalidate`：表示缓存过期后必须进行重新验证，即使缓存内容在过期后仍然可以使用，也需要先进行验证。
- `proxy-revalidate`：与 `must-revalidate` 类似，但只对代理服务器有效。

示例：

```
Cache-Control: max-age=3600, must-revalidate, public
```

3. ETag

ETag 是服务器返回的一个 **唯一标识符**，用于标识某个特定版本的资源。它通常是一个哈希值或文件指纹。客户端通过发送 If-None-Match 请求头与服务器对比资源是否发生变化，从而实现 **协商缓存**。

- **首次请求时**，服务器会返回 ETag。
- **下次请求时**，浏览器会发送 If-None-Match 请求头，带上上次返回的 ETag 值，服务器根据该值判断资源是否有变动。

示例：

```
ETag: "abc123"
```

4. If-None-Match

If-None-Match 是客户端发送的请求头字段，用于与 ETag 配合工作。它将上次响应中返回的 ETag 值发送给服务器，询问自那时以来资源是否有变化。

- 服务器响应：
 - 如果 ETag 匹配，且资源未修改，服务器返回 304 Not Modified。
 - 如果 ETag 不匹配，服务器返回新资源。

示例：

```
If-None-Match: "abc123"
```

5. Last-Modified

Last-Modified 是服务器返回的响应头，表示资源的 **最后修改时间**。客户端可以根据此字段判断资源是否发生了变化，配合 If-Modified-Since 请求头使用，来实现 **协商缓存**。

- 当浏览器缓存资源时，首次请求时会保存 Last-Modified。
- 下一次请求时，浏览器会发送 If-Modified-Since 请求头，询问服务器资源自上次修改后是否有更新。

示例：

```
Last-Modified: Wed, 24 Mar 2025 12:00:00 GMT
```

6. If-Modified-Since

If-Modified-Since 是客户端发送的请求头字段，用于与 Last-Modified 配合工作。它会将上次请求时服务器返回的 Last-Modified 时间发送给服务器，询问自那时以来资源是否有变化。

- 服务器响应：
 - 如果资源自该时间之后没有被修改，服务器返回 304 Not Modified。
 - 如果资源有修改，服务器返回最新的资源。

示例：

```
If-Modified-Since: Wed, 24 Mar 2025 12:00:00 GMT
```

7. Pragma

`Pragma` 是一个 HTTP 头字段，主要用于 HTTP/1.0 协议的缓存控制。在现代的 HTTP/1.1 中，`Cache-Control` 取代了 `Pragma`。不过，`Pragma` 依然存在于 HTTP/1.0 中，通常用来控制缓存的行为。

常用值：

- `Pragma: no-cache`：表示不缓存资源。

示例：

```
Pragma: no-cache
```

8. Vary

`Vary` 是服务器返回的响应头字段，用于告知缓存代理（如 CDN）在缓存资源时，应该根据哪些请求头的变化来选择不同的缓存版本。它告诉代理在缓存时需要考虑哪些请求头字段的值。

常用值：

- `Vary: Accept-Encoding`：根据 `Accept-Encoding` 请求头决定缓存。
- `Vary: User-Agent`：根据用户代理（浏览器）选择缓存。

示例：

```
Vary: Accept-Encoding
```

Q6：vite进行打包之后会生成什么文件？

打包完成后，会在项目根目录下生成一个 `dist` 文件夹，其中包含以下内容：

- `index.html`：项目的入口HTML文件，包含根元素和加载脚本。
- `static` 目录：存放静态资源，如图片、字体等。
- `js` 目录：包含经过压缩和优化的JavaScript文件。
- `css` 目录：包含经过压缩的CSS文件。

Q7：强缓存和协商缓存存储在哪？

- 强缓存和协商缓存的内容都存储在浏览器的 **本地缓存存储** 中，通常存储在浏览器的缓存目录或数据库（如 `IndexedDB` 或 `SQLite`）中。
- 强缓存会直接使用本地缓存，而协商缓存会在访问时与服务器进行协商，确认资源是否有更新。

Q8：强缓存和协商缓存返回的状态码是什么？

- 强缓存返回 **304 Not Modified** 状态码
- 协商缓存返回 **200 OK** 状态码

Q9：怎么判断强缓存是否过期？

判断强缓存是否过期，可以通过 `Cache-Control` 和 `Expires` 这两个字段来决定：

1. `Cache-Control: max-age=<seconds>`：
 - `max-age` 表示从响应时间开始，缓存资源可以存储多长时间，单位是秒。

- 当浏览器请求资源时，它会检查缓存中存储的资源的 `max-age` 是否超过了当前时间和资源存储时间的差值。如果过期，浏览器会发送请求到服务器，重新获取资源。
- 例如，`Cache-Control: max-age=3600` 意味着资源缓存一个小时。如果资源已经被缓存超过一个小时，浏览器就会认为缓存已过期，发起新的请求。

2. Expires :

- `Expires` 表示缓存的过期时间点，它是一个绝对的日期时间。
- 浏览器会与当前时间进行比较，判断是否超过了过期时间。如果超过了，缓存过期，浏览器需要重新请求资源。
- 例如，`Expires: Wed, 21 Oct 2025 07:28:00 GMT` 表示缓存将在 2025 年 10 月 21 日过期。

3. 缓存过期的判断:

- 如果缓存的时间已经超过了 `max-age` 或者当前时间已经超过了 `Expires` 的时间，强缓存就会被认为已经过期，浏览器会发起新的请求，重新获取资源。
- 如果缓存仍在有效期内，浏览器则会直接从缓存中读取资源，不会发起新的请求。

总结：强缓存是否过期通过比较当前时间与缓存控制字段（`max-age` 或 `Expires`）来判断，若超过有效期，则认为缓存过期，浏览器会重新请求资源。

Q10：怎么根据DOM树和CSSOM树生成渲染树？

1. DOM 树

- **DOM（文档对象模型）** 树是浏览器解析 HTML 文档时生成的结构，它表示 HTML 文档的结构层次。每个 HTML 元素都对应 DOM 树中的一个节点。

2. CSSOM 树

- **CSSOM（CSS Object Model）** 树是浏览器解析 CSS 样式表时生成的结构。它表示所有样式规则的结构。每个 CSS 规则会被转换为 CSSOM 树中的节点。

3. 生成渲染树的步骤

步骤 1：构建 DOM 树

- 浏览器首先会解析 HTML 文档，生成 **DOM 树**，每个 HTML 元素会变成树中的一个节点。
- 例如：

```
<html>
  <head>
    <style>
      body { color: red; }
    </style>
  </head>
  <body>
    <div>hello</div>
  </body>
</html>
```

上面的 HTML 会被解析成一个 DOM 树，包含 `<html>`、`<head>`、`<style>`、`<body>`、`<div>` 等节点。

步骤 2：构建 CSSOM 树

- 同时，浏览器会解析 CSS 文件或 `<style>` 标签，生成 CSSOM 树。每个 CSS 规则都会被转换为一个树节点，例如：

```
body { color: red; }
```

对应的 CSSOM 树节点包含 `body` 选择器和 `color: red;` 样式。

步骤 3：合并 DOM 树和 CSSOM 树生成渲染树

- 渲染树是页面的视觉表示，它只包含需要显示在屏幕上的节点。
 - **节点的过滤：** 渲染树并不会包含一些不可见的节点，比如 `display: none` 的元素或者在 CSS 中设置了 `visibility: hidden` 的元素。
 - **样式计算：** 对于 DOM 树中的每一个可见节点，浏览器会根据 CSSOM 树中的样式规则，计算出这些节点的最终样式。这个过程包括计算样式、布局计算、合并样式等操作。
- **合并过程：** 浏览器会将 **DOM 树中的元素** 和 **CSSOM 树中的样式** 结合起来，形成 **渲染树**。每个渲染树节点包含该元素的 **视觉样式**，比如：大小、位置、颜色等。
 - 渲染树节点包含的是 **可视化信息**（如位置、尺寸等），而 DOM 树节点包含的是 **文档结构信息**。
 - 例如，对于一个 `div` 元素，如果它的 `display` 样式为 `none`，它就不会出现在渲染树中。

步骤 4：生成渲染树

- 渲染树由浏览器根据 DOM 树和 CSSOM 树合并生成。每个渲染树节点通常包括：
 - **元素的标签类型**（如 `<div>`）。
 - **该元素的最终样式**（如 `color: red`、`width: 100px`）。
 - **元素的几何信息**（如位置和尺寸）。

例如：

- 如果有一个 `div` 元素在 CSS 中设置了 `display: block`，且在 DOM 树中它是 `<div>`，那么这个 `div` 元素就会出现在渲染树中，且它会拥有与 `display: block` 相关的样式信息（如占据一行）。

步骤 5：渲染树的作用

- 渲染树是浏览器绘制页面时的基础。它用于绘制每个可视元素，并计算其最终的 **位置和尺寸**。渲染树生成之后，浏览器可以根据渲染树信息执行 **布局 (layout)** 和 **绘制 (painting)** 阶段，最终在屏幕上呈现页面内容。

4. 总结：

1. **DOM 树** 和 **CSSOM 树** 是生成渲染树的基础。DOM 树代表页面的结构，CSSOM 树代表页面的样式。
2. **渲染树** 通过将 DOM 树和 CSSOM 树结合，过滤掉不可见元素（如 `display: none`），最终形成可视化的树形结构。
3. 渲染树的生成为后续的 **布局 (layout)** 和 **绘制 (painting)** 提供了所需的信息。

Q11：闭包

1. 闭包的定义是什么？

闭包是指一个函数能够访问其**外部作用域**的变量，即使外部函数已经执行完毕。

2. 闭包有什么作用？

- 避免变量被污染
- 私有化，外部只能使用不能修改
- 保存变量，常驻内存

3. 闭包的缺点是什么？

1. 内存泄漏

闭包会保持对外部函数变量的引用，即使外部函数已经执行完毕。这样会导致这些变量无法被垃圾回收机制回收，从而增加内存的使用量。如果闭包在某些情况下没有被正确清理，尤其是在大量数据或长时间运行的场景下，可能会导致内存泄漏。

示例：

```
function createClosure() {  
  let largeData = new Array(1000000).fill("some large data"); // 大量数据  
  return function() {  
    console.log(largeData);  
  };  
}  
  
let closure = createClosure(); // 即使createClosure执行完毕，largeData仍无法被回收
```

在上面的代码中，`largeData` 会被闭包引用并保持在内存中，导致即使 `createClosure` 执行完毕，`largeData` 仍然无法被垃圾回收机制回收。

2. 调试困难

闭包可能导致作用域链复杂，尤其是多个嵌套的闭包时，容易产生调试上的问题。调试时，如果不小心捕获了大量的变量，会使得程序难以跟踪，甚至难以确定变量到底是从哪个作用域中传递而来。

3. 性能问题

闭包可能导致内存的过度使用，因为它会一直保持对外部作用域中变量的引用，导致这些变量不能及时被回收。在频繁创建闭包的情况下，可能会影响性能，特别是在循环和递归中，如果每次都创建闭包，可能导致内存占用过多。

4. 变量污染和作用域问题

闭包可能导致意外的变量污染。例如，在不同的闭包中使用相同的变量名，可能会导致变量的值意外被覆盖或修改，尤其是当多个闭包在共享一个外部作用域时，可能会产生作用域链的冲突。

5. 代码可读性差

当闭包使用不当时，它可能让代码变得难以理解和维护，特别是在复杂的异步操作或递归中，闭包的作用域和引用可能会让代码逻辑变得不清晰。

4. 怎么解决闭包内存泄漏？

1. 及时清理引用：

- 当闭包不再需要时，手动将闭包内引用的变量设置为 `null`，以帮助垃圾回收机制回收内存。

```
function createClosure() {  
  let largeData = new Array(1000000).fill("some large data");  
  return function() {  
    console.log(largeData);  
  };  
}  
  
let closure = createClosure();  
closure = null; // 解除对 largeData 的引用
```

2. 避免不必要的闭包：

- 尽量避免在循环、递归等高频率的地方创建闭包，减少内存占用。

3. 使用 `let/const` 而非 `var`：

- 使用 `let` 和 `const` 可以避免闭包意外引用到不再使用的变量，避免污染作用域链。

4. 使用模块化和命名空间：

- 将闭包放入模块或命名空间中，减少全局作用域对变量的引用，防止无关的引用占用内存。

Q12：内存泄漏

1. 什么是内存泄漏

内存泄漏（Memory Leak）指的是程序在运行过程中，无法释放已经不再使用的内存，导致这部分内存持续占用而无法被回收，最终可能导致系统内存耗尽，影响程序的性能，甚至导致程序崩溃。

内存泄漏的表现：

- 内存占用持续增长：**即使程序执行过程中不再需要某些资源，内存使用量仍然不断增长。
- 性能下降：**程序在运行过程中变慢，尤其是在长时间运行的情况下，内存泄漏会逐渐影响到性能。

2. 什么情况下会造成内存泄漏

- 未释放的对象：**程序创建了大量的对象，但没有及时释放或清理这些对象的引用，使得它们无法被垃圾回收机制回收。
- 闭包导致的引用：**在 JavaScript 等语言中，闭包会保持对外部作用域变量的引用，即使外部函数已经执行完毕，仍然存在这些引用，导致内存无法释放。
- 定时器和事件监听器未清除：**如果程序使用了定时器（如 `setInterval`）或事件监听器，但没有在合适的时机清除它们，可能会导致相关资源无法被垃圾回收。
- 全局变量滥用：**不小心创建的全局变量（特别是在大型应用中）会一直存在，导致不再需要的对象无法被回收。
- DOM 元素未移除：**在 Web 开发中，如果不删除不再需要的 DOM 元素或事件监听器，可能导致这些 DOM 元素无法被垃圾回收。

3. 怎么避免内存泄漏

- 手动清理引用：**当不再需要某个对象或变量时，显式地将其设置为 `null` 或 `undefined`，帮助垃圾回收机制回收。
- 避免全局变量：**尽量避免使用全局变量，使用局部变量和模块化方法，减少不必要的内存占用。

- **清除事件监听器和定时器**：使用定时器或事件监听器后，记得在适当时机清除它们，防止内存泄漏。
- **使用弱引用**：一些语言提供了弱引用（如 JavaScript 中的 `WeakMap`），可以在不强制保持引用的情况下避免内存泄漏。

4. 怎么检查内存泄漏

内存泄漏通常由不正确的资源管理、未清除的事件监听器或未删除的 DOM 元素引起。通过**代码审查和单元测试**来检查这些潜在的内存泄漏问题。

- **事件监听器**：确保不再使用时移除事件监听器。
- **定时器**：清理不再需要的 `setInterval` 或 `setTimeout`。
- **DOM 元素**：确保不再使用的 DOM 元素被移除。

Q13：垃圾回收机制的原理是什么？

垃圾回收（Garbage Collection，GC）是自动管理内存的一种机制，目的是自动回收不再使用的内存空间，避免内存泄漏。在现代编程语言中（如 JavaScript、Java、Python 等），垃圾回收机制能够自动处理内存的分配和回收，开发者无需手动管理内存。

垃圾回收的基本原理

垃圾回收的核心目的是通过跟踪程序中的对象和数据，识别出不再被使用的对象，并回收其占用的内存。垃圾回收机制通常通过以下几种技术实现：

1. 引用计数（Reference Counting）

原理：

- 每个对象维护一个引用计数器，记录有多少引用指向该对象。
- 每当有新的引用指向对象时，计数器增加；当引用被移除时，计数器减少。
- 当引用计数为 0 时，表示该对象不再被使用，可以回收其占用的内存。

缺点：

- **循环引用问题**：如果对象之间存在循环引用（如两个对象互相引用对方），即使没有外部引用指向它们，计数器也不会变为 0，导致内存泄漏。

2. 标记-清除（Mark-and-Sweep）

原理：

1. **标记阶段**：垃圾回收器从根对象（如全局对象、局部变量等）开始，通过递归遍历所有可达对象，并将它们标记为活跃对象。
2. **清除阶段**：在标记阶段完成后，回收器遍历堆中的所有对象，删除没有被标记的对象（即不再被引用的对象），并回收其占用的内存。

优点：

- 解决了引用计数的循环引用问题。
- 对象的标记和清除相对简单，适用于大多数语言。

缺点：

- 可能导致“停顿”现象，因为需要暂停程序执行来进行标记和清除操作。
- 清除阶段可能导致内存碎片化，影响性能。

3. 标记-整理 (Mark-and-Compact)

原理：

- 这是标记-清除的改进版。在标记阶段之后，垃圾回收器不仅会清除不再使用的对象，还会将剩余的对象进行整理（压缩），将它们移动到内存的一个区域，从而避免内存碎片。

优点：

- 减少了内存碎片化的问题。

缺点：

- 对内存的整理需要额外的计算，可能导致更长的暂停时间。

4. 引用分代 (Generational Garbage Collection)

原理：

- 基于“年轻一代”和“老年代”概念，假设大多数对象生命周期较短。
- 内存分为多个代（通常是年轻代和老年代），垃圾回收器对不同代的对象采用不同的回收策略：
 - **年轻代**：存放生命周期较短的对象，通常使用**Minor GC**回收，频繁回收。
 - **老年代**：存放生命周期较长的对象，通常使用**Major GC**或**Full GC**回收，回收频率较低。

优点：

- 通过分代策略提高了垃圾回收的效率。年轻代对象回收频繁，老年代对象回收较少。
- 减少了不必要的垃圾回收操作，提升了性能。

缺点：

- 需要对不同代的对象进行跟踪和管理，增加了复杂性。

5. 分代回收与增量回收 (Incremental GC)

原理：

- **增量回收**：垃圾回收器将回收过程分成多个小步骤，逐步进行，而不是一次性回收。这减少了回收过程中的停顿时间。
- **分代回收**：将内存分为多个代，并采用不同的策略对不同代的对象进行回收，以提高效率。

优点：

- 降低了长时间的停顿，提高了应用程序的响应速度，尤其在实时系统中非常有用。

缺点：

- 实现复杂，需要不断追踪不同代的对象。

6. 弱引用与垃圾回收

原理：

- **弱引用 (Weak Reference)**：不同于普通的强引用，弱引用不会阻止对象被垃圾回收。当一个对象只有弱引用时，它可以被回收，即使弱引用仍然存在。
- 常用于缓存系统和对象池，避免因强引用导致对象不能及时被回收。

总结

垃圾回收机制的原理基于以下几个基本思想：

- **跟踪引用**：通过标记哪些对象被引用，识别哪些对象不再使用，从而回收它们。
- **分代回收**：对象的生命周期决定了其回收频率，年轻对象回收更频繁，老对象回收较少。
- **内存压缩和整理**：防止内存碎片化，提高回收效率。

Q14：防抖和节流的定义？

防抖 (Debouncing) :

- 防抖是指在一段时间内，某个事件（如用户输入、滚动等）触发多次时，只有最后一次触发会执行回调函数。换句话说，事件触发后，等待一定时间，如果在这段时间内没有再次触发该事件，才会执行回调。
- **用途**：防止短时间内多次触发某个操作，比如输入框自动搜索、按钮点击等。

节流 (Throttling) :

- 节流是指限制某个事件在一定时间内只能触发一次，确保事件回调不会频繁执行。即使事件被频繁触发，回调也会按照设定的间隔时间进行执行。
- **用途**：控制高频率事件的执行，比如滚动事件、窗口调整大小等。

Q15：怎么用防抖和节流实现超过时间n之后强制执行？

```
function debounceThrottleWithForce(func, delay, n) {
  let timer = null;
  let lastInvokeTime = 0; // 上次强制执行的时间
  let lastTime = 0; // 上次事件触发的时间
  let forceExecuted = false; // 是否强制执行过

  return function() {
    const now = Date.now();
    const context = this;
    const args = arguments;

    // 每次触发事件时，更新最后触发的时间
    lastTime = now;

    // 如果超过n毫秒没有触发事件，强制执行回调
    if (now - lastInvokeTime >= n && !forceExecuted) {
      func.apply(context, args); // 强制执行
      lastInvokeTime = now; // 更新强制执行时间
      forceExecuted = true; // 标记为已强制执行
    }

    // 否则进行防抖和节流处理
    if (!timer) {
      timer = setTimeout(function() {
        func.apply(context, args);
        lastInvokeTime = Date.now(); // 更新强制执行时间
        forceExecuted = false; // 重置强制执行标记
        timer = null; // 清除定时器
      }, delay);
    }
  };
}
```

3.25美团二面

1. 面试题

Q1：讲一下车间调度问题

Q2：如果进程A依赖进程B，进程B依赖进程C，进程B依赖进程D，该用什么数据结构？

对于这种 **进程依赖关系** 的问题，适合使用 **有向图 (Directed Graph)**，特别是 **有向无环图 (DAG, Directed Acyclic Graph)**，来表示依赖关系。

1. 解释为什么用有向图

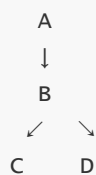
- 进程依赖关系可以表示为节点（进程）和边（依赖）：
 - 节点 (Node)：代表进程（如 A、B、C、D）。
 - 边 (Edge)：表示进程的依赖关系（如 $A \rightarrow B$ 表示 A 依赖 B）。
- 方向性：
 - 依赖关系具有方向，例如 A **依赖** B，则意味着 B **必须先完成**，才能执行 A。
- 无环性 (Acyclic)：
 - 依赖关系不应该出现循环依赖 (Deadlock)，否则进程无法执行。例如：
 - A 依赖 B，B 依赖 C，C 又依赖 A，就会形成 **循环依赖**，导致进程阻塞，无法启动。

2. 例子：有向无环图 (DAG) 表示依赖

根据你的描述：

- A 依赖 B ($A \rightarrow B$)
- B 依赖 C 和 D ($B \rightarrow C, B \rightarrow D$)

可以用 **DAG** 表示：



在 **代码** 里，这可以用 **邻接表 (Adjacency List)** 来存储：

```
const dependencies = {  
  "A": ["B"],  
  "B": ["C", "D"],  
  "C": [],  
  "D": []  
};
```

或者使用 **邻接矩阵 (Adjacency Matrix)**：

	A	B	C	D
A	0	1	0	0
B	0	0	1	1
C	0	0	0	0
D	0	0	0	0

3. 如何处理进程依赖？

- 拓扑排序 (Topological Sorting) :
 - 适用于 DAG，可以找到进程的执行顺序。
 - 典型方法：
 - **Kahn's Algorithm (基于入度)**：使用队列逐步执行 **无前置依赖** 的进程。
 - **DFS 逆后序遍历**：从没有依赖的进程开始，递归执行依赖进程。

示例 **拓扑排序**：

```
function topologicalSort(dependencies) {
  let inDegree = {}; // 记录每个节点的入度
  let graph = {}; // 邻接表
  let queue = [];
  let order = [];

  // 初始化
  for (let key in dependencies) {
    graph[key] = dependencies[key];
    inDegree[key] = 0;
  }

  // 计算入度
  for (let key in graph) {
    for (let dep of graph[key]) {
      inDegree[dep] = (inDegree[dep] || 0) + 1;
    }
  }

  // 找到入度为 0 的节点（即没有依赖的进程）
  for (let key in inDegree) {
    if (inDegree[key] === 0) {
      queue.push(key);
    }
  }

  // Kahn 算法执行拓扑排序
  while (queue.length > 0) {
    let process = queue.shift();
    order.push(process);

    for (let dep of graph[process]) {
      inDegree[dep]--;
      if (inDegree[dep] === 0) {
        queue.push(dep);
      }
    }
  }
}
```

```
return order.length === Object.keys(dependencies).length ? order : "存在循环依赖";
}

// 测试
const dependencies = {
  "A": ["B"],
  "B": ["C", "D"],
  "C": [],
  "D": []
};

console.log(topologicalSort(dependencies));
// 可能输出: ["C", "D", "B", "A"]
```

4. 总结

- 进程依赖问题适合 **有向无环图 (DAG)**。
- 可以使用 **邻接表** 或 **邻接矩阵** 存储依赖关系。
- 使用 **拓扑排序 (Kahn 算法或 DFS)** 解决进程调度问题，确保按正确顺序执行进程。
- 如果检测到 **环 (循环依赖)**，则必须重新设计依赖关系，否则进程将死锁。

Q3: 开发过程中遇到什么bug?

Q4: 遇到问题怎么去解决的?

Q5: 有用过什么社区吗? 国外的呢?

Q6: LRU怎么实现? 用什么数据结构?

LRU (Least Recently Used)，即**最近最少使用**，是一种缓存淘汰策略。

它的核心思想是：**当缓存满了时，优先移除最久未被使用的元素**，以便给新的数据腾出空间。

1. LRU 用什么数据结构?

LRU 需要高效支持：

- **O(1) 时间复杂度**的 **查找**（是否命中缓存）。
- **O(1) 时间复杂度**的 **插入/更新**（更新最近使用的元素）。
- **O(1) 时间复杂度**的 **删除**（删除最久未使用的元素）。

使用：

1. **哈希表 (HashMap)**：提供 **O(1) 时间复杂度**的查找能力。
2. **双向链表 (Doubly Linked List)**：
 - **头部** 存放 **最近使用** 的元素。
 - **尾部** 存放 **最久未使用** 的元素。
 - 插入、删除都是 **O(1) 时间复杂度**。

HashMap + 双向链表 结合，可以在 **O(1) 时间复杂度**内完成 LRU 逻辑。

2. LRU 代码实现 (JavaScript)

```
class LRUCache {
  constructor(capacity) {
    this.capacity = capacity; // 缓存的最大容量
    this.map = new Map(); // 用 Map 作为哈希表
  }

  get(key) {
    if (!this.map.has(key)) return -1; // 如果 key 不存在, 返回 -1

    // 将 key 移动到 Map 末尾, 表示最近使用
    let value = this.map.get(key);
    this.map.delete(key);
    this.map.set(key, value);
    return value;
  }

  put(key, value) {
    if (this.map.has(key)) {
      this.map.delete(key); // 删除旧数据
    }

    this.map.set(key, value); // 插入新数据

    if (this.map.size > this.capacity) {
      // 删除最久未使用的元素 (Map 的第一个元素)
      let firstKey = this.map.keys().next().value;
      this.map.delete(firstKey);
    }
  }
}

// 使用示例:
let lru = new LRUCache(2);
lru.put(1, "A"); // 添加 1:A
lru.put(2, "B"); // 添加 2:B
console.log(lru.get(1)); // 访问 1 -> 返回 A, 并将 1 移动到最近使用
lru.put(3, "C"); // 插入 3:C, 导致 2:B 被淘汰 (最久未使用)
console.log(lru.get(2)); // -1 (2 被淘汰)
```

3. LRU 的时间复杂度

操作	数据结构	时间复杂度
查询	HashMap	O(1)
插入	HashMap + 双向链表	O(1)
删除	双向链表	O(1)

4. 总结

LRU (最近最少使用) 缓存的核心是:

- ✓ 双向链表 (维护访问顺序)。
- ✓ HashMap (快速查找 $O(1)$)。
- ✓ 淘汰最久未使用的元素 (LRU 规则)。

Q7: LFU?

LFU (Least Frequently Used), 即最不经常使用, 是一种缓存淘汰策略。

它的核心思想是: 当缓存满了时, 优先移除使用频率最低的元素, 以便给新的数据腾出空间。

1. LFU 需要什么数据结构?

LFU 需要支持:

- $O(1)$ 时间复杂度的查找 (是否命中缓存)。
- $O(1)$ 时间复杂度的插入/更新 (更新访问频率)。
- $O(1)$ 时间复杂度的删除 (删除最少使用的元素)。

为了满足这些需求, 我们使用:

1. 哈希表 (HashMap) :
 - 存储 $\text{key} \rightarrow \{ \text{value}, \text{freq} \}$, 快速获取缓存值和使用频率。
 - 存储 $\text{freq} \rightarrow \{ \text{keys} \}$, 快速查找当前访问频率最小的元素。
2. 双向链表 (LinkedHashSet) :
 - 维护相同访问频率的 key, 保证 $O(1)$ 删除。

2. LFU 代码实现 (JavaScript)

```
class LFUCache {
  constructor(capacity) {
    this.capacity = capacity; // 缓存最大容量
    this.minFreq = 0; // 当前最小频率
    this.keyToVal = new Map(); // 存储 key → { value, freq }
    this.freqToKeys = new Map(); // 存储 freq → { keys }
  }

  get(key) {
    if (!this.keyToVal.has(key)) return -1;

    let { value, freq } = this.keyToVal.get(key);
    this._updateFreq(key, freq);
    return value;
  }

  put(key, value) {
    if (this.capacity === 0) return;

    if (this.keyToVal.has(key)) {
      this.keyToVal.get(key).value = value;
      this.get(key); // 增加访问频率
      return;
    }
  }
}
```

```

    }

    if (this.keyToVal.size >= this.capacity) {
        this._evict();
    }

    // 插入新数据
    this.keyToVal.set(key, { value, freq: 1 });
    if (!this.freqToKeys.has(1)) this.freqToKeys.set(1, new Set());
    this.freqToKeys.get(1).add(key);
    this.minFreq = 1;
}

_updateFreq(key, freq) {
    this.freqToKeys.get(freq).delete(key);
    if (this.freqToKeys.get(freq).size === 0) {
        this.freqToKeys.delete(freq);
        if (this.minFreq === freq) this.minFreq++;
    }

    let newFreq = freq + 1;
    if (!this.freqToKeys.has(newFreq)) this.freqToKeys.set(newFreq, new Set());
    this.freqToKeys.get(newFreq).add(key);
    this.keyToVal.get(key).freq = newFreq;
}

_evict() {
    let keys = this.freqToKeys.get(this.minFreq);
    let oldestKey = keys.values().next().value;
    keys.delete(oldestKey);
    if (keys.size === 0) this.freqToKeys.delete(this.minFreq);
    this.keyToVal.delete(oldestKey);
}
}

// 使用示例:
let lfu = new LFUCache(2);
lfu.put(1, "A");
lfu.put(2, "B");
console.log(lfu.get(1)); // 访问 1, 更新频率
lfu.put(3, "C"); // 淘汰 2 (使用频率最低)
console.log(lfu.get(2)); // -1 (被淘汰)

```

3. LFU 的时间复杂度

操作	数据结构	时间复杂度
查询	HashMap	O(1)
插入	HashMap + LinkedHashSet	O(1)
删除	HashMap + LinkedHashSet	O(1)

4. LFU vs LRU

策略	LRU (最近最少使用)	LFU (最不经常使用)
规则	淘汰最近未使用的元素	淘汰使用频率最低的元素
数据结构	HashMap + 双向链表	HashMap + LinkedHashSet
优点	适合短期热点数据	适合长期稳定访问的场景
缺点	容易被短期热点干扰	维护频率增加复杂度

5. 总结

- ✓ LFU 需要维护访问频率，因此使用 HashMap + LinkedHashSet 结构。
- ✓ 每次访问或插入时更新频率，保证最不经常使用的数据被淘汰。
- ✓ Java 可以用 TreeMap 或者 LinkedHashSet 来实现。
- ✓ 相比 LRU，LFU 更适合长期稳定访问的数据。

Q8：用过什么设计者模式？

在前端和后端开发中，常用的设计模式包括 **单例模式**、**工厂模式**、**观察者模式**、**策略模式**、**代理模式**、**装饰器模式** 等。

1. 单例模式 (Singleton)

作用：确保一个类只有一个实例，并提供全局访问点。

前端应用场景：

- **Vuex** (全局状态管理)
- **缓存管理**
- **Axios 请求实例**

示例 (JavaScript)：

```
class Singleton {
  constructor() {
    if (!Singleton.instance) {
      Singleton.instance = this;
    }
    return Singleton.instance;
  }
}

const instance1 = new Singleton();
const instance2 = new Singleton();
console.log(instance1 === instance2); // true
```

2. 工厂模式 (Factory)

作用：提供一个创建对象的接口，而无需指定具体的类。

前端应用场景：

- Vue / React 组件工厂
- Axios API 封装
- ElementUI、Ant Design 组件创建

示例 (JavaScript) :

```
class User {
  constructor(name, role) {
    this.name = name;
    this.role = role;
  }
}

class UserFactory {
  createUser(name, role) {
    return new User(name, role);
  }
}

const factory = new UserFactory();
const user1 = factory.createUser("Alice", "Admin");
const user2 = factory.createUser("Bob", "User");
console.log(user1, user2);
```

3. 观察者模式 (Observer)

作用：一个对象（被观察者）维护一组观察者，当自身状态变化时通知所有观察者。

前端应用场景：

- Vue 组件通信 (事件总线)
- Redux 订阅模式
- WebSocket 消息推送
- MutationObserver 监听 DOM 变化

示例 (JavaScript) :

```
class EventEmitter {
  constructor() {
    this.events = {};
  }

  on(event, listener) {
    if (!this.events[event]) {
      this.events[event] = [];
    }
    this.events[event].push(listener);
  }

  emit(event, data) {
    if (this.events[event]) {
```

```
        this.events[event].forEach(listener => listener(data));
    }
}
}

const eventBus = new EventEmitter();
eventBus.on("message", data => console.log("收到消息: ", data));
eventBus.emit("message", "Hello, Observer!");
```

4. 策略模式 (Strategy)

作用：定义一组算法，将它们封装起来，并使它们可以互相替换。

前端应用场景：

- 表单校验（不同校验规则）
- 支付方式（微信支付、支付宝支付、银行卡支付）
- 不同用户角色的权限管理

示例 (JavaScript)：

```
const strategies = {
  "wechat": () => console.log("微信支付"),
  "alipay": () => console.log("支付宝支付"),
  "bank": () => console.log("银行卡支付"),
};

function pay(type) {
  strategies[type] ? strategies[type]() : console.log("不支持该支付方式");
}

pay("wechat"); // 微信支付
pay("alipay"); // 支付宝支付
```

5. 代理模式 (Proxy)

作用：为其他对象提供一个代理，控制对原对象的访问。

前端应用场景：

- Vue3 的 Proxy 代替 Object.defineProperty
- 虚拟代理（懒加载图片）
- API 代理（拦截请求，防止频繁请求）

示例 (JavaScript)：

```
const target = { message: "Hello, Proxy!" };

const handler = {
  get: (obj, prop) => {
    console.log(`访问了属性 ${prop}`);
    return obj[prop];
  },
};

const proxy = new Proxy(target, handler);
console.log(proxy.message); // 访问了属性 message
```

6. 装饰器模式 (Decorator)

作用：动态地给对象添加新的功能，而不会改变其结构。

前端应用场景：

- Vue 的 mixins
- ES7 装饰器

示例 (JavaScript, ES7 装饰器)：

```
function log(target, name, descriptor) {
  const original = descriptor.value;
  descriptor.value = function (...args) {
    console.log(`调用方法 ${name}, 参数: `, args);
    return original.apply(this, args);
  };
}

class MathOperations {
  @log
  add(a, b) {
    return a + b;
  }
}

const math = new MathOperations();
console.log(math.add(2, 3)); // 调用方法 add, 参数: [2,3]
```

总结

设计模式	作用	前端应用场景
单例模式	确保一个类只有一个实例	Redux Store、Axios 单例
工厂模式	创建对象的接口	组件工厂、API 请求工厂
观察者模式	发布-订阅	事件总线、WebSocket
策略模式	允许动态切换算法	表单校验、支付方式
代理模式	代理访问对象	Vue3 Proxy、API 请求拦截
装饰器模式	动态添加功能	React 高阶组件、Vue mixins

Q9：策略模式的具体使用场景？

Q10：DNS劫持是什么意思？常见的DNS劫持有哪些？该怎么防范？

DNS劫持是什么意思？

DNS劫持（DNS Hijacking）是指**攻击者通过篡改DNS解析结果**，使用户访问的域名被重定向到恶意网站，而非用户期望的正常网站。

DNS 劫持可能会导致：

- 访问假冒网站（如伪造的银行、支付平台，窃取账号密码）。
- 跳转至带有恶意广告、病毒的网页。
- 阻止用户访问特定网站（如政府审查、ISP 强制跳转）。

常见的 DNS 劫持类型

1. 本地 DNS 劫持

攻击方式：

- 恶意软件或病毒修改本机的 **hosts 文件** 或 **DNS 设置**，让域名解析到攻击者的 IP 地址。

特点：

- 仅影响被感染的设备。
- 可通过检查本地 DNS 配置或 `hosts` 文件修复。

防范方法：

- 定期检查 `C:\windows\System32\drivers\etc\hosts` (Windows) 或 `/etc/hosts` (Linux/macOS) 。
- 确保 **DNS 设置正确**，避免被篡改 (Windows 可运行 `ipconfig /all` 检查 DNS) 。
- 使用防病毒软件检测木马程序。

2. 路由器 DNS 劫持

攻击方式：

- 攻击者入侵用户的 Wi-Fi 路由器，修改其 DNS 设置，使所有连接设备的域名解析到错误 IP。

特点：

- 影响整个局域网内的设备（手机、电脑、平板等）。

- 可能通过公共 Wi-Fi 进行劫持，尤其是 **未修改默认管理员密码的路由器**。

防范方法：

- **修改默认路由器管理员密码**，避免被暴力破解。
 - 禁止远程管理（关闭 **WAN 远程管理**）。
 - 固件升级，防止漏洞利用。
 - 手动设置可信 DNS 服务器，如：
 - **Google DNS**: 8.8.8.8 / 8.8.4.4
 - **Cloudflare DNS**: 1.1.1.1 / 1.0.0.1
 - **OpenDNS**: 208.67.222.222 / 208.67.220.220
-

3. ISP（运营商）DNS 劫持

攻击方式：

- 互联网服务提供商（ISP，如某些宽带运营商）强行修改用户 DNS 请求，可能：
 - 插入广告。
 - 强制跳转至指定网站。
 - 屏蔽访问特定网站。

特点：

- 影响范围较广（整个网络）。
- 用户通常无法直接修复。

防范方法：

- 使用加密 DNS（DNS over HTTPS / DNS over TLS），如：
 - **Cloudflare WARP (1.1.1.1)**
 - **Google DoH**
 - **使用 VPN**，避免本地 ISP 劫持流量。
 - 修改 DNS 服务器，避免使用 ISP 提供的默认 DNS。
-

4. 服务器端 DNS 劫持

攻击方式：

- 攻击者入侵 **权威 DNS 服务器**，篡改域名解析，导致所有用户访问错误站点。

特点：

- 影响范围极大（全球范围内访问该域名的用户）。
- 可能导致大规模**钓鱼攻击**、**数据泄露**。

防范方法：

- 网站管理员

应使用：

- **DNSSEC**（DNS 安全扩展）防止解析篡改。
 - **双因素认证 (2FA)** 保护 DNS 账户。
 - **启用 Cloudflare / 阿里云 DNS 保护**。
-

如何检测是否遭遇 DNS 劫持？

1. Ping 检测：

```
ping google.com
```

如果返回 IP 地址异常（与 Google 官方 IP 不符），可能被劫持。

2. Nslookup 解析检查（Windows/Linux/macOS 通用）：

```
nslookup google.com
```

- 如果 DNS 服务器不是自己设定的，可能被篡改。
- 比较 `nslookup google.com` 和 `nslookup google.com 8.8.8.8` 是否返回一致。

3. 访问安全 DNS 测试：

- 访问 <https://dnsleaktest.com> 检查 DNS 解析服务器是否正常。
- 访问 [Cloudflare DNS Check](#) 检查是否启用加密 DNS。

如何防范 DNS 劫持？

劫持类型	防范措施
本地 DNS 劫持	- 定期检查 <code>hosts</code> 文件 - 使用防病毒软件 - 确保 DNS 设置正确
路由器 DNS 劫持	- 修改默认路由器密码 - 关闭远程管理 - 使用可信 DNS（Google、Cloudflare）
ISP DNS 劫持	- 使用 加密 DNS（DoH/DoT） - 使用 VPN 避免流量监控
服务器端 DNS 劫持	- 网站管理员应启用 DNSSEC - 保护 DNS 账户 - 使用 Cloudflare DNS 保护

总结

DNS 劫持是一种严重的网络安全威胁，可能导致隐私泄露、钓鱼诈骗甚至大规模攻击。

防范关键点：

- 修改默认 DNS 服务器（使用 1.1.1.1 / 8.8.8.8）。
- 使用加密 DNS（DoH/DoT），避免 ISP 劫持。
- 定期检查路由器和本机 DNS 设置，防止恶意篡改。
- 谨慎连接公共 Wi-Fi，避免恶意 DNS 劫持。

如果发现 DNS 被劫持，建议清除 DNS 缓存 并重启设备：

```
# windows 清除 DNS 缓存
ipconfig /flushdns

# macOS 清除 DNS 缓存
sudo killall -HUP mDNSResponder

# Linux 清除 DNS 缓存（基于 systemd）
sudo systemd-resolve --flush-caches
```

Q11: 前端开发过程中遇到过死锁吗？JS单线程会造成死锁吗？

在前端开发过程中，很少会遇到真正意义上的“死锁（Deadlock）”，但可能会遇到一些类似的阻塞问题，例如：

1. **Web Worker 线程间的互相等待**（使用 `postMessage` 但未正确处理）。
2. **异步回调陷入循环依赖**（`Promise / async-await`）。
3. **多 `setTimeout / requestAnimationFrame` 任务互相依赖，导致 UI 卡死。**

虽然这些情况不会像多线程编程那样导致真正的“死锁”，但它们可能会导致**页面无响应（UI 假死）**，甚至崩溃。

JS 单线程会造成死锁吗？

JS 是**单线程**的，通常不会造成传统的**死锁（Deadlock）**，但会出现**类似的“阻塞”问题**，主要有以下几种情况：

1. 同步代码阻塞（主线程卡死）

JS 是单线程的，所有任务都在**同一个 Event Loop** 中运行。如果有**长时间运行的同步任务**，会阻塞后续任务，使 UI 无法更新，类似“死锁”：

```
while (true) {} // 无限循环，导致浏览器卡死
```

解决方案：

- **拆分任务**（使用 `setTimeout`、`requestIdleCallback`）。
- **使用 Web Worker 处理计算密集型任务。**

2. `await / Promise` 互相等待（异步死锁）

JS 虽然是单线程，但 `async/await` 和 `Promise` 可能导致**互相等待**，造成**类似死锁**：

```
async function task1() {
  console.log("task1 开始");
  await task2();
  console.log("task1 结束"); // 永远不会执行
}

async function task2() {
  console.log("task2 开始");
  await task1(); // task2 等待 task1 结束
  console.log("task2 结束"); // 也不会执行
}

task1();
```

解释：

- `task1` 等 `task2` 结束，`task2` 又等 `task1` 结束，造成**循环等待**，类似死锁。

解决方案：

- 避免循环依赖。
- 设置超时控制（`Promise.race`）。

3. Web Worker 互相等待

虽然 Web Worker 让 JS 具备了多线程能力，但如果两个 Worker 相互等待对方的消息，就会产生“死锁”：

```
// worker1.js
self.onmessage = function(e) {
  self.postMessage("给 worker2 发送消息");
};

// worker2.js
self.onmessage = function(e) {
  self.postMessage("给 worker1 发送消息");
};
```

如果 Worker1 先等待 Worker2，Worker2 也在等待 Worker1，就会陷入死循环，两个 Worker 都无法继续执行。

解决方案：

- 使用超时检测（如 `setTimeout` 限制等待时间）。
- 使用消息队列，避免循环依赖。

总结

情况	会死锁吗？	解决方案
同步代码阻塞（无限循环）	☑ 可能导致 UI 卡死	拆分任务 / <code>setTimeout</code>
async / await 互相等待	☑ 类似死锁	避免循环依赖 / 设置超时
Web Worker 互相等待	☑ 可能导致死锁	设超时检测 / 用消息队列

结论：

- JS 由于是单线程，不会产生传统多线程的死锁。
- 但 异步任务互相等待、事务锁等情况 可能导致类似死锁。
- 合理使用 `setTimeout`、消息队列、超时检测，可以避免 JS 代码“假死”。

2.代码题

Q1：单词遍历

一个词典库，假设有10W单词，想把所有单词这样一组一组的找出来：长度相同，并且所含字母相同（所含字母数量也要相同）。

如：cat、act、tca；ate、eat、tea；demo、dome、mode。

```
function groupwordsByAnagram(words){
  const groups = {};
  //遍历每个单词
  for (const word of words) {
    //将单词的字母排序，作为分组的键
    const sortedword = word.split('').sort().join('');
    //如果键不存在，创建一个新的数组
```

```

    if (!groups[sortedword]){
      groups[sortedword] = [];
    }

    //将单词加入对应的分组
    groups[sortedword].push(word);
    //返回分组结果
    return Object.values(groups);
  }

```

Q2:不用排序怎么实现单词遍历呢?

```

function groupWordsByAnagramNoSort(words){
  const groups = {};
  //遍历每个单词
  for (const word of words) {
    //构造字母频率计数数组
    const charCount = new Array(26).fill(0); //假设单词只包含小写字母
    for (const char of word) {
      charCount[char.charCodeAt(0) - 'a'.charCodeAt(0)]++;
    }
    //将字母频率数组转换为字符串作为键
    const key = charCount.join('#'); //使用"#"分隔，避免数字混淆
    //如果键不存在，创建一个新的数组
    if (!groups[key]) {
      groups[key] = [];
    }
    //将单词加入对应的分组
    groups[key].push(word);
    //返回分组结果
    return Object.values(groups);
  }
}

```

3.31美的面试

Q1：后端接口没开发好时，怎么测试

1. Mock 数据

- 使用 **Mock 服务**：如 `Mock.js`、`json-server`、`Easy Mock` 等，可以模拟接口返回的数据格式，使前端开发不受后端进度影响。
- **前端代码内 Mock**：在 `Vue` 或 `React` 项目中，可以在 `axios` 拦截器或 `fetch` 的 `then` 处理中直接返回模拟数据。

2. 使用 API 测试工具

- **Apifox**：支持 API 文档、接口 Mock、自动化测试等功能，前端可直接调用 Mock 地址，避免依赖后端。
- **Postman**：可手动发送 HTTP 请求，模拟后端接口返回。
- **Swagger**：如果后端提供了 Swagger 文档，可以直接在其 UI 里进行测试。

3. 使用本地开发服务器

- **json-server**: 基于 `db.json` 模拟 RESTful API, 适合前端调试。
- **FastAPI/Fake API**: 用 Python、Node.js 快速搭建一个临时 API 服务。

4. 使用代理拦截

- **使用 Nginx 代理**: 拦截前端请求并返回本地 Mock 数据。
- **使用 Charles、Fiddler** 等抓包工具拦截请求, 并自定义返回数据。

Q2: http状态码

2xx: 成功响应

200OK: 请求成功, 返回正常数据。

201 Created: 资源创建成功(如 POST 创建资源)。

202Accepted: 请求已接受, 但尚未处理完成。

204NoContent:请求成功, 但无返回内容。

3xx: 重定向

301 Moved Permanently: 资源永久重定向到新位置。

302 Found: 资源临时重定向到新位置。

304 Not Modified: 资源未修改, 可使用缓存。

4xx: 客户端错误

400 Bad Request: 请求参数错误或语法错误。

401 Unauthorized: 未授权, 需身份验证。

403 Forbidden: 服务器拒绝执行请求, 无权限。

404 Not Found: 请求的资源不存在。

405 Method Not Allowed: 请求方法不被允许 (如用 GET 调用 POST 接口) 。

409 Conflict: 请求冲突 (如资源已存在) 。

422 Unprocessable Entity: 请求格式正确, 但语义错误 (常用于表单验证失败) 。

5xx: 服务器错误

500 Internal Server Error: 服务器内部错误。

502 Bad Gateway: 网关错误, 服务器无效响应。

503 Service Unavailable: 服务不可用 (可能超载或维护中) 。

504 Gateway Timeout: 网关超时, 未收到服务器响应。

Q3: http状态码错误怎么引导用户操作

1. 4xx 客户端错误 (用户请求问题)

400 Bad Request (请求错误)

◆ **原因**: 参数格式不正确、必填字段缺失等。

☑ **引导方案**:

- 提示用户检查输入内容是否正确, 并提供格式示例。
- 例如: “请输入正确的手机号 (11位数字)”。

401 Unauthorized (未授权)

◆ **原因：**用户未登录或身份验证失败。

✓ **引导方案：**

- 跳转到登录页，提示用户登录后重试。
- 例如：“您的登录已过期，请重新登录”。

```
import axios from 'axios';
import { useRouter } from 'vue-router';
import { ElMessage } from 'element-plus'; // 适用于 Element Plus UI 组件

const router = useRouter();

// 创建 Axios 实例
const api = axios.create({
  baseURL: 'https://your-api.com',
  timeout: 5000,
});

// 请求拦截器（可添加 Token）
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token'); // 获取 Token
    if (token) {
      config.headers.Authorization = `Bearer ${token}`; // 携带 Token
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// 响应拦截器（处理 401 错误）
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      ElMessage.error('您的登录已过期，请重新登录'); // 显示错误提示
      localStorage.removeItem('token'); // 清除 Token
      router.push('/login'); // 跳转到登录页
    }
    return Promise.reject(error);
  }
);

export default api;
```

403 Forbidden (无权限)

◆ **原因：**用户无权访问资源，如未授权操作。

✓ **引导方案：**

- 显示 **无权限提示**，如“您没有权限执行此操作”。
- 引导用户 **联系管理员** 或 **切换账号**。

404 Not Found (资源不存在)

◆ **原因：**请求的接口、页面或资源不存在。

☑ **引导方案：**

- **跳转到首页或返回上一级**，显示友好提示“**页面不存在，返回首页**”。
- 提供搜索框或推荐相关内容。

2. 5xx 服务器错误 (系统问题)

500 Internal Server Error (服务器内部错误)

◆ **原因：**服务器崩溃、代码错误等。

☑ **引导方案：**

- **提示用户稍后重试**，如“**系统开小差了，请稍后再试**”。
- 记录错误日志，后台监控告警。

502 Bad Gateway (网关错误)

◆ **原因：**服务器代理或网关异常。

☑ **引导方案：**

- **提示用户刷新页面 或 稍后重试**。

503 Service Unavailable (服务不可用)

◆ **原因：**服务器维护或过载。

☑ **引导方案：**

- **提供维护通知**，如“**系统维护中，预计 12:00 完成**”。

Q4: websocket与http流式数据传播的区别

1. WebSocket

☑ **特点**

- **全双工通信** (双向实时通信)
- **长连接** (只需建立一次连接)
- **低延迟** (减少 HTTP 轮询开销)
- 适用于**高频、双向交互**场景，如**在线聊天、实时数据推送、在线游戏**等。

🌀 **工作方式**

1. 客户端与服务器通过 **HTTP 握手** (Upgrade: websocket)。
2. 连接建立后，客户端和服务器可以**双向发送消息**，不需要重复建立连接。
3. **数据以帧 (frame) 格式传输**，减少开销 (比 HTTP 头部小)。
4. 连接持续存在，直到 **客户端或服务器主动断开**。

📌 应用场景

- 即时通讯（微信、QQ Web 版）
- 股票行情、体育赛事直播（实时推送数据）
- 多人在线游戏
- 协同编辑（Google Docs、在线白板）

2. HTTP 流式数据传输

✓ 特点

- 单向数据流（通常是服务器向客户端推送）
- 基于 HTTP 长连接，但仍需 HTTP 请求
- 适用于服务端主动推送但客户端无需反馈的场景

🔧 主要方式

◆ (1) SSE (Server-Sent Events, 服务器推送事件)

- 基于 HTTP/1.1，但仅支持服务器向客户端推送数据。
- 自动重连（如果连接断开，浏览器会自动尝试重连）。
- 文本数据传输（仅支持 UTF-8），不支持二进制数据。

📌 示例

```
const eventSource = new EventSource('/sse-endpoint');
eventSource.onmessage = (event) => {
  console.log('Received:', event.data);
};
```

✓ 适用场景：微博热搜、天气推送、社交媒体通知

◆ (2) HTTP Chunked (分块传输)

- 基于 HTTP 长连接，服务器可以逐步发送数据块，适用于大文件下载或实时日志流。
- 每个数据块都有分块标识，客户端按需解析。

📌 示例

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked

5
Hello
7
, world!
0
```

✓ 适用场景：日志流、视频流、实时 AI 生成内容（如 ChatGPT 流式回复）

3. WebSocket vs HTTP 流式数据传播

特性	WebSocket	SSE (Server-Sent Events)	HTTP Chunked
通信方向	双向 (全双工)	服务器 → 客户端 (单向)	服务器 → 客户端 (单向)
协议	ws:// / wss://	http://	http://
是否需要多次请求	仅一次握手	需要请求 SSE 资源	需要请求 HTTP 资源
数据格式	二进制 / 文本	纯文本 (UTF-8)	任意格式 (分块传输)
自动重连	✗ (需手动实现)	☑ (浏览器自动支持)	✗ (需重新请求)
适用场景	即时通讯、实时游戏、协同编辑	推送通知、实时更新 (新闻、天气)	日志流、视频流、AI 流式输出

总结

- 1. **WebSocket** 适合 **双向、低延迟** 的通信，如**聊天、游戏**。
- 2. **SSE (Server-Sent Events)** 适合**服务器单向推送**，如**通知、新闻更新**。
- 3. **HTTP Chunked** 适用于**流式数据**，如**日志、视频流、ChatGPT AI 输出**。

Q5: vue2与vue3的生命周期

1. Vue 2 生命周期

阶段	Vue 2 生命周期钩子	说明
创建阶段 (beforeCreate → created)	beforeCreate	组件实例被初始化, 但 <code>data</code> 、 <code>methods</code> 还未创建。
	created	组件实例创建完成, 可访问 <code>data</code> 、 <code>methods</code> , 但 <code>DOM</code> 还未渲染。
挂载阶段 (beforeMount → mounted)	beforeMount	模板已编译, <code>render</code> 函数即将执行, 但 <code>DOM</code> 还未挂载。
	mounted	组件已挂载, <code>DOM</code> 渲染完成, 可操作 <code>this.\$el</code> 。
更新阶段 (beforeUpdate → updated)	beforeUpdate	组件数据变化, <code>DOM</code> 更新前触发。
	updated	组件数据变化, <code>DOM</code> 更新后触发。
销毁阶段 (beforeDestroy → destroyed)	beforeDestroy	组件即将销毁, 仍可访问 <code>data</code> 、 <code>methods</code> 。
	destroyed	组件销毁后, 解绑事件、销毁子组件, 数据被清理。

2. Vue 3 生命周期

Vue 2 钩子	Vue 3 组合式 API 等效钩子	说明
beforeCreate	✕ (移除)	在 <code>setup()</code> 之前, Vue 3 省略此钩子, 因 <code>setup</code> 之前数据尚未创建。
created	✕ (移除)	由于 <code>setup</code> 早于 <code>created</code> 运行, 因此不再需要该钩子。
beforeMount	onBeforeMount	组件挂载前。
mounted	onMounted	组件挂载后, 可操作 <code>DOM</code> 。
beforeUpdate	onBeforeUpdate	组件数据变更, <code>DOM</code> 更新前。
updated	onUpdated	组件数据变更, <code>DOM</code> 更新后。
beforeDestroy	onBeforeUnmount	组件销毁前。
destroyed	onUnmounted	组件销毁后, 资源清理。

3. Vue 2 vs Vue 3 生命周期对比总结

对比项	Vue 2	Vue 3
核心 API	选项式 API (<code>data()</code> , <code>methods</code> , <code>computed</code>)	组合式 API (<code>setup()</code> , <code>ref</code> , <code>reactive</code>)
创建阶段	<code>beforeCreate</code> / <code>created</code>	省略, 使用 <code>setup()</code> 代替
挂载阶段	<code>beforeMount</code> / <code>mounted</code>	<code>onBeforeMount</code> / <code>onMounted</code>
更新阶段	<code>beforeUpdate</code> / <code>updated</code>	<code>onBeforeUpdate</code> / <code>onUpdated</code>
销毁阶段	<code>beforeDestroy</code> / <code>destroyed</code>	<code>onBeforeUnmount</code> / <code>onUnmounted</code>

Q6：箭头函数与普通函数的区别

- 语法**：箭头函数更简洁，适用于短小的函数。普通函数需要 `function` 关键字，箭头函数使用 `=>` 省略 `function` 关键字。
- this 绑定**：普通函数的 `this` 由调用方式决定，可能指向不同对象；箭头函数的 `this` 继承自定义时的外层作用域，不能更改。
- arguments 绑定**：普通函数可以访问 `arguments`，而箭头函数没有 `arguments`，需要使用剩余参数 `...args` 代替。
- 对象方法**：普通函数适合作为对象方法，`this` 指向该对象；箭头函数的 `this` 不指向对象，而是继承外层作用域的 `this`。
- 构造函数**：普通函数可以使用 `new` 创建实例，箭头函数不能作为构造函数，使用 `new` 会报错。
- call、apply、bind 绑定**：普通函数可以使用 `call`、`apply`、`bind` 显式修改 `this`，箭头函数无法改变 `this` 绑定。
- return 语法**：箭头函数在单行返回时可以省略 `{}` 和 `return`，普通函数需要显式 `return`。
- 适用场景**：普通函数适用于对象方法、构造函数和需要 `this` 的情况；箭头函数适用于回调函数、定时器和短小的逻辑。

Q7：怎么看待前端技术更新

前端技术的更新是常态，**基础扎实** 才能快速适应新技术；**关注趋势但不过度追新**，结合 **项目需求** 做出合理选择。

3.31作业帮二面

1. 面试题

Q1：项目开发中遇到哪些难题?怎么克服的？

Q2：为什么选择使用vue不用react？

Q3: Vue与react的区别是什么?

Q4: vue2与vue3的区别?

Q5: 有没有使用过静态资源服务器? 它的原理是什么?

例如 **Nginx** 和 **Apache** 来托管前端静态资源 (HTML、CSS、JS、图片等)。

原理:

静态资源服务器的工作方式是**直接从磁盘读取静态文件并返回给客户端**, 而不涉及动态计算。其核心原理如下:

1. **请求解析**: 用户在浏览器访问网页时, 请求会发送到服务器 (例如 `https://example.com/index.html`)。
2. **文件查找**: 服务器会在指定的静态资源目录 (如 `public/` 或 `www/`) 查找请求的文件。
3. **响应返回**: 找到文件后, 服务器会将其以 HTTP 响应的形式返回, 并附带正确的 **Content-Type** (如 `text/html`、`application/javascript`)。
4. **缓存优化**: 通常会配合**缓存策略** (如 `Cache-Control`、`ETag`) 减少重复请求, 提高访问速度。

💡 常见的静态资源服务器:

- **Nginx**: 高性能、支持反向代理、负载均衡和 Gzip 压缩。
- **Apache**: 功能丰富, 可用于静态和动态内容的托管。
- **Node.js + Express**: 可用于小型项目的静态资源托管 (`express.static()`)。

Q6: CDN (内容分发网络) 的工作原理

CDN (Content Delivery Network) 是**加速网站访问的技术**, 它的工作原理是**在全球范围内部署缓存服务器**, 把静态资源分发到用户最近的节点, 减少服务器压力, 提高访问速度。

CDN 的核心原理:

1. **DNS 解析**: 当用户访问某个网站时 (如 `https://cdn.example.com/script.js`), DNS 解析会将请求指向离用户最近的 CDN 服务器。
2. **缓存策略**:
 - **如果 CDN 节点已有缓存**: 直接返回缓存内容, 提高访问速度。
 - **如果没有缓存**: CDN 节点向源服务器请求资源, 并将其缓存, 以便后续用户访问时使用。
3. **回源机制**: 当资源过期或缓存未命中时, CDN 向源服务器请求最新内容并更新缓存。
4. **内容加速**: CDN 服务器通常支持 **Gzip 压缩**、**HTTP/2**、**TLS 负载均衡**, 提高传输效率。

💡 CDN 的优点:

- **提高访问速度**: 用户访问最近的 CDN 节点, 减少延迟。
 - **降低服务器压力**: 减少源站负载, 提高网站稳定性。
 - **抗 DDoS 攻击**: CDN 可分散流量, 减少攻击影响。
-

Q7：顺序表与链表的区别

对比项	顺序表 (ArrayList)	链表 (LinkedList)
存储方式	连续的内存块	不连续，靠指针连接
插入/删除	低效，需移动大量元素， $O(n)$	高效，只需修改指针， $O(1)$
访问速度	$O(1)$ (数组索引访问)	$O(n)$ (需要遍历)
空间利用	需要预分配固定大小	节省空间，但额外存储指针
适用场景	适用于随机访问多的情况	适用于插入删除多的情况

💡 总结：

- 顺序表 (ArrayList) 适合随机访问，但插入删除较慢。
- 链表 (LinkedList) 适合频繁插入/删除，但查找慢。
- 实际开发中，大多数情况下 ArrayList 更常用，因为现代计算机的缓存机制让数组更高效。

Q8：了解过http网络安全相关的内容吗？

1. HTTPS 加密传输（防窃听、中间人攻击）

- 问题：HTTP 是明文传输，容易被中间人攻击 (MITM) 窃取数据。
- 防护措施：
 - 使用 HTTPS (HTTP + TLS/SSL) 加密传输数据，确保数据完整性和机密性。
 - 启用 HSTS (HTTP Strict Transport Security) 强制浏览器使用 HTTPS 访问，防止降级攻击。
 - 证书管理：使用可信 CA 机构颁发 SSL 证书，避免自签名证书带来的安全风险。

2. XSS（跨站脚本攻击，Cross-Site Scripting）

- 问题：攻击者在网页中注入恶意 JavaScript，窃取用户数据、劫持会话或执行未授权操作。
- 防护措施：
 - 输入过滤：使用正则表达式或白名单过滤用户输入，防止注入恶意脚本。
 - 输出编码：
 - HTML 编码 (<script> 变成 <script>)
 - JavaScript 编码 (escape() 或 encodeURIComponent())
 - CSP (内容安全策略)：限制页面加载的外部资源 (如 script-src 限制 JS 只能来自可信源)。
 - HttpOnly Cookie：防止 JavaScript 读取敏感 Cookie 数据，避免劫持。

3. CSRF（跨站请求伪造，Cross-Site Request Forgery）

- 问题：攻击者诱导用户访问恶意页面，使其浏览器在用户不知情的情况下执行未授权请求 (如转账、修改密码)。
- 防护措施：
 - CSRF Token：每次请求附带唯一的 Token，服务器验证 Token 有效性。
 - SameSite Cookie：

- `SameSite=Strict`：禁止跨站请求携带 Cookie。
 - `SameSite=Lax`：仅允许部分跨站请求（如 GET）。
 - **Referer 头检查**：服务器检查请求来源是否可信。
 - **验证码 (CAPTCHA)**：增加用户交互，防止自动化攻击。
-

4. SQL 注入 (SQL Injection)

- **问题**：攻击者通过输入恶意 SQL 语句，篡改数据库、获取敏感数据或执行恶意操作。
 - **防护措施**：
 - **使用预处理语句 (Prepared Statements)**，避免直接拼接 SQL 语句。
 - **参数化查询**（如 `MyBatis`、`Hibernate` 提供的 `?` 占位符）。
 - **输入校验**：限制输入字符，如禁止 `--`、`' OR '1'='1` 这类 SQL 语句。
 - **最小权限原则**：数据库用户只授予必要权限，防止越权访问。
-

5. 点击劫持 (Clickjacking)

- **问题**：攻击者利用 `<iframe>` 嵌套目标网站，诱导用户点击 **透明按钮**，执行未授权操作。
 - **防护措施**：
 - **X-Frame-Options 头**：
 - `DENY`：禁止嵌入 iframe。
 - `SAMEORIGIN`：仅允许同源 iframe。
 - **CSP (Content Security Policy)**：`frame-ancestors` 限制可嵌入的页面来源。
-

6. HTTP 请求劫持 (CRLF Injection)

- **问题**：攻击者在 URL 参数中插入换行符 (`\r\n`)，伪造 HTTP 头，导致响应劫持。
 - **防护措施**：
 - 过滤输入，禁止 `\r` (回车) 和 `\n` (换行)。
 - 服务器使用 **严格的 HTTP 解析规则**，避免解析漏洞。
-

7. 不安全的 CORS 配置

- **问题**：CORS (跨域资源共享) 配置不当，可能导致敏感数据泄露。
 - **防护措施**：
 - **限制** `Access-Control-Allow-Origin` 只允许特定域名访问，不要设为 `*`。
 - **限制** `Access-Control-Allow-Credentials`，避免跨域携带 Cookie，防止 CSRF 攻击。
-

总结

安全问题	解决方案
HTTP 窃听	使用 HTTPS (TLS/SSL)、HSTS
XSS	输入过滤、CSP、HttpOnly Cookie
CSRF	CSRF Token、SameSite Cookie、Referer 检查
SQL 注入	预处理语句、参数化查询
点击劫持	X-Frame-Options、CSP
CRLF 注入	过滤换行符 \r\n
CORS 漏洞	限制 Access-Control-Allow-Origin

Q9：如何预防XSS和CSRF攻击？

预防 XSS (Cross-Site Scripting)

XSS 攻击本质：攻击者在网页中注入恶意 JavaScript，劫持用户数据或篡改页面内容。

1. 输入过滤

目的：防止用户输入恶意代码。

- 采用 **白名单** 过滤，仅允许安全字符（如 `a-zA-Z0-9`）。
- 禁止 `<script>`、`<iframe>`、`onerror`、`javascript:` 等关键字。
- 使用正则表达式限制输入格式，例如：

```
const safeInput = input.replace(/</g, "&lt;").replace(/>/g, "&gt;");
```

2. 输出编码 (Escaping Output)

目的：避免 HTML 解析恶意代码。

- **HTML 编码：**将 `<script>` 转换为 `<script>`。
- **JavaScript 编码：**
 - `escape()`
 - `encodeURIComponent()`
- **示例**（在 Vue/Node.js 中使用）：
 - Vue：使用 `v-text` 或 `{{ message }}` 自动转义。
 - Node.js：

```
const escapeHTML = (str) => str.replace(/</g, "&lt;").replace(/>/g, "&gt;");
```

3. CSP (内容安全策略, Content Security Policy)

目的：限制 JavaScript 执行来源，阻止恶意脚本加载。

- 服务器设置 `Content-Security-Policy` 头：

```
Content-Security-Policy: default-src 'self'; script-src 'self'
https://trustedscripts.example.com
```

- 只允许 **可信域名** 的脚本执行，阻止 `inline script` (`<script>alert(1)</script>`)。
- 推荐：禁止 `eval()`，防止恶意 JS 执行。

4. HttpOnly Cookie

目的：防止 JavaScript 读取敏感 Cookie，防止 `document.cookie` 被 XSS 窃取。

- 服务器设置：

```
Set-Cookie: sessionid=abc123; HttpOnly; Secure
```

5. 其他安全策略

- **WAF (Web 应用防火墙)**：自动检测 XSS 攻击请求。
- **安全库：**
 - 前端：DOMPurify (清理 HTML)
 - 后端：Helmet.js (Express.js 中间件)

预防 CSRF (Cross-Site Request Forgery)

CSRF 攻击本质：利用用户已登录的身份，诱导其发送伪造请求。

1. CSRF Token 机制

目的：在每个请求中添加**随机 Token**，服务器验证合法性。

- **后端生成 Token** (Django 示例)：

```
def get_csrf_token():
    return secrets.token_hex(32)
```

- **前端发送 Token** (Ajax 示例)：

```
fetch('/api/transfer', {
  method: 'POST',
  headers: { 'X-CSRF-Token': csrfToken },
  body: JSON.stringify({ amount: 100 })
});
```

- **服务器验证：**
 - 服务器检查 `X-CSRF-Token` 是否匹配，若不匹配则拒绝请求。

2. SameSite Cookie

目的：限制跨站请求携带 Cookie，防止 CSRF。

- **配置 SameSite 选项：**

```
Set-Cookie: sessionid=xyz; Secure; HttpOnly; SameSite=Strict
```

- `SameSite=Strict`：完全阻止跨站请求携带 Cookie（最安全）。
- `SameSite=Lax`：允许部分跨站 GET 请求（适合大多数情况）。
- `SameSite=None; Secure`：允许跨站请求，但**必须使用 HTTPS**。

3. Referer 头检查

目的：服务器检查请求来源，防止跨站提交数据。

- **服务器验证 Referer：**

```
def validate_referer(request):
    referer = request.headers.get("Referer")
    if referer and not referer.startswith("https://yourdomain.com"):
        return False # 拒绝请求
```

- 限制请求必须来自 `yourdomain.com`。

4. 用户操作验证 (CAPTCHA)

目的：增加人工交互，防止自动化 CSRF 攻击。

- **验证码示例 (Google reCAPTCHA)：**

```
<form action="/submit" method="POST">
  <input type="text" name="username">
  <div class="g-recaptcha" data-sitekey="your-site-key"></div>
  <button type="submit">提交</button>
</form>
```

5. 仅使用 POST 进行重要操作

目的：防止 GET 请求触发敏感操作。

- **示例 (禁止 GET 方式删除数据)：**

```
DELETE /delete-account ❌ (避免直接点击 URL 触发)
```

总结

攻击类型	防御手段
XSS (跨站脚本)	输入过滤、输出编码、CSP、安全 Cookie (HttpOnly)
CSRF (跨站请求伪造)	CSRF Token、SameSite Cookie、Referer 检测、验证码

最佳实践

1. **使用 HTTPS**，加密传输，防止数据窃取。
2. **CSP + HttpOnly Cookie**，避免恶意脚本执行和数据窃取。
3. **对用户输入进行严格验证**，防止 XSS 和 SQL 注入攻击。
4. **启用 CSRF Token 和 SameSite Cookie**，防止 CSRF 攻击。
5. **后端限制跨域访问** (CORS 仅允许可信域名)。

Q10: 为什么CSRF token可以预防CSRF攻击?

攻击者无法获取或伪造有效的 CSRF Token，原因如下：

- ✓ Token 只能由服务器生成，攻击者无法预测 Token 值。
- ✓ Token 需要与用户的 Session 关联，不同用户的 Token 不一样，攻击者无法通用。
- ✓ 攻击者无法跨域读取 Token：
 - 由于浏览器的同源策略（SOP），attacker.com 无法访问 bank.com 的页面源代码，因此无法获取 Token。
 - HTTP 头中的 Token 只能由同源 AJAX 请求获取，攻击者无法在恶意网站中发起有效请求。

2.代码题

Q1: 实现retry函数，尽量去满足更通用的重试需求

```
const retry = () => {};  
const request = (params) => fetch('/api/get - movie', {  
  body: JSON.stringify(params)  
});  
const retryRequest = retry(request, 2); //这里的 2 表示最多重试 2 次（调用 3 次）  
const response = await retryRequest({ movieId: 1001 });
```

具体实现

```
const retry = (fn, maxRetries = 3) => {  
  return async (...args) => {  
    let retries = 0;  
    while (retries < maxRetries) {  
      try {  
        return await fn(...args);  
      } catch (error) {  
        retries++;  
        console.log(`Retrying request, attempt ${retries} of  
${maxRetries}`);  
        await new Promise(resolve => setTimeout(resolve, 1000 *  
retries)); // 延迟重试  
      }  
    }  
    throw new Error(`Maximum retries (${maxRetries}) exceeded`);  
  };  
};  
  
// 2. 定义 request 函数  
const request = async (params) => {  
  const response = await fetch('/api/get - movie', {  
    method: 'POST',  
    headers: {  
      'Content - Type': 'application/json'  
    },  
    body: JSON.stringify(params)  
  });  
  return await response.json();  
};
```

```
const retryRequest = retry(request, 2); // 最多重试 2 次
const response = await retryRequest({ movieId: 1001 });
```

4.2饿了么面试

1. 面试题

Q1: 怎么选择的技术栈框架?

4.2腾讯AI面试

Q1: 应聘前端岗位的原因? 为此做了哪些努力? 有哪些知识储备?

应聘前端岗位的原因

我对前端岗位充满热情, 主要有以下几点原因:

- 用户体验至上:** 前端是用户与产品之间的第一接触点, 能够直接影响用户的感受。我热爱通过代码和设计来打造流畅、美观且易用的界面, 让用户在使用产品时感到愉悦。例如, 之前在开发页面时, 通过优化页面布局和交互设计, 将用户停留时间提升了30%, 这让我深刻感受到前端工作对用户体验的巨大价值。
- 技术挑战与创新:** 前端技术日新月异, 从早期的HTML、CSS到如今的React、Vue等框架, 再到WebAssembly等新兴技术, 不断有新的挑战和创新机会。我享受这种不断学习和探索的过程, 比如在Google时, 我们尝试将WebAssembly用于一些复杂计算任务, 提升页面性能的同时, 也让我对前端性能优化有了更深入的理解。
- 跨领域合作:** 前端开发需要与设计师、后端工程师、产品经理等多角色紧密合作。这种跨领域的合作模式让我能够从不同角度思考问题, 提升综合能力。例如, 在一个大型项目中, 我与设计师紧密配合, 将设计稿高效转化为可交互的页面, 同时与后端团队协作, 确保前后端数据交互的稳定性和安全性。

为此做的努力

1. 技术学习与提升:

- 系统学习基础知识:** 深入学习HTML5、CSS3、JavaScript等基础技术, 不仅掌握语法, 还理解其底层原理。例如, 通过阅读《JavaScript高级程序设计》等书籍, 深入理解原型链、闭包等核心概念。
- 掌握主流框架与工具:** 熟练使用React、Vue、Angular等主流前端框架, 并掌握Webpack、Babel等构建工具。在实际项目中, 我使用React开发过多个大型单页应用, 通过合理配置Webpack, 优化打包速度和资源加载性能。
- 关注新技术动态:** 订阅前端技术博客、参加技术会议和培训课程, 及时了解新技术趋势。例如, 我曾参加过Google I/O大会, 学习最新的Web技术进展, 并尝试将其应用到实际项目中。

2. 项目实践与积累:

- 独立开发项目:** 利用业余时间开发一些小型项目, 锻炼自己的开发能力和问题解决能力。我从零开始搭建前端页面, 过程中遇到的跨域问题、性能优化等问题都让我积累了宝贵的经验。

3. 提升软技能:

- 沟通协作能力:** 在团队项目中, 积极与成员沟通协作, 学会表达自己的想法, 同时也能理解他人的需求。例如, 在项目中, 我与后端团队频繁沟通, 确保前后端接口的顺利对接。

- **问题解决能力**：面对开发中遇到的难题，学会冷静分析问题，通过查阅文档、社区问答等方式寻找解决方案。比如在处理一个复杂的页面布局问题时，我通过查阅MDN文档和Stack Overflow上的相关问题，最终找到了优雅解决方案。

知识储备

1. 前端基础技术：

- **HTML5**：熟悉HTML5的新特性，如语义化标签、Canvas、SVG等。在实际开发中，合理使用语义化标签可以提升页面的可访问性和搜索引擎优化效果。
- **CSS3**：掌握CSS3的布局方式，如Flexbox、Grid等，以及动画、渐变等视觉效果。例如，使用Flexbox可以轻松实现响应式布局，提升页面在不同设备上的适配性。
- **JavaScript**：深入理解JavaScript的运行机制，包括事件循环、异步编程等。在开发中，合理使用Promise和async/await可以简化异步代码的编写，提升代码的可读性。

2. 前端框架与库：

- **React**：熟练掌握React的组件化开发模式，理解虚拟DOM、生命周期等核心概念。在实际项目中，通过合理拆分组件，可以提高代码的复用性和可维护性。
- **Vue**：了解Vue的双向数据绑定、指令系统等特性。在一些小型项目中，Vue的简洁语法可以快速搭建出功能丰富的页面。
- **其他库**：熟悉如Axios、Lodash等常用库，提升开发效率。例如，使用Axios可以方便地进行HTTP请求，Lodash提供的工具函数可以简化代码逻辑。

3. 构建工具与工程化：

- **Webpack**：掌握Webpack的配置和优化技巧，如代码分割、Tree Shaking等。在大型项目中，合理配置Webpack可以显著提升打包速度和资源加载性能。
- **Babel**：了解Babel的转译原理，将ES6+代码转译为兼容性更好的代码，确保项目在不同浏览器环境下都能正常运行。

4. 性能优化与安全：

- **性能优化**：掌握前端性能优化的常见方法，如懒加载、缓存策略、代码压缩等。例如，在一个图片展示页面中，通过实现图片懒加载，将页面首屏加载时间缩短了50%。
- **安全**：了解前端安全相关知识，如XSS、CSRF等攻击方式及防御措施。在实际开发中，通过严格过滤用户输入、使用HTTPS等方式，保障应用的安全性。

5. 跨平台与跨浏览器兼容性：

- **跨平台开发**：了解React Native、Flutter等跨平台开发框架，能够开发出兼容多平台的应用。例如，在一个移动应用项目中，使用React Native实现了iOS和Android平台的统一开发。
- **跨浏览器兼容性**：掌握不同浏览器的兼容性问题及解决方案，确保页面在主流浏览器上都能正常显示。例如，通过使用CSS前缀、Polyfill等方式，解决浏览器兼容性问题。

Q2: `<script>` 标签属性 `async` 和 `defer` 的区别

1. 底层原理

`<script>` 标签的 `async` 和 `defer` 属性都是用来控制脚本的加载和执行时机的，但它们的实现机制和使用场景有所不同。

- **`async` 属性：**

- **加载时机**：脚本会异步加载，不会阻塞页面的解析和渲染。
- **执行时机**：脚本加载完成后，会立即执行，但执行顺序不保证。如果多个 `async` 脚本同时加载，它们的执行顺序可能与它们在页面中的顺序不同。
- **原理**：浏览器会立即开始下载脚本文件，同时继续解析和渲染页面。当脚本文件下载完成后，浏览器会暂停页面解析，执行该脚本，然后继续解析页面。

- **`defer` 属性：**

- **加载时机**：脚本会异步加载，不会阻塞页面的解析和渲染。
- **执行时机**：脚本会在页面解析完成后、`DOMContentLoaded` 事件触发之前执行。如果多个 `defer` 脚本同时加载，它们的执行顺序会与它们在页面中的顺序一致。
- **原理**：浏览器会立即开始下载脚本文件，同时继续解析和渲染页面。当页面解析完成后，浏览器会按顺序执行所有 `defer` 脚本，然后触发 `DOMContentLoaded` 事件。

2. 实际开发场景中的问题

- **async 的问题**：
 - **执行顺序不确定**：由于 `async` 脚本的执行顺序不保证，可能会导致依赖关系的问题。例如，如果 `data.js` 依赖于 `layout.js` 中定义的某些变量或函数，可能会因为 `data.js` 先执行而导致错误。
 - **最佳实践**：尽量避免在 `async` 脚本中使用全局变量或依赖其他脚本。如果需要依赖，可以使用模块化的方式，通过 `import` 和 `export` 来管理依赖关系。
- **defer 的问题**：
 - **执行延迟**：`defer` 脚本会在页面解析完成后执行，可能会导致页面的某些功能延迟加载。例如，如果页面布局初始化脚本使用了 `defer`，用户可能会看到一个未初始化的页面布局。
 - **最佳实践**：对于需要立即执行的脚本，可以使用 `async` 或直接内嵌在页面中。对于不依赖页面解析结果的脚本，可以使用 `defer` 来优化页面加载性能。

Q3: https的加密流程

HTTPS（超文本传输安全协议）通过结合对称加密和非对称加密，以及数字证书验证，确保了数据传输的安全性和完整性。以下是详细的加密流程：

1. 客户端发起请求

客户端（通常是浏览器）向服务器发送一个安全连接请求，使用 HTTPS 的 URL 或点击 HTTPS 链接触发。其中包含**第1个随机数**。

2. 服务器发送证书

服务器收到请求后，将自己的数字证书发送给客户端。证书中包含了**服务器的公钥**、数字签名和其他相关信息。数字证书是由证书颁发机构（CA）签发的，用于验证服务器的身份。其中包含**第2个随机数**。

3. 客户端验证证书

客户端接收到服务器的数字证书后，会进行一系列的验证步骤：

- 验证证书是否由受信任的证书颁发机构签发。
- 验证证书中的域名是否与目标服务器的域名相匹配。
- 验证证书的有效期。

如果验证通过，客户端可以确定所连接的服务器是可信的。

4. 密钥协商

一旦服务器证书被验证通过，客户端会生成一个随机的**对称密钥（第3个随机数）**，用于后续的数据加密和解密。该对称密钥通过服务器证书中的公钥进行加密，并发送给服务器。

5. 服务器解密密钥

服务器使用其**私钥**解密客户端发送的对称密钥，从而获取到**对称密钥**。

6. 建立加密通道

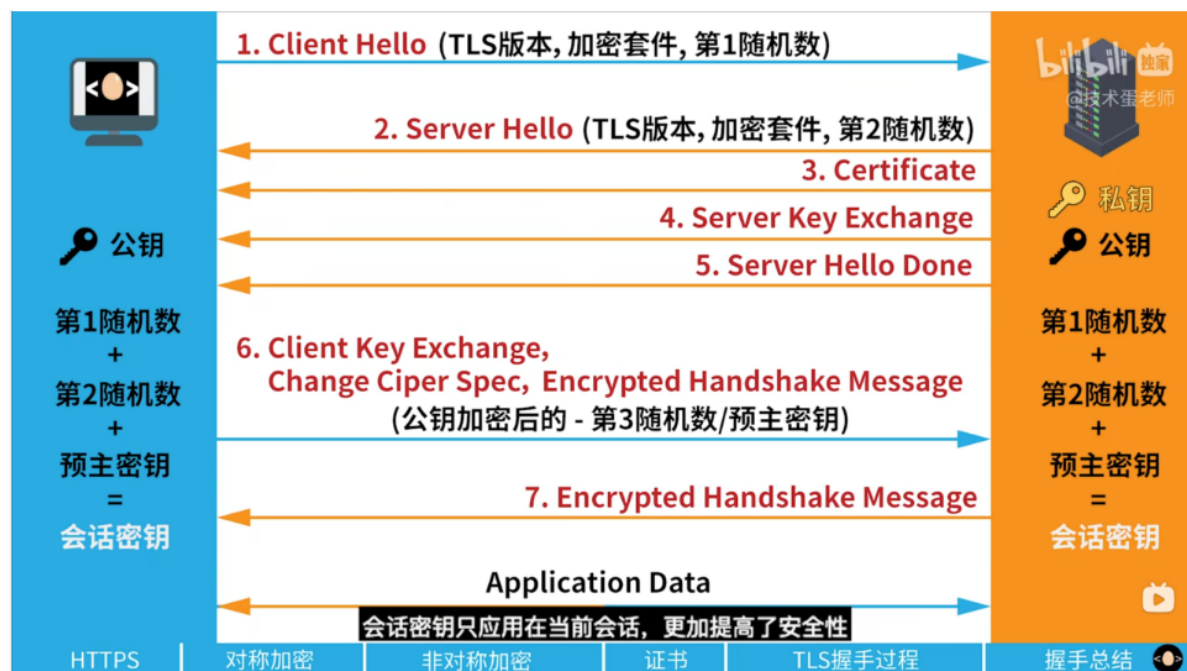
客户端和服务端之间建立起一个加密的安全通道。从此之后，客户端和服务端之间的数据传输都会在此加密通道中进行，使用**对称密钥对数据进行加密和解密**。

7. 安全数据传输

在建立了安全通道后，客户端和服务端可以安全地传输数据了。数据在发送前，会使用对称密钥对数据进行加密，然后在接收端使用相同的对称密钥解密。

实际开发场景中的问题及注意事项

- **证书管理**：确保服务器的证书是由受信任的 CA 签发的，否则客户端会提示证书不可信。
- **性能优化**：虽然 HTTPS 提供了安全性，但加密和解密过程会增加一定的计算开销。可以通过优化服务器配置和使用硬件加速来缓解这一问题。
- **中间人攻击**：即使使用了 HTTPS，如果证书管理不当，仍然可能受到中间人攻击。确保证书的安全存储和更新。



Q4: 单向认证和双向认证的区别

1. 单向认证

单向认证是指客户端验证服务器的身份，但服务器不验证客户端的身份。这是最常见的认证方式，广泛应用于HTTPS等场景。

原理

- **客户端请求**：客户端向服务器发起请求，请求建立安全连接。
- **服务器响应**：服务器向客户端发送其数字证书，证书中包含服务器的公钥、证书颁发机构（CA）的签名等信息。
- **客户端验证**：客户端验证服务器的证书是否有效，验证过程包括：
 - 检查证书是否由受信任的CA签发。

- 检查证书是否在有效期内。
- 检查证书中的域名是否与目标服务器的域名一致。
- **密钥协商**：客户端生成一个对称密钥，使用服务器的公钥加密后发送给服务器。
- **建立加密通道**：服务器使用私钥解密对称密钥，双方使用对称密钥进行后续通信。

实际案例

在HTTPS通信中，客户端（浏览器）通过验证服务器的证书来确保连接的安全性。例如，当你访问一个银行网站时，浏览器会验证该网站的证书是否由受信任的CA签发，从而确保你正在与正确的银行服务器通信。

问题及注意事项

- **证书管理**：确保服务器的证书是由受信任的CA签发的，否则客户端会提示证书不可信。
- **中间人攻击**：如果证书管理不当，可能会受到中间人攻击。确保证书的安全存储和更新。

2. 双向认证

双向认证是指客户端和服务器都验证对方的身份。这种方式提供了更高的安全性，通常用于对安全性要求较高的场景，如金融交易系统。

原理

- **客户端请求**：客户端向服务器发起请求，请求建立安全连接。
- **服务器响应**：服务器向客户端发送其数字证书。
- **客户端验证**：客户端验证服务器的证书是否有效。
- **客户端证书**：客户端向服务器发送自己的数字证书。
- **服务器验证**：服务器验证客户端的证书是否有效。
- **密钥协商**：客户端生成一个对称密钥，使用服务器的公钥加密后发送给服务器。
- **建立加密通道**：服务器使用私钥解密对称密钥，双方使用对称密钥进行后续通信。

实际案例

在金融交易系统中，银行服务器不仅需要验证客户端的身份，客户端也需要验证银行服务器的身份。例如，一个移动支付应用在与银行服务器通信时，双方都会验证对方的证书，确保通信的安全性。

问题及注意事项

- **证书管理**：客户端和服务器都需要管理自己的证书和对方的证书，增加了管理复杂性。
- **性能开销**：双向认证增加了额外的证书验证步骤，可能会导致性能下降。
- **用户友好性**：客户端需要管理证书，可能会给用户带来不便。在实际应用中，可以通过证书自动管理工具来简化这一过程。

总结

- **单向认证**：客户端验证服务器的身份，适用于大多数Web应用。
- **双向认证**：客户端和服务器互相验证对方的身份，适用于对安全性要求较高的场景。

Q5：怎么判断一个 JavaScript 对象是否为空？

1. 使用 `Object.keys()` 方法

`Object.keys()` 方法会返回一个由给定对象的自身可枚举属性组成的数组。如果返回的数组长度为 0，则说明对象为空。

底层原理

`Object.keys()` 遍历对象的自有属性（不包括原型链上的属性），并返回一个包含这些属性键名的数组。

```
> Object.keys({name:"liming",age:18})  
↳ (2) ['name', 'age']
```

代码示例

```
function isEmptyObject(obj) {  
    return Object.keys(obj).length === 0;  
}  
  
// 示例  
const obj1 = {};  
const obj2 = { key: 'value' };  
  
console.log(isEmptyObject(obj1)); // true  
console.log(isEmptyObject(obj2)); // false
```

实际开发场景

在实际开发中，这种方法非常常用，因为它简单且高效。例如，在处理用户输入的表单数据时，可能需要检查某些字段是否为空对象。

问题及注意事项

- **性能：**`Object.keys()` 的时间复杂度为 $O(n)$ ，其中 n 是对象的属性数量。对于大多数场景来说，性能是足够的。
- **只检查自有属性：**`Object.keys()` 不会检查原型链上的属性。如果需要检查原型链上的属性，可以使用 `Object.getPrototypeOfNames()` 或 `Reflect.ownKeys()`。

2. 使用 `for...in` 循环

`for...in` 循环可以遍历对象的所有可枚举属性（包括原型链上的属性）。如果循环体从未执行，则说明对象为空。

底层原理

`for...in` 循环会遍历对象的所有可枚举属性，包括继承的属性。如果对象为空，则循环体不会执行。

代码示例

```
function isEmptyObject(obj) {  
    for (const key in obj) {  
        if (obj.hasOwnProperty(key)) {  
            return false;  
        }  
    }  
    return true;  
}  
  
// 示例  
const obj1 = {};  
const obj2 = { key: 'value' };
```

```
console.log(isEmptyObject(obj1)); // true
console.log(isEmptyObject(obj2)); // false
```

实际开发场景

这种方法在处理继承属性时非常有用。例如，在某些框架中，对象可能会继承一些默认属性，需要排除这些属性来判断对象是否为空。

问题及注意事项

- **性能**：for...in 循环的时间复杂度为 O(n)，性能与 Object.keys() 类似。
- **排除继承属性**：使用 hasOwnProperty 方法可以排除继承属性，确保只检查对象的自有属性。

3. 使用 JSON.stringify() 方法

将对象序列化为 JSON 字符串，如果结果是 {}，则说明对象为空。

底层原理

JSON.stringify() 方法将对象序列化为 JSON 字符串。如果对象为空，序列化结果为 {}。

代码示例

```
function isEmptyObject(obj) {
    return JSON.stringify(obj) === '{}';
}

// 示例
const obj1 = {};
const obj2 = { key: 'value' };

console.log(isEmptyObject(obj1)); // true
console.log(isEmptyObject(obj2)); // false
```

实际开发场景

这种方法简单直观，适用于快速判断对象是否为空。例如，在调试阶段，可以快速检查对象的内容。

问题及注意事项

- **性能**：JSON.stringify() 的性能可能不如 Object.keys() 和 for...in，尤其是在处理大型对象时。
- **特殊值处理**：如果对象中包含特殊的值（如 undefined、function 等），JSON.stringify() 可能会忽略这些值，导致误判。

4. 使用 Object.entries() 方法

Object.entries() 方法会返回一个由给定对象的自身可枚举属性组成的数组。如果返回的数组长度为 0，则说明对象为空。

底层原理

Object.entries() 遍历对象的自有属性，返回一个包含键值对的数组。

```
> Object.entries({name:"liming",age:18})
(2) [[Array(2), Array(2)]]
  0: (2) ['name', 'liming']
  1: (2) ['age', 18]
  length: 2
  [[Prototype]]: Array(0)
```

代码示例

```
function isEmptyObject(obj) {
  return Object.entries(obj).length === 0;
}

// 示例
const obj1 = {};
const obj2 = { key: 'value' };

console.log(isEmptyObject(obj1)); // true
console.log(isEmptyObject(obj2)); // false
```

实际开发场景

这种方法与 `Object.keys()` 类似，但返回的是键值对数组，适用于需要同时检查键和值的场景。

问题及注意事项

- **性能：**`Object.entries()` 的时间复杂度为 $O(n)$ ，性能与 `Object.keys()` 类似。
- **只检查自有属性：**`Object.entries()` 不会检查原型链上的属性。

总结

以下是几种判断对象是否为空的方法及其适用场景：

方法	优点	缺点	适用场景
<code>Object.keys()</code>	简单高效，只检查自有属性	性能略逊于 <code>for...in</code>	通用场景，不涉及继承属性
<code>for...in</code>	可以检查继承属性	需要结合 <code>hasOwnProperty</code> 排除继承属性	需要排除继承属性的场景
<code>JSON.stringify()</code>	简单直观，代码简洁	性能较差，可能忽略特殊值	调试阶段，快速检查对象内容
<code>Object.entries()</code>	返回键值对数组，方便操作	性能略逊于 <code>Object.keys()</code>	需要同时检查键和值的场景

在实际开发中，推荐使用 `Object.keys()` 或 `for...in`，因为它们性能较好且功能强大。如果需要快速检查对象内容，可以使用 `JSON.stringify()`，但要注意其可能的局限性。

Q6: Vue3的proxy比Vue2的Object.defineProperty做了哪些性能提升?

1. 性能提升

- **代理整个对象**: Vue 2 使用 `Object.defineProperty` 为每个属性单独设置 getter 和 setter, 这在嵌套对象中会导致递归遍历, 性能开销较大。而 Vue 3 使用 `Proxy` 直接代理整个对象, 避免了递归遍历的开销。
- **动态属性监听**: Vue 2 无法监听对象新增或删除的属性, 需要通过 `vue.set()` 和 `vue.delete()` 显式处理。Vue 3 的 `Proxy` 可以自动监听新增和删除的属性, 无需额外操作。
- **数组操作优化**: Vue 2 在处理数组时, 需要特殊的处理方法 (如 `push`、`splice`) 来检测数组变动, 且无法直接监听数组的长度变化。Vue 3 的 `Proxy` 可以直接拦截数组的所有操作, 包括 `push`、`pop` 等, 性能更优。

2. 功能增强

- **全面拦截**: `Proxy` 可以拦截整个对象的所有操作, 包括属性的读取、写入、删除、枚举等, 还可以拦截 `in` 操作符、`for...in` 循环、`Object.keys` 等方法。
- **灵活的拦截器**: `Proxy` 提供了多种拦截器方法, 如 `get`、`set`、`apply`、`construct` 等, 可以实现更复杂的逻辑。

3. 实际案例

假设有一个包含大量数据的复杂对象, 需要实现响应式更新。在 Vue 2 中, 递归遍历每个属性设置 getter 和 setter 会导致性能瓶颈。而在 Vue 3 中, 使用 `Proxy` 可以直接代理整个对象, 显著提升性能。

```
// Vue 3 的 Proxy 实现
import { reactive } from 'vue';
const state = reactive({
  count: 0,
  nested: {
    key: 'value'
  }
});

state.count++; // 视图自动更新
state.nested.newKey = 'newValue'; // 新增属性自动响应
```

总结

Vue 3 通过引入 `Proxy`, 在响应式系统上实现了显著的性能提升和功能增强。`Proxy` 可以直接代理整个对象, 避免了递归遍历的开销, 支持动态属性监听和数组操作优化。这些改进使得 Vue 3 在处理复杂数据和大型应用时更加高效。

Q7: 重排与重绘的定义和触发场景, 怎么能减少重排与重绘?

1. 定义

- **重排 (Reflow)**: 重排其实就是回流, 可能会引起重绘。当页面的布局发生变化时, 浏览器需要重新计算元素的几何属性 (如位置、大小等), 这个过程称为重排。重排是一个较为耗时的操作, 因为它涉及到页面布局的重新计算。
- **重绘 (Repaint)**: 当页面的样式发生变化, 但布局未发生变化时, 浏览器需要重新绘制页面的一部分或全部, 这个过程称为重绘。重绘通常比重排更快, 但如果频繁发生, 也会对性能产生影响。

2. 触发场景

- **重排的触发场景：**
 - 修改元素的尺寸（如宽度、高度）。
 - 修改元素的边距、内边距。
 - 添加或删除可见的DOM元素。
 - 内容变化（如文本内容的增加或减少）。
 - 浏览器窗口大小改变。
- **重绘的触发场景：**
 - 修改元素的颜色、背景颜色等样式属性。
 - 修改元素的可见性（如 `display: none` 或 `visibility: hidden`）。
 - 动画效果导致的样式变化。

3. 减少重排与重绘的方法

- **减少重排：**
 - **批量修改样式：** 尽量将多个样式修改操作集中在一起，减少触发重排的次数。例如，使用 `class` 或 `style` 属性一次性修改多个样式。
 - **使用 `transform` 和 `opacity`：** 对于动画效果，优先使用 `transform` 和 `opacity` 属性，因为这些属性不会触发重排。
 - **避免使用 `table` 布局：** `table` 布局的重排成本较高，尽量使用 `div` 和 `flexbox` 布局。
 - **使用 `getComputedStyle`：** 如果需要获取元素的样式信息，尽量使用 `window.getComputedStyle`，而不是直接读取样式属性，因为后者可能会触发重排。
- **减少重绘：**
 - **使用 `will-change` 提示：** 对于即将发生变化的元素，可以使用 `will-change` 属性提前告知浏览器，让浏览器提前做好优化准备。
 - **避免频繁修改样式：** 尽量减少对元素样式的频繁修改，可以通过动画或过渡效果来实现平滑的变化。
 - **使用 `backface-visibility`：** 在3D变换中，使用 `backface-visibility: hidden` 可以减少重绘。

4. 最佳实践

- **性能优化：** 在开发过程中，可以通过开发者工具（如 Chrome DevTools）的性能分析工具来检测重排和重绘的发生频率，并针对性地进行优化。
- **代码示例：**

```
// 避免频繁修改样式
const element = document.getElementById('myElement');
element.style.transition = 'all 0.3s ease';
element.style.transform = 'translateX(100px)';
element.style.opacity = '0.5';
```

Q8：前端怎么解决网络延迟问题？

1. 减少 HTTP 请求

HTTP 请求是导致网络延迟的主要原因之一。减少 HTTP 请求可以有效降低延迟。

- **合并文件**：将多个 CSS 或 JavaScript 文件合并成一个文件，减少请求次数。
- **使用 CSS Sprites**：将多个小图片合并成一个大图片，通过 CSS 的 `background-position` 属性来显示需要的部分。
- **内嵌小型资源**：对于小的 CSS 和 JavaScript 文件，可以直接内嵌到 HTML 中。

2. 使用 CDN

CDN（内容分发网络）可以将静态资源分发到多个地理位置的服务器上，用户可以从最近的服务器获取资源，从而减少网络延迟。

- **选择合适的 CDN 服务**：如 Cloudflare、Akamai 和 AWS CloudFront 等。
- **配置 CDN**：将静态资源（如图片、CSS、JavaScript）部署到 CDN 上。

3. 压缩资源

压缩文件可以减少文件大小，从而减少 HTTP 请求的时间和带宽。

- **使用 Gzip 或 Brotli**：对 HTML、CSS、JavaScript 等文本文件进行压缩。
- **优化图片大小和格式**：使用工具（如 TinyPNG）对图片进行压缩。

4. 利用浏览器缓存

合理利用浏览器缓存可以避免重复加载相同的资源，提高页面加载速度。

- **设置缓存策略**：通过设置 HTTP 缓存头（如 `Cache-Control`、`ETag`），让浏览器缓存静态资源。
- **使用本地缓存**：如 `localStorage` 或 `sessionStorage`，缓存一些不经常变化的数据。

5. 优化图片加载

图片是页面加载中体积最大的资源之一，优化图片加载可以显著减少网络延迟。

- **使用懒加载**：对于非首屏图片，可以使用懒加载技术，当用户滚动到图片位置时再加载。
- **使用现代图片格式**：如 WebP，它在保持高质量的同时，文件大小更小。

6. 异步加载资源

异步加载资源可以避免阻塞页面的渲染，提高页面加载速度。

- **使用 `async` 和 `defer`**：在 `<script>` 标签中添加 `async` 或 `defer` 属性，异步加载 JavaScript 文件。
- **按需加载**：对于一些不需要立即加载的资源，可以采用按需加载的方式。

7. 优化第三方资源加载

第三方资源（如广告、分析工具和社交媒体插件）可能会影响页面加载速度。

- **延迟加载第三方资源**：在用户交互或滚动到特定位置时再加载。
- **使用异步加载第三方资源**：减少网络请求和加载时间。

8. 使用 HTTP/2 或 HTTP/3

新版本的 HTTP 协议支持多路复用、头部压缩等功能，可以有效减少网络延迟。

9. 减少重定向

重定向会增加额外的 HTTP 请求和网络延迟。

- **避免不必要的重定向：**确保所有链接都使用统一的 URL 格式。
- **使用永久重定向：**对于必须进行的重定向，使用 301 永久重定向。

总结

通过减少 HTTP 请求、使用 CDN、压缩资源、利用浏览器缓存、优化图片加载、异步加载资源等策略，可以有效解决前端网络延迟问题，提升页面加载速度和用户体验。在实际项目中，可以根据具体需求选择合适的优化策略，并结合性能监控工具（如 Google PageSpeed Insights、Lighthouse）来评估优化效果。

Q9：开发中怎么考虑不同浏览器版本的兼容性？

1. 使用标准的 HTML/CSS/JavaScript

- **遵循 Web 标准：**使用 HTML5、CSS3 和 ES6+ 等现代标准，这些标准在大多数现代浏览器中得到了广泛支持。
- **避免过时技术：**避免使用已废弃的 HTML 标签和属性，以及过时的 JavaScript 方法。

2. CSS 兼容性问题及解决方案

- **盒模型差异：**通过 `<!DOCTYPE html>` 强制浏览器使用标准模式，并统一使用 `box-sizing: border-box`。
- **Flexbox/Grid 布局兼容：**使用 Autoprefixer 自动生成浏览器前缀，并为旧版本浏览器提供替代方案。
- **CSS 属性前缀：**使用 PostCSS + Autoprefixer 自动添加前缀。
- **默认样式差异：**使用 `normalize.css` 等重置样式表。

3. JavaScript 兼容性问题及解决方案

- **ES6+ 语法支持：**使用 Babel 转译代码到 ES5，并引入 `core-js` 或 `@babel/polyfill`。
- **DOM 事件差异：**为不同浏览器提供兼容的事件绑定方法。
- **AJAX 兼容性：**为不同浏览器提供兼容的 AJAX 请求方法。
- **API 缺失：**使用 Polyfill（如 `whatwg-fetch`、`storage-polyfill`）。

4. HTML 兼容性问题及解决方案

- **HTML5 新标签支持：**引入 `html5shiv.js`。
- **表单控件兼容性：**使用 Polyfill（如 `flatpickr` 日期选择库）。

5. 进行跨浏览器测试

- **手动测试：**在不同浏览器（如 Chrome、Firefox、Safari、Edge、IE）和设备上进行测试。
- **自动化测试：**使用工具如 Selenium、BrowserStack、Sauce Labs。

6. 使用 Polyfills 和 Transpilers

- **Polyfills**: 填补浏览器功能缺失的代码片段, 如 `ES6-Promise`、`fetch()`。
- **Transpilers**: 将高级语言特性转译为低级语言, 如 Babel。

7. 借助 CSS 前缀

- **浏览器前缀**: 使用 `-webkit-`、`-moz-`、`-ms-` 等前缀, 确保 CSS 特性在不同浏览器中的兼容性。
- **自动添加前缀**: 使用 Autoprefixer 等工具。

8. 选择合适的框架和库

- **前端框架**: 选择成熟且广泛使用的框架 (如 React、Vue.js、Angular) 。
- **UI 库**: 使用成熟的 UI 库 (如 Bootstrap、Material-UI、Ant Design) 。

9. 采用渐进增强和优雅降级策略

- **渐进增强**: 从基础功能开始构建网站, 逐步添加高级功能。
- **优雅降级**: 从高级功能开始构建网站, 为不支持的浏览器提供替代方案。

10. 特殊浏览器处理

- **IE 条件注释**: 使用 IE 条件注释针对 IE 浏览器。
- **IE 专属 CSS Hack**: 使用特定的 CSS Hack 针对不同版本的 IE。

总结

通过使用标准的 HTML/CSS/JavaScript、应用渐进增强和优雅降级策略、进行跨浏览器测试、使用 Polyfills 和 Transpilers、借助 CSS 前缀、选择合适的框架和库, 以及使用版本控制和 CI/CD 工具, 开发者可以有效地解决兼容性问题, 提供更好的用户体验。

Q10: 怎么减少图片压缩带来的负面影响?

1. 选择合适的图片格式

根据图片的用途和特点选择合适的格式, 可以平衡文件大小和图片质量:

- **JPEG**: 适合照片和复杂图像, 支持有损压缩, 文件大小较小, 但不支持透明背景。
- **PNG**: 适合图标、插图和需要透明背景的图像, 无损压缩, 文件大小较大。
- **WebP**: 现代浏览器支持的格式, 结合有损和无损压缩, 文件大小小, 支持透明背景。
- **SVG**: 适合矢量图形, 如图标和简单图形, 文件小, 任意缩放不失真。

2. 合理使用压缩技术

根据需求选择无损压缩或有损压缩:

- **无损压缩**: 去除冗余数据, 不影响图片质量, 但压缩比有限。适合需要保持原始质量的图片。
- **有损压缩**: 通过去除不易察觉的细节来减少文件大小, 但会导致质量下降。适合对质量要求不高的图片。

3. 前端压缩技术

在用户上传图片时，可以使用前端技术进行压缩：

- **Canvas API**：通过 `drawImage` 和 `toDataURL` 方法，调整图片尺寸或质量进行压缩。
- **FileReader 和 Blob**：结合使用读取图片，通过 Canvas 处理后导出为压缩后的 Blob 对象。
- **第三方库**：如 `compress.js`、`pica` 等，提供便捷的 API 实现图片压缩。

4. 响应式图片和懒加载

根据设备和网络状况动态加载适合的图片：

- **响应式图片**：使用 `<picture>` 元素和 `srcset` 属性，根据屏幕尺寸和分辨率加载不同大小的图片。
- **懒加载**：在图片进入可视区域时再加载，减少页面初始加载时间。

5. 平衡质量和文件大小

在压缩图片时，需要在质量和文件大小之间找到平衡：

- **调整压缩参数**：多次尝试找到合适的压缩比，根据图片用途选择不同的压缩策略。
- **按需压缩**：例如，缩略图可以适当降低质量，展示图片则保持较高质量。

6. 使用自动化工具

利用自动化工具批量处理图片，确保每张图片都经过优化：

- **构建工具**：如 Webpack、Gulp 等，结合插件（如 `imagemin`、`image-webpack-loader`）自动压缩图片。
- **脚本处理**：使用 Node.js 脚本批量压缩图片，集成到 CI/CD 流程中。

7. 渐进增强和优雅降级

根据浏览器和设备的支持情况，选择最佳的图片格式和压缩技术：

- **渐进增强**：从基础功能开始，逐步添加高级功能。
- **优雅降级**：从高级功能开始，为不支持的浏览器提供替代方案。

Q11：在异步加载中怎么确保用户界面的一致性？

1 加载状态处理 (Loading)

- 在数据还没返回前，给用户明确的反馈，使用：
 - loading 动画/占位符
 - 灰色块 or 骨架屏
 - loading spinner 等
- 避免空白区域或突然变化

2 防止竞态条件 (Race Condition)

- 当用户连续发起多个请求（如切换分页、搜索），应只保留最后一个结果
- 使用请求的唯一标识、时间戳、或封装取消逻辑（如 `AbortController`）

```
const controller = new AbortController();
fetch(url, { signal: controller.signal }).catch(err => {
  if (err.name === 'AbortError') console.log('请求被取消');
});
```

3 异步顺序控制（异步一致性）

- 使用 `async/await` 保证逻辑顺序清晰
- 避免用户触发前一个请求未完成又发起新请求的混乱

```
async function loadData() {
  setLoading(true);
  try {
    const data = await fetchData();
    setData(data);
  } finally {
    setLoading(false);
  }
}
```

4 错误处理（Error State）

- 接口失败时给出明确反馈：如 Toast、提示框、重试按钮等
- 保证 UI 状态明确，不留“加载中”假象

```
{error ? <ErrorView /> : isLoading ? <Loading /> : <Data />}
```

5 数据缓存 / 乐观更新（Optional）

- 对于频繁操作的接口可考虑：
 - 客户端缓存（如 React Query / SWR）
 - **乐观更新**：数据预更新 UI，接口失败再回滚

6 避免不一致 UI 状态

- 用户跳转页面后，异步请求仍返回会影响 UI，可：
 - 在组件卸载时中断请求
 - 使用 `useEffect` 清理副作用

```
useEffect(() => {
  let isActive = true;
  fetchData().then(data => {
    if (isActive) setData(data);
  });
  return () => { isActive = false; };
}, []);
```

✓ 总结一句话：

通过 loading 占位、竞态控制、错误处理、顺序保证、取消请求等手段，确保异步加载过程中 UI 状态一致、用户体验连贯。

Q12：怎么确定前端项目方案的可行性？

1 明确项目目标与业务需求

- 与产品、设计、后端沟通，确认功能范围与核心目标
- 明确性能要求、兼容性要求（浏览器、平台）、上线时间等关键点

✓ 示例：是否需要移动端适配？是否支持 IE？是否对加载速度有严格要求？

2 技术选型可行性

- 是否有现成方案（框架/库）可用？
- 是否团队已有经验？
- 是否容易集成并维护？

✓ 示例：是否选择 React、Vue 还是低代码？是否选用 SSR、SPA 还是 MPA？

3 开发资源评估

- 团队成员是否熟悉相关技术？
- 是否能按时完成？开发成本是否可控？

✓ 示例：如果项目要用 TypeScript + React + Tailwind，团队是否有经验？人力是否够？

4 性能与用户体验可行性

- 是否会带来性能负担？
- 数据量大时是否能流畅加载？有缓存策略吗？
- 异步加载是否有 loading 状态？页面响应是否流畅？

✓ 示例：表格加载 1000 条数据时是否卡顿？是否分页或虚拟滚动？

5 兼容性与适配性检查

- 是否支持目标浏览器/设备？
- 是否有响应式布局方案？（移动端、PC）
- 是否存在跨域、权限等问题？

✓ 示例：使用 WebSocket 是否后端支持？使用某 API 是否兼容 Safari？

6 风险评估与可替代方案

- 该方案的潜在风险点有哪些？
- 有没有 Plan B？是否容易回滚？

✓ 示例：使用某个新库，如果它不再维护是否有替代？业务变更时如何应对？

7 原型验证 (PoC)

- 先做一个简单的原型 (PoC) 快速验证核心技术点
- 看是否能跑通关键流程

✓ 示例：先用 mock 数据快速搭一套骨架 UI + 接口流程，确保选型无问题。

4.8 腾讯面试

1. 面试题

Q1: vue2和vue3的响应式原理的底层逻辑

Q2: Vue3 的组合式 API 相比 Vue2 的选项式 API 有哪些好处?

1. **逻辑更清晰：**
相同功能相关的逻辑写在一起，不再分散在 data、methods 等多个区域，提升代码可读性和可维护性。
2. **复用性强：**
可通过封装组合函数 (composables) 进行逻辑复用，避免 mixin 带来的命名冲突与调试困难。
3. **类型支持好：**
更好地支持 TypeScript，变量类型推导明确，适合大型项目开发。
4. **灵活性更高：**
完全基于函数式调用，可根据业务需求灵活组合逻辑，适配更复杂的应用场景。

Q3: 引入虚拟 DOM 的好处? 为什么不直接操作 DOM?

1. 引入虚拟 DOM 的好处

1. **性能优化：** 直接操作浏览器的真实 DOM 性能较差，尤其是在频繁更新的情况下。每次操作 DOM 都会触发浏览器的重绘和重排，这会导致性能瓶颈。虚拟 DOM 是一种轻量级的 JavaScript 对象，它在内存中表示真实 DOM。当需要更新 UI 时，首先会在虚拟 DOM 上进行操作，生成新的虚拟 DOM 树，然后与旧的虚拟 DOM 树进行比较（这个过程叫做“Diff”），最终只将发生变化的部分更新到真实 DOM。这种方式大大减少了浏览器的重绘和重排次数，从而提高了性能。
2. **提高开发效率：** 使用虚拟 DOM 可以让开发者专注于描述 UI 如何随着状态变化而变化，而不需要关注具体的 DOM 操作。这使得代码更简洁、可维护。虚拟 DOM 通过自动更新最小的 DOM 部分，减少了手动操作 DOM 的复杂性，使得开发过程更加高效。
3. **跨平台能力：** 虚拟 DOM 抽象了与浏览器环境的耦合，使得 UI 组件可以在不同的平台（如 Web、React Native）上共享。开发者不需要为不同平台分别编写更新 UI 的逻辑，虚拟 DOM 可以在不同平台间统一处理，从而增强了跨平台的能力。

2. 为什么不直接操作 DOM?

- **性能问题：** 每次修改 DOM 都会触发浏览器的重新渲染，导致性能下降，尤其是在复杂应用中，频繁的 DOM 操作会严重影响性能。
- **代码复杂性：** 手动操作 DOM 很容易出错，尤其是在需要多次修改 DOM 的场景下。直接操控 DOM 的逻辑繁琐且难以维护。
- **无法高效进行批量更新：** DOM 更新是逐个元素处理的，无法像虚拟 DOM 一样在内存中先计算出差异，再进行最小化更新。

Q4: 如果改变了两个兄弟元素的样式（比如 margin）会造成几次重排？

这个问题的关键在于理解浏览器的 **重排（Reflow）** 原理。简单来说，**每次影响元素几何结构（大小、位置等）的样式变动**，都会触发重排。

- 如果**依次**改变了两个兄弟元素的样式（比如 margin），**可能会造成两次重排**。
- 但如果使用了**批量操作或者优化手段（如修改 class 或使用 `requestAnimationFrame` 批处理）**，也可能只触发一次重排。

🔍 分析细节：

- **兄弟元素之间是有布局关系的**，比如它们是 `display: block`，一个在上一个的下方，`margin` 变化会影响位置，触发重排。
- 如果你这样写：

```
e11.style.margin = '20px'; // 第一次重排
e12.style.margin = '10px'; // 第二次重排
```

那就可能会产生两次重排。

- 但如果你这样写（批量处理）：

```
e11.style.margin = '20px';
e12.style.margin = '10px';
document.body.offsetHeight; // 强制触发回流（或者等下一帧统一渲染）
```

或者：

```
e11.classList.add('new-margin'); // 假设两个都用这个类
e12.classList.add('new-margin');
```

浏览器可能只进行一次重排。

💡 总结记忆：

- **改变一个影响布局的样式属性 = 可能触发一次重排**
- **多个操作连续发生 = 浏览器可能优化批处理，但不能保证一定只有一次**
- **尽量用 class 批处理样式、避免频繁 DOM 操作，使用 `requestAnimationFrame` 更安全**

Q5: 加载 CSS 会不会阻塞渲染？

是的，加载外部 CSS 文件会阻塞渲染。浏览器会在页面渲染之前下载并解析 CSS 文件，这是因为 CSS 可能会影响页面布局和样式。如果 CSS 加载完成之前没有进行渲染，可能导致页面内容显示为空或不完整，直到样式完全加载和应用完毕。

Q6: requestAnimationFrame

`requestAnimationFrame` 是一个用于优化动画的 API，允许你在浏览器的下一个重绘周期之前执行一个回调函数。它的好处在于：

1. 它可以与浏览器的刷新率（通常是 60Hz，即每秒 60 帧）同步，从而减少了不必要的重绘和提高动画的流畅度。

2. 使用 `requestAnimationFrame` 可以在动画更新时自动暂停或调整频率，确保动画只在浏览器渲染时进行，而不会造成性能浪费。
3. 它也有助于减少页面上的不必要的计算，提升性能。

为什么 `requestAnimationFrame` 和刷新率关系这么大？

- 它会在浏览器准备开始下一帧渲染时才执行你的函数。
- 所以你能安全地做 DOM 操作、修改样式，不会打断当前帧，避免重复渲染。
- 如果标签页被隐藏，`requestAnimationFrame` 还会自动暂停，节省资源。

Q7: Canvas, WebGL, SVG的区别

1. Canvas:

- **原理:** Canvas 是一个 HTML 元素，允许你通过 JavaScript 绘制图形。它本质上是一个画布，通过在其上绘制像素来创建图像。Canvas 提供了一系列的绘图 API（如 `2d` 和 `webgl`）。
- **适用场景:** 适用于动态绘制图形、游戏、数据可视化等场景。对于需要频繁更新的图像（如动画或交互式应用），Canvas 是一个理想的选择。
- **特点:**
 - 直接操作像素。
 - 绘制的内容不可修改，除非重新绘制。
 - 对于复杂的场景（如3D）使用 WebGL 模式来实现。

2. WebGL:

- **原理:** WebGL 是一个 JavaScript API，基于 OpenGL ES 2.0，允许在浏览器中渲染 3D 图形。它利用 GPU 来处理图形渲染，因此适合高性能的图形任务。
- **适用场景:** 适用于需要高性能图形渲染、3D 模型、游戏、虚拟现实（VR）等应用。
- **特点:**
 - 基于 GPU 渲染。
 - 支持 3D 图形和复杂的着色器效果。
 - 较为底层的接口，使用起来比 Canvas 更复杂，需要更多的数学和图形学知识。

3. SVG (可缩放矢量图形) :

- **原理:** SVG 是一种基于 XML 的文件格式，用于描述二维矢量图形。通过 XML 标记定义图形，支持动画和交互性。SVG 的图形是基于向量的，可以无限缩放而不失真。
- **适用场景:** 适用于需要高质量、分辨率独立的图形，如图标、图表、界面元素等。对于需要精确和可访问的图形（如绘制地图、流程图、图标等），SVG 是理想选择。
- **特点:**
 - 基于 XML，易于编辑和操作。
 - 可以通过 CSS 和 JavaScript 进行样式和动画控制。
 - 图形文件较大时性能较差，不适合动态复杂的场景。

总结对比:

特性	Canvas	WebGL	SVG
类型	位图绘制	3D 渲染（基于 GPU）	矢量图形
渲染方式	像素级操作	使用 GPU 渲染 3D 图形	矢量计算，不失真
性能	动态渲染高效，但处理复杂场景时较慢	高效，适合复杂的 3D 图形和大规模渲染	性能受限，复杂图形时可能变慢
操作	直接绘制像素	使用底层图形 API，如着色器，适合复杂 3D 渲染	使用 XML 描述，易于编辑和操作
适用场景	动态图形、游戏、数据可视化等	高性能 3D 图形、虚拟现实、游戏等	矢量图形、图标、地图、UI 元素等
交互性	通过 JavaScript 控制	通过 JavaScript 和 GLSL 编写着色器	可以通过 CSS、JavaScript 动态修改图形

总结起来：

- **Canvas** 更适合动态绘制位图图形，尤其是需要高性能图形更新时。
- **WebGL** 更适合需要高性能的 3D 图形渲染，利用 GPU 处理复杂的图形。
- **SVG** 更适合静态或动画矢量图形，特别适用于界面设计、图标、数据可视化等需要高质量缩放的场景。

Q8：JS事件的循环机制

JavaScript 的事件循环（Event Loop）是其处理异步代码、事件和回调函数的机制。它使得 JavaScript 能够在单线程的环境下执行异步操作，同时避免阻塞 UI 的渲染。事件循环的基本原理涉及多个队列（如任务队列）和栈（调用栈）。**每次执行之前都要检测微任务**

事件循环机制的主要流程：

1. **调用栈（Call Stack）**：
 - JavaScript 是单线程的，每当执行一个函数时，函数会被推入调用栈中。当函数执行完毕时，调用栈会将其弹出。
 - 任何**同步任务**（如普通函数调用）都会按照顺序进入调用栈并被依次执行。
2. **任务队列（Task Queue）**：
 - 异步任务（如 `setTimeout`、`setInterval`、事件监听器等）会被添加到任务队列中。任务队列中的任务会在调用栈为空时逐一执行。
 - 任务队列中的任务根据类型分为宏任务（Macro Task）和微任务（Micro Task）。
3. **宏任务和微任务**：
 - **宏任务（Macro Task）**：包括 `setTimeout`、`setInterval`、DOM 事件、I/O 操作等。它们会被加入到任务队列中。
 - **微任务（Micro Task）**：包括 Promise 的 `then`、`catch`、`finally` 回调、`MutationObserver` 等。微任务会在每个宏任务执行完之后执行，并且它们的优先级高于宏

任务。

4. 事件循环的执行顺序：

- **执行顺序：**事件循环机制的执行顺序通常如下：
 1. **执行栈：**首先执行栈中的同步代码。
 2. **微任务队列：**执行完栈中的同步代码后，事件循环会先检查并执行微任务队列中的所有任务（如果有的话）。
 3. **宏任务队列：**在微任务队列为空时，事件循环会从宏任务队列中取出一个任务执行（例如定时器的回调函数、DOM 事件等）。
 4. **渲染：**如果需要渲染（如更新 DOM、页面布局等），浏览器会在事件循环的这一阶段进行。
 5. **重复循环：**重复执行上述步骤。

事件循环的执行顺序图：

1. 执行同步代码（压入栈）；
2. 执行微任务（从微任务队列取出并执行）；
3. 执行宏任务（从宏任务队列取出并执行）；
4. 渲染（浏览器进行渲染更新）。

举例说明：

```
console.log('A'); // 同步任务
setTimeout(() => { // 宏任务
  console.log('B');
}, 0);
Promise.resolve().then(() => { // 微任务
  console.log('C');
});
console.log('D'); // 同步任务
```

执行顺序：

1. 执行同步代码：`console.log('A')` 输出 A。
2. 执行同步代码：`console.log('D')` 输出 D。
3. 微任务：`Promise.resolve().then()` 的回调函数会被加入到微任务队列，在同步代码执行完之后立即执行，因此输出 C。
4. 宏任务：`setTimeout` 的回调会被加入到宏任务队列中，微任务执行完毕后，宏任务才会执行，因此输出 B。

输出顺序：

```
A
D
C
B
```

总结：

- JavaScript 是单线程的，事件循环确保了异步代码不会阻塞 UI 渲染。
- 事件循环首先执行同步任务，然后执行微任务，最后才执行宏任务。
- 微任务的优先级高于宏任务，因此会在每次宏任务之前执行。

这种机制使得 JavaScript 可以高效地处理异步操作，避免阻塞用户界面，并提供良好的性能。

Q9：虚拟列表的底层原理？怎么实现虚拟列表滚动条精准定位？

当列表项的高度不固定时，虚拟列表的实现会更加复杂，因为每个列表项的高度都不同，因此需要动态计算每个列表项的高度，同时保证滚动条同步。为了实现这个目标，我们可以使用 `IntersectionObserver` 结合上下两个哨兵，并且通过动态计算每个列表项的高度来渲染虚拟列表。此时，除了渲染可见区域的列表项，我们还需要确保滚动条的准确性。

核心思路

1. 容器 `div` 和内容 `div`：
 - 容器 `div` 用于展示虚拟列表，并且具有固定的高度和滚动条。
 - 内容 `div` 用来存储所有的列表项，其总高度是所有项高度的总和，保证滚动条的正确显示。
2. 上下哨兵：
 - 上哨兵：位于列表顶部，用来监测用户滚动到顶部时，加载之前的列表项。
 - 下哨兵：位于列表底部，用来监测用户滚动到底部时，加载后续的列表项。
3. `IntersectionObserver`：
 - 通过 `IntersectionObserver` 观察上、下哨兵的进出，触发加载更多内容。
4. 动态计算高度：
 - 对于每个列表项，我们都需要记录它的实际高度。这可以通过一个数组来存储每个列表项的高度，以便在渲染时动态计算需要展示哪些项目。
5. 同步滚动条：
 - 为了保持虚拟列表与滚动条的同步，我们需要根据当前滚动的位置动态调整容器的滚动条位置。

步骤实现

1. 计算每个列表项的高度：我们使用一个数组来存储每个列表项的高度。
2. 上下哨兵加载：通过监听上下哨兵的进出，来触发前后加载的动作。
3. 更新容器的总高度：虚拟列表的总高度是每个项目的高度总和，用来更新滚动条的位置。

代码实现示例

```
<div id="container" style="overflow-y: auto; height: 300px; position: relative;">
  <div id="virtual-list" style="position: absolute; top: 0; left: 0; width: 100%;"></div>
</div>

<script>
  const container = document.getElementById('container');
  const virtualList = document.getElementById('virtual-list');

  const totalItems = 1000; // 列表总项数
  const visibleItems = Math.ceil(container.clientHeight / 50); // 计算可视区域内的项目数量
  const itemHeights = []; // 每个项目的高度

  // 随机生成项目高度（用于演示）
  for (let i = 0; i < totalItems; i++) {
    itemHeights.push(30 + Math.floor(Math.random() * 100)); // 高度范围：30px 到 130px
  }
}
```

```

let startIndex = 0;
let endIndex = visibleItems - 1;

// 设置虚拟列表总高度
function setVirtualListHeight() {
  const totalHeight = itemHeights.reduce((sum, height) => sum + height, 0);
  virtualList.style.height = totalHeight + 'px';
}

// 渲染当前可见项目
function renderItem() {
  const fragment = document.createDocumentFragment();
  let topOffset = 0;

  for (let i = startIndex; i <= endIndex; i++) {
    const div = document.createElement('div');
    div.style.height = itemHeights[i] + 'px'; // 设置每个项目的高度
    div.textContent = `Item ${i + 1}`;
    fragment.appendChild(div);
    topOffset += itemHeights[i]; // 计算当前项目的顶部偏移量
  }

  virtualList.innerHTML = ''; // 清空旧的内容
  virtualList.appendChild(fragment);

  // 设置虚拟列表的位置
  virtualList.style.top = topOffset + 'px';
}

// 上下哨兵
const topSentinel = document.createElement('div');
const bottomSentinel = document.createElement('div');
topSentinel.style.height = '1px';
bottomSentinel.style.height = '1px';
virtualList.appendChild(topSentinel);
virtualList.appendChild(bottomSentinel);

// 设置IntersectionObserver
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.target === topSentinel && entry.isIntersecting) {
      // 上哨兵进入视口，加载前面的项目
      startIndex = Math.max(0, startIndex - 10); // 向上加载10个项目
      endIndex = startIndex + visibleItems - 1;
      renderItem();
    } else if (entry.target === bottomSentinel && entry.isIntersecting) {
      // 下哨兵进入视口，加载后面的项目
      startIndex = Math.min(totalItems - visibleItems, endIndex + 1); // 向下加载10个项目
      endIndex = startIndex + visibleItems - 1;
      renderItem();
    }
  });
}, {
  root: container,
  rootMargin: '0px',
  threshold: 1.0
});

```

```
// 监听上、下哨兵
observer.observe(topSentinel);
observer.observe(bottomSentinel);

// 渲染初始项
setVirtualListHeight();
renderItems();

</script>
```

关键点说明

1. **动态高度**：通过 `itemHeights` 数组存储每个列表项的高度。每个项的高度是随机生成的，实际应用中可以根据数据计算高度。
2. **renderItems 函数**：根据 `startIndex` 和 `endIndex` 渲染当前可见的项目。每次渲染时都根据项目的高度调整偏移量。
3. **上下哨兵**：`topSentinel` 和 `bottomSentinel` 作为监控点，分别用于触发加载前面或后面的项目。通过 `IntersectionObserver` 来检测这些哨兵的进出，并动态调整 `startIndex` 和 `endIndex`。
4. **更新虚拟列表的高度**：`setVirtualListHeight` 函数计算所有项的总高度并更新 `virtualList` 的高度，以保证滚动条的比例和容器内容一致。
5. **IntersectionObserver 的作用**：通过观察上下哨兵与视口的交集，触发加载新项目。加载的项目会被动态添加到视图中，避免一次性渲染所有项目。

总结

通过使用 `IntersectionObserver` 和上下两个哨兵，我们可以高效地实现高度不固定的虚拟列表。每个列表项的高度是动态计算的，且通过渲染当前可见的项目来减少不必要的 DOM 操作。

`IntersectionObserver` 用于监听滚动进度，并触发加载更多项的操作，从而保持虚拟列表与滚动条的同步。这种方法不仅能大幅提高性能，还能使得大数据量的列表展示变得流畅。

Q10：如何用 Token 实现登录鉴权？

Token 登录鉴权 是一种通过生成并传递令牌（Token）来验证用户身份的方式。常见的实现方式包括使用 **JWT (JSON Web Token)** 来进行身份验证和授权。以下是实现步骤：

1. **用户登录**：
 - 用户通过用户名和密码进行登录。
 - 服务器验证用户名和密码的正确性。
2. **生成 Token**：
 - 一旦用户登录成功，服务器会生成一个 Token（通常是 JWT）作为认证凭证。Token 中包含用户的基本信息和有效期。
 - 生成的 Token 会经过加密签名，防止被篡改。
3. **返回 Token**：
 - 服务器将生成的 Token 返回给客户端。
 - 客户端将 Token 存储在本地（如 `localStorage` 或 `cookie` 中）。
4. **发送 Token**：
 - 每次客户端发送请求时，会将 Token 作为请求头的一部分（例如，`Authorization: Bearer <token>`）发送到服务器。
5. **验证 Token**：

- 服务器收到请求后，会解析 Token，验证其有效性。如果 Token 未过期且未被篡改，服务器认为用户是合法的，可以继续处理请求。

6. 过期和刷新 Token：

- 如果 Token 过期，用户需要重新登录或者通过刷新 Token 获取新的 Token。

优点：

- 无状态的认证方式，减少服务器存储负担。
- 适用于分布式系统和微服务架构。

代码示例：JWT 生成和验证 (Node.js)

```
// 使用 jsonwebtoken 包
const jwt = require('jsonwebtoken');

// 用户登录时生成 JWT
function generateToken(user) {
  const payload = { id: user.id, username: user.username };
  const secretKey = 'your-secret-key'; // 应该保密存储
  const options = { expiresIn: '1h' }; // 设置过期时间
  const token = jwt.sign(payload, secretKey, options);
  return token;
}

// 验证 JWT
function verifyToken(token) {
  try {
    const secretKey = 'your-secret-key';
    const decoded = jwt.verify(token, secretKey);
    return decoded;
  } catch (err) {
    console.error('Invalid Token');
    return null;
  }
}
```

Q11: CSRF 是什么？怎么解决？

CSRF (跨站请求伪造) 是一种攻击方式，攻击者诱导已登录用户访问恶意链接，借此发起未授权的请求。攻击者伪造用户的请求，导致用户不知情地执行操作（例如，修改密码、转账等）。

解决方法：

1. 使用 Token 防护：

- 在每个请求中使用 CSRF Token。这个 Token 是在用户每次请求时生成并且与会话绑定的，只有用户的会话中含有正确的 Token 才能执行操作。

2. 检查 Referer 和 Origin：

- 服务器可以检查请求中的 `Referer` 和 `Origin` 头，确保请求来自合法的来源。

3. 使用 SameSite Cookies：

- 通过设置 Cookie 的 `SameSite` 属性，限制跨站点请求中的 Cookie 发送行为。
`SameSite=Strict` 或 `SameSite=Lax` 可以有效防止 CSRF 攻击。

4. POST 请求防护：

- 对敏感操作（如账户修改、支付等），只允许通过 POST 请求来提交，避免 GET 请求的意外触发。

Q12: HTTPS 的加密过程

Q13: 闭包的定义？如何实现不同调用不被污染？

闭包 是 JavaScript 中的一个特性，它指的是一个函数可以“记住”并访问定义时的词法作用域，即使函数是在其词法作用域之外执行的。简而言之，闭包使得一个函数能够访问其外部函数的变量。

闭包的定义：

- 闭包是由函数和声明该函数的词法环境（即函数外部的变量）组成的。闭包让函数可以继续访问其定义时的环境。

如何实现不同调用不被污染？

闭包实现私有变量封装数据的底层逻辑 是 JavaScript 中一个非常重要的概念。它通过利用函数作用域链和词法作用域的特性，使得函数可以访问外部函数的变量，即使外部函数已经返回。这种机制使得变量可以被“封装”在闭包内部，不会被外部访问或修改，从而实现了私有变量的封装。

底层原理：

1. 词法作用域：

- 在 JavaScript 中，变量的作用域是由函数的定义位置决定的（即词法作用域），而不是调用位置。即使外部函数已经执行完成，内部函数依然可以访问外部函数的变量，因为这些变量会“保存在”作用域链中。

2. 闭包的作用域链：

- 每当一个函数被调用时，JavaScript 引擎会为它创建一个 **执行上下文**。该上下文包含：
 - **变量环境**：用于存储函数内部定义的变量。
 - **词法环境**：包含函数外部的变量和它所依赖的父级作用域的环境。
- 即使外部函数已经返回，内部函数依然可以通过其 **词法环境**（即它创建时的作用域链）访问外部函数的变量。

3. 私有变量的封装：

- 通过闭包，我们可以将变量放在外部函数的作用域内，而不是暴露给全局作用域。外部函数返回一个内部函数，这个内部函数通过闭包机制引用外部函数的变量，而外部函数的变量本身则无法直接访问。
- 只有通过内部函数，才能访问和操作外部函数中的“私有”变量。

下面我们通过一个例子来解释闭包是如何实现私有变量封装的。

```
function createCounter() {  
  let count = 0; // 私有变量  
  
  return {  
    increment: function() {  
      count++; // 访问并修改 count  
      console.log(count);  
    },  
    decrement: function() {  
      count--; // 访问并修改 count  
      console.log(count);  
    },  
    getCount: function() {
```

```
    return count; // 访问 count，但不能直接修改
  }
};
}

const counter = createCounter();
counter.increment(); // 输出 1
counter.increment(); // 输出 2
counter.decrement(); // 输出 1
console.log(counter.getCount()); // 输出 1
```

底层逻辑分解：

1. 创建执行上下文：

- 当调用 `createCounter` 函数时，JavaScript 引擎会为 `createCounter` 创建一个执行上下文。这个上下文包括 `count` 变量和返回的对象。即 `createCounter` 函数内部的 `count` 变量在该上下文中是私有的。

2. 闭包的创建：

- `createCounter` 返回的对象中包含了多个方法（`increment`、`decrement`、`getCount`），这些方法都可以访问到 `count` 变量。即使 `createCounter` 函数执行完毕，返回的对象依然保留对 `count` 变量的访问权限。

3. 作用域链：

- 返回的每个方法都形成一个 **闭包**。这些方法通过作用域链访问 `createCounter` 中的 `count` 变量。`count` 变量不是全局可访问的，而是通过闭包访问的，外部无法直接修改它。

4. 私有数据的保护：

- 通过这种方式，`count` 变量被“封装”在闭包内，并且不能被外部直接访问或修改。外部只能通过提供的 `increment`、`decrement`、`getCount` 方法来间接访问和修改它。这样就实现了数据的封装和私有性。

底层机制解析：

- 作用域链的创建：**每个闭包（比如 `increment` 和 `decrement` 函数）都会持有对其**创建时**所在环境的引用，即它们会形成一个作用域链，其中包含了对外部 `createCounter` 函数环境的引用。这个环境包含 `count` 变量，因此即使 `createCounter` 函数已经执行完毕，闭包依然可以通过作用域链访问和操作 `count` 变量。
- 内存管理：**闭包会保持对外部变量的引用，因此只要闭包仍然存在（即返回的对象被引用），外部变量（如 `count`）也不会被垃圾回收机制销毁。这是闭包的一个特点，它延长了外部变量的生命周期，直到闭包本身不再使用。

总结：

- 私有变量封装：**闭包通过将变量放在外部函数作用域内，并返回一个能够访问这些变量的内部函数来实现私有数据的封装。
- 作用域链：**闭包会保存对外部作用域的引用，使得内部函数能够访问外部函数的变量，从而实现数据的封装和访问控制。
- 内存管理：**闭包会延长外部变量的生命周期，直到闭包本身不再使用。

闭包是 JavaScript 中一个非常强大的特性，它为函数提供了持久化的外部数据访问能力，并通过作用域链来实现私有数据的封装，从而使得 JavaScript 更加灵活和强大。

2. 笔试题

Q1: 版本号对比

```
// 定义一个函数来比较两个版本号
var compareVersion = function(ver1, ver2) {
  // 将版本号按 '.' 分割为数组
  const v1 = ver1.split('.').map(Number);
  const v2 = ver2.split('.').map(Number);

  // 比较两个版本号的每一段
  const maxLength = Math.max(v1.length, v2.length);
  for (let i = 0; i < maxLength; i++) {
    const num1 = v1[i] || 0; // 如果某版本号段不存在, 补零
    const num2 = v2[i] || 0; // 如果某版本号段不存在, 补零

    if (num1 < num2) return -1;
    if (num1 > num2) return 1;
  }

  // 如果所有段都相等, 返回 0
  return 0;
};

// 接收输入, 无需更改
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

rl.question('', (input) => {
  const arr = input.split(',');
  console.log(compareVersion(arr[0], arr[1]));
  rl.close();
});
```

Q2: 最长递增连续子序列的长度

```
function lengthOfLIS(nums) {
  // 如果数组为空, 返回0
  if (nums.length === 0) return 0;

  // dp[i] 表示以 nums[i] 结尾的最长递增子序列的长度
  const dp = new Array(nums.length).fill(1);

  for (let i = 1; i < nums.length; i++) {
    for (let j = 0; j < i; j++) {
      if (nums[i] > nums[j]) {
        dp[i] = Math.max(dp[i], dp[j] + 1);
      }
    }
  }

  // 返回 dp 数组中的最大值
  return Math.max(...dp);
}
```

```
// 测试代码框架
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

rl.question('', (data) => {
  const nums = JSON.parse(data); // 解析输入的数组
  console.log(lengthOfLIS(nums)); // 输出结果
  rl.close();
});
```

Q3: 输出最长递增连续子序列的序列

```
function lengthOfLISwithSequence(nums) {
  // 如果数组为空，返回0和空数组
  if (nums.length === 0) return { length: 0, sequence: [] };

  // dp[i] 表示以 nums[i] 结尾的最长递增子序列的长度
  const dp = new Array(nums.length).fill(1);
  // prev[i] 用于记录 nums[i] 在最长递增子序列中的前一个元素的索引
  const prev = new Array(nums.length).fill(-1);

  for (let i = 1; i < nums.length; i++) {
    for (let j = 0; j < i; j++) {
      if (nums[i] > nums[j] && dp[i] < dp[j] + 1) {
        dp[i] = dp[j] + 1;
        prev[i] = j; // 记录前一个元素的索引
      }
    }
  }

  // 找到最长递增子序列的长度和最后一个元素的索引
  let maxLength = 0;
  let lastIndex = -1;
  for (let i = 0; i < dp.length; i++) {
    if (dp[i] > maxLength) {
      maxLength = dp[i];
      lastIndex = i;
    }
  }

  // 回溯 prev 数组找到完整的子序列
  const sequence = [];
  while (lastIndex !== -1) {
    sequence.unshift(nums[lastIndex]); // 将元素添加到子序列的开头
    lastIndex = prev[lastIndex]; // 移动到前一个元素
  }

  return { length: maxLength, sequence };
}

// 测试示例
const nums = [10, 9, 2, 5, 3, 7, 101, 18];
const result = lengthOfLISwithSequence(nums);
console.log("最长递增子序列的长度:", result.length);
```

```
console.log("最长递增子序列:", result.sequence);
```

4.9 饿了么面试

Q1：为什么选择 Vue3 作为技术框架？

1. 性能更优：

- Vue3 使用 Proxy 实现响应式，替代 Vue2 的 `Object.defineProperty`，性能提升显著。
- 支持编译时优化（如 tree-shaking、更小的打包体积）。

2. 组合式 API（Composition API）：

- 更好地组织逻辑复用，解决 Vue2 中 mixin 命名冲突、逻辑分散问题。
- 更适合大型项目、TS 支持更完善。

3. TypeScript 支持更完善：

- Vue3 从设计上就全面支持 TypeScript，更容易集成类型检查和 IDE 提示。

4. 生态逐步迁移，未来可期：

- Vue 官方库（如 Vue Router、Vuex → Pinia）都已优先支持 Vue3。
- 新项目使用 Vue3 能减少未来的技术债。

Q2：从 Vue 迁移到 React 怎么做效率更高？

策略性迁移方法：

1. 组件分层迁移：

- 从页面级、业务无关组件开始迁移（如按钮、弹窗等通用组件）。
- 后面逐步替换业务模块，避免一次性大改。

2. 使用微前端架构（如 qiankun）：

- 在大项目中将 Vue 和 React 作为子应用共存，逐步替换。

3. 封装中间层（adapter）：

- 通过接口适配、工具函数复用，使迁移成本降低（如 axios、权限、国际化等公共模块保持一致）。

4. 编码风格对比学习：

- Vue 的指令式模板 → React 的 JSX；
- Vue 的响应式系统 → React 的 Hooks；
- Vue 的事件修饰符、指令系统 → React 的原生事件绑定。

Q3：页面（视频）出现卡顿但无报错，怎么排查？

排查步骤：

1. 性能分析（Chrome DevTools → Performance）：

- 录制 Timeline 查看：
 - 哪段时间主线程占用高；
 - 是否存在大量 Layout、Recalculate Style；
 - JavaScript blocking；
 - 长任务（Long Task）是否频繁出现。

2. 视频播放排查：

- 使用 Media 面板观察是否丢帧、解码缓慢；
- 检查 video 标签的解码类型（软解/硬解）；

- 是否频繁触发 reflow（如 resize、滚动监听等）。
3. 查看资源占用：
 - 是否有大图、动画、Canvas 绘制影响帧率；
 - 是否存在大量 setTimeout/setInterval 影响主线程。
 4. 使用 `requestIdleCallback` 或 `requestAnimationFrame` 优化任务调度。

Q4：一个页面有6个图表，其中一个图表 a 需要权限，怎么设计？

建议设计方案：

```
<template>
  <ChartA v-if="hasPermission('chart:a')" />
  <ChartB />
  <ChartC />
  ...
</template>
```

权限判断方法：

1. 基于权限列表判断：

```
function hasPermission(code) {
  return userPermissions.includes(code);
}
```

1. 权限控制封装为指令或组件（提高复用性）：

```
<Permission code="chart:a">
  <ChartA />
</Permission>
// Permission.vue
defineProps(['code']);
const has = checkPermission(code);
return has ? slots.default() : null;
```

Q5：Vue3 拆分组件的标准？组件通信怎么做？

拆分标准：

1. 关注点分离（职责单一）：
 - 一个组件只负责一个功能（例如：图表组件、列表组件、表单组件分离）。
2. 可复用性：
 - 通用组件（如按钮、弹窗）vs 业务组件（如用户卡片、订单列表）。
3. 大小控制：
 - 控制每个组件在 300~500 行以内，避免复杂逻辑耦合。
4. 视图 + 逻辑分离（组合式 API 推荐）：
 - 使用 `usexxx.ts` 文件封装逻辑，比如 `useUserList()`、`useChartData()`。

组件通信方式：

场景	方法	说明
父传子	props	标准方式，单向数据流
子传父	emit	\$emit('eventName', data)
兄弟组件通信	父组件作为中介	或使用状态管理（如 Pinia）
跨层级通信	provide / inject	Vue3 Composition API 推荐
全局共享状态	Pinia（代替 Vuex）	更轻量，组合式风格更好用
非响应式通信（如通知）	EventBus、自定义 hooks	仅适用于解耦通信，需谨慎使用

4.15 京东

Q1：ts的类型，Partial和必选

Partial<T> 会把类型 T 的所有属性变成 可选 (optional) 的。

 基本语法：

```
Partial<T>
```

- 输入：一个类型 T
- 输出：这个类型的所有属性都变成了可选

 举个例子：

```
interface User {
  name: string;
  age: number;
}

// 普通 User 类型，必须传 name 和 age
const u1: User = {
  name: "小明",
  age: 18
}

// Partial<User>, name 和 age 都可以不写
const u2: Partial<User> = {
  name: "小红"
} //  age 可以省略
```

相当于：

```
type PartialUser = {
  name?: string;
  age?: number;
}
```

💡 使用场景

✓ 1. 更新数据时可选属性

```
function updateUser(id: number, userInfo: Partial<User>) {  
  // 只传需要修改的部分  
}
```

你只改 name，不必传完整的 `User` 对象：

```
updateUser(1, { name: "小刚" });
```

🔍 实现原理（源码）：

```
type Partial<T> = {  
  [P in keyof T]?: T[P];  
}
```

这是 TypeScript 的 内置映射类型 写法：

- 遍历 T 的每个属性 P；
- `?` 表示每个属性都变成可选；
- 值类型不变。

🧠 和其他工具类型一起记忆：

工具类型	含义
<code>Partial<T></code>	所有属性变可选
<code>Required<T></code>	所有属性变必填
<code>Readonly<T></code>	所有属性变只读
<code>Pick<T, K></code>	从 T 中挑选一组属性
<code>Omit<T, K></code>	从 T 中排除一组属性
<code>Record<K, T></code>	构造一个对象类型，键为 K，值为 T

4.16 腾讯二面

1. 面试题

Q1: echarts渲染百万数据点怎么优化？

Q2: 地图可以设置缓存吗？

Q3：设置验证码的作用是什么？

✓ 1. 防止恶意机器人攻击

- 比如注册账号、登录暴力破解、刷票、刷评论、刷秒杀等操作。
- 图形验证码通过“人类识别难度小，机器识别难度大”的设计，阻止自动脚本重复访问系统。

✓ 2. 提高系统安全性

- 验证码可以防止黑客批量尝试用户名密码组合，保护用户数据。
- 在登录、注册、找回密码等敏感操作中设置验证码，可提升系统抗攻击能力。

✓ 3. 降低服务器压力

- 防止恶意爬虫刷请求、导致服务器资源被浪费甚至崩溃。
- 有效识别出非正常用户行为，减少服务器资源的无效消耗。

✓ 4. 防止恶意提交表单

- 如留言板、论坛、注册页，如果没有验证码，攻击者可以通过脚本大量灌水、发垃圾广告。

✓ 常见验证码类型

验证码类型	特点
图形验证码	常见，用户识别字符
滑动验证码	用户拖动滑块或拼图验证
点选验证码	点击指定图片/位置验证
数学题验证码	要用户输入算式结果
音频验证码	为视障用户设计，用听的识别

Q4：未来的职业规划

Q5：期待的工作城市

2.代码题

Q1：将k个链表合并，按照升序排列，输出数组；如果输入的是数组转换为链表。

```
// 定义链表节点
class ListNode {
    constructor(val, next = null) {
        this.val = val;    // 当前节点的值
        this.next = next;  // 指向下一个节点的指针
    }
}

// 数组转链表
function arrayToList(arr) {
    let dummy = new ListNode(0); // 创建虚拟头节点，方便操作
    let current = dummy;        // 指针用于遍历和构建链表
```

```

    for (let num of arr) { // 遍历数组
      current.next = new ListNode(num); // 创建新节点并连接
      current = current.next; // 移动指针到新节点
    }
    return dummy.next; // 返回链表的实际头节点
  }

  // 合并k个升序链表
  function mergeKLists(lists) {
    // 用最小堆思想，这里用数组+sort模拟
    let heap = [];
    for (let node of lists) { // 遍历每个链表的头节点
      if (node) heap.push(node); // 非空则加入堆
    }
    heap.sort((a, b) => a.val - b.val); // 按节点值排序，最小值在前

    let dummy = new ListNode(0); // 创建虚拟头节点
    let current = dummy; // 指针用于构建结果链表
    while (heap.length > 0) { // 堆不为空时
      let node = heap.shift(); // 弹出堆顶（最小值节点）
      current.next = node; // 连接到结果链表
      current = current.next; // 指针后移
      if (node.next) {
        heap.push(node.next); // 将下一个节点加入堆
        heap.sort((a, b) => a.val - b.val); // 重新排序堆
      }
    }
    return dummy.next; // 返回合并后的链表头节点
  }

  // 链表转数组
  function listToArray(head) {
    const res = [];
    while (head) { // 遍历链表
      res.push(head.val); // 收集每个节点的值
      head = head.next; // 移动到下一个节点
    }
    return res; // 返回结果数组
  }

```

4.24字节面试

1.面试题

Q1: JS 的类的继承方法？ES6 的继承实现？

ES5 实现继承方式：

- 原型链继承
- 借用构造函数继承（经典继承）
- 组合继承
- 寄生组合继承（最优）

ES6 实现继承：

- 使用 `class` 和 `extends` 关键字实现继承：

```

class Parent {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}`);
  }
}

class Child extends Parent {
  constructor(name, age) {
    super(name); // 调用父类构造函数
    this.age = age;
  }
}

```

Q2: JS 的异步操作和异步方法

异步操作:

- 定时器 (`setTimeout`、`setInterval`)
- 网络请求 (`fetch`、`XMLHttpRequest`)
- 事件监听 (如点击事件)
- 文件读写 (Node.js 中)

异步方法:

- 回调函数 (callback)
- Promise
- `async/await` (基于 Promise 的语法糖)
- 事件监听 (EventEmitter)

```

// async/await 示例
async function fetchData() {
  try {
    const res = await fetch('/api');
    const data = await res.json();
    console.log(data);
  } catch (e) {
    console.error(e);
  }
}

```

Q3: JS 的模块化有哪些? ES6 和 CommonJS 的区别?

- 模块化方式: CommonJS、AMD、CMD、ES6 Modules
- 区别见下表:

特性	CommonJS	ES6 Modules
加载方式	运行时	编译时（静态）
语法	<code>require</code> / <code>exports</code>	<code>import</code> / <code>export</code>
是否异步	否	是（支持异步）
是否支持 Tree Shaking	否	是

Q4：说一下 Flex 布局，Flex 和 Grid 的区别

Flex 布局： 一维布局，适用于元素的横向或纵向排列。

- 常用属性：`flex-direction`, `justify-content`, `align-items`, `flex-wrap`

Grid 布局： 二维网格系统，更适合复杂页面布局。

- 常用属性：`grid-template-rows/columns`, `grid-area`, `grid-gap`

比较点	Flex	Grid
维度	一维（横或纵）	二维（横+纵）
控制单元	项目间对齐	明确区域划分
场景	内容驱动型布局	结构驱动型布局

Q5：position 的属性值？sticky 是什么？

- `static`：默认值，不受 top/left 等影响
- `relative`：相对原位置偏移
- `absolute`：相对最近的定位父元素偏移
- `fixed`：相对视口定位，页面滚动不变
- `sticky`：根据滚动位置在 `relative` 和 `fixed` 之间切换（适用于标题悬停等）

注意：`sticky` 父元素不能有 `overflow: hidden`

Q6：代码分割了解过吗？

代码分割（Code Splitting）： 将一个完整打包的大文件（bundle）拆分成多个小模块，并根据需要按需加载（懒加载），以提升页面加载速度和性能。

✓ 为什么要代码分割？

- **减少首屏加载时间：** 只加载当前页面需要的代码。
- **按需加载：** 用户访问某个页面或点击某个按钮时，才加载相关代码。
- **提升用户体验：** 减少资源浪费，加快交互响应速度。

🔧 常见的代码分割方式：

1. Webpack 动态 `import()`

```
// 普通方式（全部打包）
import MyComponent from './MyComponent';

// 动态加载（代码分割）
const MyComponent = () => import('./MyComponent');
```

2. Vue 路由懒加载

```
const Home = () => import('./views/Home.vue');
```

常见的打包工具支持代码分割：

- Webpack（主流）
- Vite（更快、更轻）
- Parcel、Rollup（也支持）

优化点：

- 和 Tree Shaking 配合，去掉未使用代码。
- 配合 HTTP2，多文件可并发加载。

Q7：优化方式？性能指标？查看工具？

性能优化方式：

- 静态资源压缩与缓存（gzip、CDN）
- 图片懒加载、合并资源、SVG 使用
- Tree Shaking、按需加载、虚拟列表
- DOM 减少重排重绘

性能指标（Core Web Vitals）：

- FCP（First Contentful Paint）
- LCP（Largest Contentful Paint）
- CLS（Cumulative Layout Shift）
- TTI（Time to Interactive）
- FID（First Input Delay）

查看工具：

- Chrome DevTools > Performance
- Lighthouse 报告
- Web Vitals 插件
- Pagespeed Insights

Q8：进程与线程的区别？

区别点	进程 (Process)	线程 (Thread)
概念	资源分配的最小单位	程序执行的最小单位
是否独立	彼此独立	同一进程下共享资源
开销	创建、切换开销大	开销小
通信方式	IPC, 如管道/Socket	内存共享
异常影响	一个进程崩溃不影响其他	一个线程崩溃可能影响整个进程

2.笔试题

Q1：写一个函数判断是不是素数？如何判断[n,m]区间的数是不是素数？

```
function isPrime(num) {
  if (num < 2) {
    return false;
  }
  for (let i = 2; i <= Math.sqrt(num); i++) {
    if (num % i === 0) {
      return false;
    }
  }
  return true;
}

function primesInRange(n, m) {
  const result = [];
  for (let num = n; num <= m; num++) {
    if (isPrime(num)) {
      result.push(num);
    }
  }
  return result;
}
```

Q2：写一个函数实现数组的扁平化

```
function flattenArray(arr) {
  const result = [];
  arr.forEach(item => {
    if (Array.isArray(item)) {
      result.push(...flattenArray(item));
    } else {
      result.push(item);
    }
  });
  return result;
}
```

Q3: 写一个函数合并两个有序数组

```
function mergeSortedArrays(arr1, arr2) {  
  const result = [];  
  let i = 0;  
  let j = 0;  
  while (i < arr1.length && j < arr2.length) {  
    if (arr1[i] < arr2[j]) {  
      result.push(arr1[i]);  
      i++;  
    } else {  
      result.push(arr2[j]);  
      j++;  
    }  
  }  
  while (i < arr1.length) {  
    result.push(arr1[i]);  
    i++;  
  }  
  while (j < arr2.length) {  
    result.push(arr2[j]);  
    j++;  
  }  
  return result;  
}
```

4.24腾讯面试

1.面试题

Q1: 进程与线程的区别?

Q2: http可以使用udp连接吗?

Q3: https的加密过程? 证书包含什么?

HTTPS 证书（即 SSL/TLS 证书）是用来加密 HTTP 数据传输的一种**数字证书**，确保数据安全、验证网站身份。

☑ HTTPS 证书包含的内容:

内容	说明
证书持有者信息	网站域名、公司名称、组织单位、所在地（省/市/国家）等
公钥 (Public Key)	用于加密数据，浏览器用它加密信息发给服务器
签名算法信息	CA 使用的加密算法（如 SHA-256）
颁发者信息	CA（证书颁发机构）的名称、地址、签名等
有效期	证书的起始时间和到期时间
证书序列号	每张证书唯一标识编号
使用目的	指明证书的用途（如服务器身份验证、数据加密）
版本号	通常是 X.509 v3
扩展字段（可选）	比如 SAN（Subject Alternative Name）用于支持多个域名等

浏览器验证流程简略：

1. 浏览器发起 HTTPS 请求，服务端返回证书。
2. 浏览器验证证书是否可信（CA 是否可信，是否过期、是否吊销）。
3. 浏览器提取公钥，与服务器协商对称密钥进行加密通信。

查看证书的方法：

- 浏览器中点击地址栏  图标 → 查看证书详细信息
- 命令行查看：

```
openssl s_client -connect example.com:443
```

Q4：编译器做什么工作（以 V8 为例）

虽然 JavaScript 不是传统意义上的“编译型语言”，但现代 JS 引擎使用了解释 + 编译混合模式（JIT），也就是说，JavaScript 是“先编译再执行”的语言。

✓ 1. 词法分析 (Lexical Analysis)

- 把源代码变成 **Token 流**（词法单元）
- 比如：

```
const a = 10;
```

会被拆分成：["const", "a", "=", "10", ";"]

✓ 2. 语法分析 (Parsing)

- 把 Token 流转成 **AST（抽象语法树）**
- AST 是一个结构化的、可编程的表示方式，方便后续分析、转换

例如：

```
const a = 10;
```

的 AST 类似于：

```
{
  "type": "VariableDeclaration",
  "kind": "const",
  "declarations": [
    {
      "id": { "type": "Identifier", "name": "a" },
      "init": { "type": "Literal", "value": 10 }
    }
  ]
}
```

✓ 3. 生成字节码 (Bytecode)

- V8 引擎中，**解释器 (Ignition)** 会把 AST 转换为一种中间格式：**字节码**
- 这是比源代码更接近机器但还不是机器码的一种表示

✓ 4. 执行与热点收集

- 字节码开始执行，同时 V8 会**收集运行信息**（比如哪些函数被频繁调用）
- 热门函数会被标记为**热点代码 (hot path)**

✓ 5. 优化编译 (JIT 编译)

- 热点代码会被交给 V8 的优化编译器 (TurboFan)，生成**更高性能的机器码**
- 提高运行效率！

🧐 举个例子：浏览器如何执行 JS

假设你在浏览器控制台输入：

```
function add(a, b) {
  return a + b;
}
```

浏览器做的事情：

1. 词法分析 → `function`、`add`、`(`、`a`、`,`...等 token
2. 语法分析 → 构建 AST（函数声明节点 + return 表达式）
3. 编译成字节码 → `add` 函数被转成内部字节码
4. 运行时监控 → 如果你频繁调用 `add(1, 2)`，引擎会优化它
5. 热点优化 → 转成机器码，运行更快！

📝 总结：JS 编译器的完整工作流程

阶段	内容说明
词法分析	源码 → Token 流
语法分析	Token → AST
字节码生成	AST → 字节码 (Ignition)
字节码执行	执行字节码, 收集热点数据
JIT 优化	热点代码 → 优化编译 → 机器码 (TurboFan)
回退 (Deopt)	当假设失败, 可能回退到字节码执行

Q5: TypeScript 编译成 JavaScript 的过程

TypeScript → JavaScript 的过程是一个“类型擦除式编译”:

编译过程 (由 `tsc` 或构建工具完成)

- 语法检查 (Type Check)**
根据 `.d.ts` 声明文件、类型推断, 检查是否有类型错误。
- 类型擦除 (Type Erasure)**
所有类型信息在编译后都会被移除, 不影响最终运行。
- 转成 JS 代码 (Emit)**
依据 `tsconfig.json` 设置输出 JS (ES5、ES6...)

```
const add = (a: number, b: number): number => a + b;  
// 编译后:  
var add = function (a, b) { return a + b; };
```

- 输出 sourcemap (可选)**
用于调试时映射 TS → JS 行号。

总结一下:

步骤	JS 编译器如 Babel/V8	TS 编译器 (tsc)
语法分析	✓	✓
类型检查	✗ (JS 无类型)	✓
转换代码	✓ (如 ES6 → ES5)	✓ (TS → JS)
移除类型信息	无需	✓
输出中间产物	可选 (Babel plugins / 字节码)	✓ (如 sourcemap、声明文件)

Q6: 对大模型的看法? 遇到过回答问题不对的情况吗?

Q7: 项目中做了哪些工作? 哪些优化?

Q7: 团队之间发生冲突怎么解决?

1. 明确需求和职责

- **需求文档**: 确保前后端团队在项目开始时达成共识, 明确需求, 统一规范, 避免因理解不同导致冲突。例如, 前端和后端应该在接口设计、数据格式、返回值等方面进行充分的沟通。
- **职责划分**: 清晰划分前后端职责, 避免重叠。前端负责界面和用户交互, 后端负责数据处理、存储和业务逻辑。

2. 早期沟通和定期协作

- **跨部门会议**: 定期举行前后端会议, 进行需求评审、接口设计讨论和开发进度检查。通过讨论, 确保各方理解一致, 避免偏差。
- **文档共享**: 共享技术文档和接口文档, 确保前后端团队有共同的参考基础, 减少误解和沟通成本。
- **设计评审**: 前后端在功能设计和技术实现时可以共同参与, 提前评审, 以避免后期开发过程中出现偏差。

3. 使用接口文档和工具

- **API 设计和接口文档**: 前后端团队可以使用 **Swagger** 或 **Postman** 等工具来定义和共享接口, 确保双方对接口的返回数据结构、请求方式、参数等一致。
- **Mock 服务**: 前端在后端 API 未完成时, 可以使用 Mock 数据进行开发, 避免前端开发进度滞后于后端。

4. 版本控制和协同开发

- **Git Flow 工作流**: 使用版本控制工具 (如 Git) 时, 保持规范的分支管理和合并流程 (如 Git Flow), 确保代码的独立性和协作效率。
- **CI/CD 流程**: 建立自动化构建和部署流程, 确保前后端的代码能够顺利集成和发布, 避免因集成失败而导致的冲突。

5. 及时反馈与问题跟踪

- **问题跟踪工具**: 使用 Jira、Trello 等工具来管理任务和问题, 明确优先级, 避免工作重复和遗漏。
- **定期回顾**: 在开发周期结束后, 进行项目回顾, 找出冲突发生的根本原因, 并提出解决方案, 以避免类似问题的再次发生。

6. 容错与灵活应对

- **调试工具**: 在前后端开发过程中, 如果出现接口数据错误或无法对接的情况, 可以使用 **Postman** 或 **Fiddler** 等工具进行调试, 快速找出问题所在。
- **兼容方案**: 前端可以在接口不稳定时, 采用默认值或回退方案, 保证前端展示不受影响; 后端也可以通过兼容前端的请求参数或响应格式, 确保两方接口能顺利对接。

7. 建立相互信任和支持的文化

- **尊重和理解**: 前后端团队应该互相尊重对方的工作, 理解对方面临的挑战, 减少敌对情绪。
- **共享目标**: 确保团队都有共同的项目目标, 并且关注整个项目的成功, 而不仅仅是各自的部分。

8. 冲突解决策略

- **明确优先级和紧急性**：有些冲突可能是由于某些任务或问题的优先级不清楚导致的。通过与产品经理、项目经理协商，明确哪些任务是最优先解决的。
- **妥协和折中**：在某些情况下，前后端之间的需求可能难以完全达成一致，可以通过折中方案解决冲突。例如，在接口数据格式上进行一些妥协，既满足前端需求又不破坏后端的设计。
- **调解人员**：如果冲突无法通过团队内部解决，可以引入项目经理或技术负责人作为调解者，帮助团队找到最佳解决方案。

9. 技术培训和知识共享

- **跨领域学习**：鼓励前端开发者了解一些后端知识，反之亦然。通过团队内部的知识分享，增加相互理解，从而减少冲突。
- **文档化代码和设计**：前后端开发者在工作中，应该遵循统一的编码规范和文档化流程，这样可以确保代码更加清晰，减少沟通成本。

2. 笔试题

Q1: [79. 单词搜索](#)

给定一个 $m \times n$ 二维字符网格 `board` 和一个字符串单词 `word`。如果 `word` 存在于网格中，返回 `true`；否则，返回 `false`。

单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

```
var exist = function(board, word) {
    const m = board.length, n = board[0].length;
    function dfs(i, j, k) {
        if (board[i][j] !== word[k]) {
            return false; // 匹配失败
        }
        if (k + 1 === word.length) {
            return true; // 匹配成功!
        }
        board[i][j] = 0; // 标记访问过
        for (const [x, y] of [[i, j - 1], [i, j + 1], [i - 1, j], [i + 1, j]]) {
            // 相邻格子
            if (0 <= x && x < m && 0 <= y && y < n && dfs(x, y, k + 1)) {
                return true; // 搜到了!
            }
        }
        board[i][j] = word[k]; // 恢复现场
        return false; // 没搜到
    }
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            if (dfs(i, j, 0)) {
                return true; // 搜到了!
            }
        }
    }
    return false; // 没搜到
};
```

Q2: [746. 使用最小花费爬楼梯](#)

给你一个整数数组 `cost`，其中 `cost[i]` 是从楼梯第 `i` 个台阶向上爬需要支付的费用。一旦你支付此费用，即可选择向上爬一个或者两个台阶。

你可以选择从下标为 `0` 或下标为 `1` 的台阶开始爬楼梯。

请你计算并返回达到楼梯顶部的最低花费。

```
// 会超时的递归代码
var minCostClimbingStairs = function(cost) {
  const n = cost.length;
  function dfs(i) {
    if (i <= 1) { // 递归边界
      return 0;
    }
    return Math.min(dfs(i - 1) + cost[i - 1], dfs(i - 2) + cost[i - 2]);
  }
  return dfs(n);
};
```

4.27腾讯二面

1.面试题

Q1:

Q2:

2.笔试题

Q1:

Q2:

4.28字节二面

1.面试题

Q1:

Q2:

2.笔试题

Q1:

Q2:

4.2xx面试

1.面试题

Q1:

Q2:

2.笔试题

Q1:

Q2: